

Rheinisch-Westfälische Technische Hochschule Aachen
Informatik 5, Information Systems
Prof. Dr. Stefan Decker

BACHELOR THESIS

Privacy-Preserving Federated Graph Learning for AML

Till Stinner

September 22, 2026

1 st Advisor	Felix Hermsen, M.Sc.
1 st Supervisor	Prof. Dr. Stefan Decker
2 nd Supervisor	Prof. Dr. Ulrike Meyer

Abstract

Money laundering detection can benefit from analyzing relationships between accounts, as suspicious patterns may become apparent only when transactions are considered together. Transaction graphs capture these relationships, enabling graph convolutional networks (GCNs) to learn account representations by combining their attributes with information propagated from neighbors. When laundering patterns span institutions, a single bank’s graph can omit relevant relational context, motivating collaborative analysis. Conventional federated learning enables shared training without pooling raw records, but model aggregation alone leaves graph propagation confined to each bank’s local graph. To recover this missing relational context, we realize FedGCN’s one-shot feature pre-aggregation as a role-bounded protocol that forms contributions from bank-owned data and delivers only the aggregated features each bank requires. We compare centralized learning, cut-edge federation, and one-/two-hop pre-aggregation on synthetic three-bank graphs under symmetric and asymmetric client configurations.

On the symmetric dataset, area under the precision-recall curve falls from 0.8017 centrally to 0.7194 with cut-edge federation, before two-hop pre-aggregation recovers it to 0.7599. Both pre-aggregation depths outperform cut-edge federation across all datasets, with two-hop leading under client symmetry and one-hop in both asymmetric settings.

We further evaluate the privacy–utility–efficiency trade-offs of protecting information exchanged during feature pre-aggregation and federated model training. Homomorphic encryption and pairwise masking preserve predictive utility, although encryption increases pre-training traffic roughly fortyfold; differential privacy, by contrast, severely degrades utility. Overall, cross-bank graph information improves federated GCN utility in this synthetic setting, but its recovery and protection create distinct trade-offs in predictive performance, information exposure, communication, and memory.

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	3
1.3 Research Questions	4
1.4 Contributions	4
1.5 Thesis Structure	6
2 Background	7
2.1 Anti-Money Laundering	7
2.2 Transaction Graphs	7
2.3 Graph Neural Networks	9
2.3.1 Message Passing and Node Representations	9
2.3.2 Graph Convolutional Networks	11
2.4 Federated Learning	12
2.5 Federated Graph Learning	14
2.6 Privacy Enhancements	16
2.6.1 Homomorphic Encryption	16
2.6.2 Differential Privacy	17
2.6.3 Secure Aggregation	18
3 Related Work	20
3.1 Synthetic AML Data and Evaluation Benchmarks	20
3.2 Graph-Based Anti-Money Laundering	21
3.3 Federated Graph Learning	22
3.3.1 Cross-Client Graph Information	22
3.3.2 Privacy-Preserving Federated Graph Learning	23
3.4 Collaborative and Federated Anti-Money Laundering	25
3.5 Research Gap	26
4 Methodology	28
4.1 Synthetic Multi-Bank Data Generation	28
4.1.1 AMLSim Pipeline and Generator Adaptations	30
4.1.2 Dataset Validation and Acceptance	33
4.2 Graph Construction and Feature Representation	35

4.2.1	Nodes, Directed Account-Pair Edges, and Labels	35
4.2.2	Node Features	36
4.2.3	Data Splits and Feature Scaling	37
4.3	Centralized Graph-Learning Model	38
4.4	Federated Graph Learning	40
4.4.1	Federated Setting and Cut-Edge Baseline	40
4.4.2	Cross-Client Feature Pre-Aggregation	41
4.4.3	One-Hop and Two-Hop Graph Context	46
4.5	Privacy Enhancements	48
4.5.1	Threat Model and Protection Surfaces	48
4.5.2	Recipient-Routed CKKS Aggregation	51
4.5.3	Contribution-Row Differential Privacy	53
4.5.4	Pairwise-Masked Federated Averaging	55
4.6	Evaluation Methodology	57
4.6.1	Comparative Research Design	58
4.6.2	Utility Metrics and Model Selection	58
4.6.3	Privacy and Information-Exposure Assessment	59
4.6.4	Memory and Communication Accounting	60
5	Experimental Evaluation	62
5.1	Experimental Setup	62
5.1.1	Primary Symmetric Three-Bank Dataset	62
5.1.2	Asymmetric Dataset Variants	63
5.1.3	Common Experimental Configuration	65
5.2	Centralized Reference Performance	65
5.2.1	Model Calibration	65
5.2.2	Retained Centralized References	67
5.3	Federated Utility and Cross-Bank Graph Information	68
5.3.1	Federated Configuration Calibration	68
5.3.2	Cut-Edge Federated Utility	70
5.3.3	Federated Learning With Cross-Bank Information	71
5.4	Evaluation of Privacy Enhancements	74
5.4.1	Boundary-Compliant Features	75
5.4.2	CKKS Transport and Pairwise-Masked FedAvg	75
5.4.3	Contribution-Row DP	77
5.5	Synthesis of Experimental Findings	80
6	Discussion	82
6.1	Federated Adaptation and Cross-Bank Context	82
6.2	Privacy–Utility–Efficiency Trade-offs	84
6.3	Operational Interpretation for Cross-Bank AML	87
6.4	Limitations and Threats to Validity	89
7	Conclusion	92
7.1	Answers to the Research Questions	92

7.2 Outlook	93
A Supplementary Material	95
A.1 Dataset and Generator Configurations	95
A.1.1 IBM AML HI-Large Suitability Audit	95
A.1.2 AMLSim Preliminary Calibration	95
A.1.3 Primary Symmetric Three-Bank Dataset Profile	96
A.1.4 Strongly Asymmetric Stress Dataset	96
A.2 Dataset Validation Coverage	96
A.3 Node Feature Definitions	101
A.4 Protocol Details and Mathematical Derivations	104
A.4.1 CKKS Transport Configuration	104
A.4.2 Analytic Gaussian Calibration	104
A.5 Supplementary Experimental Results	105
A.5.1 Centralized Training Trajectory	105
A.5.2 Federated Configuration Calibration	105
A.5.3 Federated Utility Support	106
A.5.4 Feature-Observability Boundary	107
A.5.5 CKKS and Pairwise-Masked FedAvg Utility	108
A.5.6 Contribution-Row DP Results	109
References	111
License	120

1 Introduction

1.1 Motivation

In crime films, investigators are famously told to “follow the money”. Money laundering turns that seemingly simple instruction into a demanding analytical task, as criminal proceeds may be moved through complex chains of transactions involving intermediaries and asset conversions. The resulting transaction trail can obscure where the funds originated and who ultimately controls them, allowing the proceeds to appear legitimate [1, 2]. Because this process is deliberately concealed, its total scale cannot be measured directly. A widely cited consensus range from the International Monetary Fund (IMF) places the annual scale of money laundering between 2 and 5% of global GDP [3]. A later study by the United Nations Office on Drugs and Crime produced a best estimate of approximately 2.7% in 2009 [4]. Applied illustratively to the World Bank’s reported global GDP for 2025 [5], the broader range corresponds to approximately USD 2.4 to 5.9 trillion.¹

Beyond its sheer scale, money laundering poses a serious threat to financial integrity and economic stability. The IMF warns that laundering activity can undermine governance and regulatory effectiveness, discourage foreign investment, and destabilize international capital markets. Jurisdictions with weak anti-money laundering (AML) controls are also particularly attractive to criminal networks, amplifying cross-border risks and compromising global financial security [6, 7].

Combating these risks is not optional for financial institutions. Internationally, the Financial Action Task Force (FATF) Recommendations serve as a common standard for AML measures [8]. Under the EU AML framework, financial institutions must conduct customer due diligence, monitor transactions and business relationships, and report suspicious activity [9]. Comparable recordkeeping and suspicious-activity reporting obligations in the United States arise under the Bank Secrecy Act, as expanded by the Anti-Money Laundering Act of 2020 [10, 11]. Banks must perform these tasks across transaction volumes that are too large and interconnected to assess solely through manual review. Automated

¹The monetary range extrapolates a historical proportional estimate to 2025 global GDP and does not constitute a contemporary direct measurement of laundered funds.

detection methods support this process by identifying accounts and transactions whose behavior warrants further investigation [12].

The resulting detection problem is inherently relational. An individual transaction records a transfer between two accounts, but relevant behavior may emerge only from sequences of transfers, repeated interactions, shared counterparties, or the wider movement of funds. Because these patterns are expressed through connections between accounts, transaction graphs provide a natural representation for their analysis [13]. In such a graph, accounts are connected by the transactions between them, preserving the relationships through which funds flow. This allows a model to consider not only the attributes of an individual account or transaction, but also its position and interactions within the surrounding transaction network.

Graph neural networks (GNNs) provide an intuitive modeling approach for this representation because they learn account representations from both account attributes and neighboring graph information. Graph convolutional networks (GCNs), a prominent GNN architecture, are particularly relevant because they can exploit graph structure under limited supervision and have been extended to large graphs through scalable variants [14, 15]. These capabilities match important aspects of AML settings, where transaction networks can be extensive and reliable labels scarce. AML detection is further complicated by severe *class imbalance*, as suspicious accounts constitute only a small fraction of the overall population. Despite these challenges, prior AML studies have reported promising results for graph-based detection, including GCN-based methods [13, 16, 17].

Their effectiveness, however, depends on sufficiently broad transaction data that preserve relevant relationships and provide reliable labels [16, 18]. In practice, laundering activity may span several financial institutions and intermediaries, distributing potentially relevant transaction context across banks [19, 20]. This creates a need for collaborative analysis across institutional datasets, so that models can incorporate cross-bank neighborhood context instead of being restricted to a single institution’s view.

Yet the records required for such collaboration are among the most sensitive data held by financial institutions. Their use and disclosure are governed by frameworks such as the EU General Data Protection Regulation (GDPR), sector-specific confidentiality obligations, and national bank secrecy laws [19, 21]. These frameworks do not prohibit every form of information exchange, but they rule out pooling raw customer and transaction records across institutions as a general approach to collaborative model training. The FATF has likewise documented legal restrictions, data-localization requirements, liability concerns, and regulatory uncertainty as barriers to private-sector data pooling and collaborative analytics [19].

A machine-learning setting called federated learning (FL) may nonetheless offer a way to facilitate such collaboration. In this setting, each partic-

ipant trains on its own locally held data and contributes model information to a shared training process, avoiding the need to collect the underlying training records in a central repository [22, 23]. While this makes FL a promising basis for collaborative AML detection, combining local model information alone does not recover the cross-bank neighborhood context needed to capture transaction patterns that span multiple institutions. This tension between institutional data boundaries and relational learning motivates the federated graph-learning setting examined in this thesis.

1.2 Problem Statement

When multi-bank transaction data are analyzed centrally, GNN-based AML detection can operate on the global transaction graph, allowing account representations to incorporate information from neighboring accounts and transactions across bank boundaries. Under conventional federated learning, however, the underlying transaction data remain distributed across banks. Although cross-bank transactions link these partial graph views, each client performs model training on its own local graph. GNN message passing, through which neighboring information is incorporated into account representations, therefore remains confined to that graph. This confinement matters because laundering activity may become apparent only when activity across multiple institutions is considered together [19]. Model updates produced from isolated graph views can communicate what each client learned locally, but combining them afterward cannot convey the predictive information contained in cross-institutional transaction patterns that were not visible during local training [24]. A federated GNN adaptation must therefore enable the shared model to learn from such patterns without centralizing the underlying transaction data.

Evaluating such an adaptation requires several points of comparison. A centralized baseline establishes the utility available when the global transaction graph is accessible. A federated configuration without cross-client graph propagation shows what conventional federated model aggregation retains when every client trains only on its local graph. Comparison with federated GNN configurations that incorporate cross-bank context then provides the empirical basis for estimating the value of that context. That value may also depend on the structural and statistical characteristics of the client datasets. Banks can differ in the sizes and topology of their local graphs, transaction activity, feature distributions, and positive-account rates. Such heterogeneity can affect local optimization and the resulting shared model [25]. A symmetric client configuration therefore provides a controlled reference, while asymmetric configurations deliberately amplify client heterogeneity and test whether the observed conclusions persist.

Keeping raw records at the clients does not by itself make federated

training private. Communicated model updates can still reveal information about the local training data used to produce them [26]. Addressing the missing-neighborhood problem identified above also requires graph-derived information to be exchanged in addition to ordinary model updates. These two potentially sensitive information flows carry different content to different participants and therefore form distinct protection surfaces. Protecting them requires targeted safeguards, since a mechanism that protects one flow does not automatically cover the other. Privacy must consequently be interpreted in relation to the information being protected, the roles from which it is concealed, and the guarantee provided, rather than treated as a binary property of the complete system. Because protection mechanisms may also affect predictive utility or impose runtime, communication, and memory costs, a meaningful evaluation must consider privacy, utility, and efficiency together.

1.3 Research Questions

Together, these challenges motivate the following research questions:

- RQ1.** How can established GCN and federated graph-learning methods be adapted for AML detection over bank-partitioned transaction graphs, and to what extent does the resulting federated system retain the predictive utility of a centralized GCN baseline?
- RQ2.** How much predictive utility is attributable to cross-bank graph information across symmetric and asymmetric federated multi-bank settings with differing degrees of client heterogeneity?
- RQ3.** What privacy–utility–efficiency trade-offs arise when protecting the different information flows in federated cross-bank graph learning?

1.4 Contributions

To answer these questions, the thesis proceeds from controlled multi-bank data generation to federated graph-learning adaptation and mechanism-specific protection. The resulting contributions are summarized below.

1. **Controlled multi-bank evaluation substrate.** We adapted and calibrated the AMLSim generator [27] to produce synthetic transaction data for controlled symmetric and asymmetric multi-bank settings. From these data, we constructed the transaction-graph representation used throughout the centralized, federated, and privacy-mechanism evaluations.

2. Role-bounded realization of cross-client pre-aggregation.

FedGCN proposes a one-shot exchange of pre-aggregated neighborhood features [24]. Its reference implementation does not enforce the contributor–server–recipient boundaries described in the paper. Instead, it supplies clients with hop-expanded graph structure and raw feature rows from nodes assigned to other clients [28]. The later FedGraph implementation more closely realizes the aggregation step, but constructs and encrypts contributions in the global node-feature space, delivers the complete aggregate to every client, and selects the required rows only after decryption [29, 30]. We independently realized the FedGCN design as an explicit message protocol with separate contribution, aggregation, and delivery stages. Contributors construct feature aggregates only from bank-owned nodes and receive only the required target identifiers and propagation coefficients. The coordinating server aggregates bounded, recipient-aligned blocks, and each client receives only the aggregate rows required for its local computation. For two-hop pre-aggregation, cross-client context is represented through aggregate placeholders rather than raw feature rows. This formulation makes ownership, aggregation, and recipient-visibility boundaries explicit. Its bounded, recipient-aligned messages avoid global-sized contribution tensors, support plaintext and CKKS transport, and provide a more direct basis for realizing the protocol across separated organizational roles.²

- 3. Cross-dataset empirical characterization.** We compared centralized GCN learning and cut-edge federation with one-hop and two-hop cross-client pre-aggregation across symmetric and asymmetric client configurations with differing degrees of client heterogeneity. These comparisons quantify the utility attributable to cross-client graph information and show that the preferred pre-aggregation depth is dataset-dependent.

4. Layered privacy mechanisms and systems evaluation.

We protected recipient-routed feature aggregation with CKKS-based homomorphic encryption [31]. We combined this mechanism with pairwise-masked aggregation of model updates and characterized their utility, residual visibility, communication, and memory implications.

5. Formal contribution-row privacy and negative result.

We formalized contribution-row substitution differential privacy for released pre-training payloads, implemented calibrated local and central variants, and observed severe utility degradation across the full tested range: even the weakest formal setting remained substantially below the cut-edge federated baseline.

²The source code, experiment configurations, and supporting artifacts are available in the [public project repository](#).

1.5 Thesis Structure

This thesis is organized as follows. Chapter 2 establishes the foundations of transaction-graph AML detection, graph neural networks, federated graph learning, and the relevant privacy concepts. Chapter 3 positions the thesis within graph-based AML, collaborative and federated AML, coupled federated graph learning, privacy-preserving graph learning, and synthetic AML benchmarking. Chapter 4 presents the multi-bank data generation, graph representation, centralized and federated learning methods, privacy mechanisms, and evaluation methodology.

Chapter 5 presents the experimental setup and evaluates predictive utility, privacy-mechanism behavior, and systems costs across the centralized and federated configurations. It closes in Section 5.5 with a synthesis that organizes the principal findings by research question, providing the empirical basis for RQ1–RQ3. Chapter 6 develops the broader argument from this evidence and examines its interpretation, limitations, and implications. Chapter 7 concludes the thesis by answering the research questions directly and outlining future research directions. Appendix A provides dataset configurations, methodological details, derivations, and supplementary experimental results.

2 Background

2.1 Anti-Money Laundering

Money laundering conceals the illicit origin of funds so that proceeds from criminal activity appear to have a legitimate source [1]. The process is commonly described through placement, layering, and integration. Placement introduces illicit proceeds into the financial system, layering obscures their origin through subsequent movements, and integration gives the funds an apparently legitimate economic purpose. These stages provide a conceptual description rather than a fixed sequence that every laundering operation must follow [2].

Anti-money laundering (AML) measures aim to prevent and detect such activity. Transaction monitoring supports this process by identifying behavior that warrants closer review. An unusual transaction is not necessarily evidence of laundering; its significance depends on the surrounding activity and available information about the customer [32]. When observed activity meets the applicable reporting criteria, a financial institution may file a suspicious activity report (SAR) [33]. A SAR records suspicion rather than establishing that money laundering occurred [32]. Automated detection therefore supports investigation by identifying potentially relevant activity rather than determining criminal responsibility.

Transaction relationships can provide evidence beyond a single account. Repeated transfers, shared counterparties, and movement across several accounts describe relational context that may be difficult to assess from individual records alone. Graph representations make these relationships explicit and provide a basis for incorporating them into predictive models [13, 20].

2.2 Transaction Graphs

Transaction graphs can represent different objects and relationships. Some approaches treat individual transactions as nodes and connect them through the flow of funds, as in the Elliptic Bitcoin dataset [17]. For account-level analysis, a natural formulation instead represents accounts as nodes and transfers between them as directed edges. The choice determines whether a node-level prediction concerns an account or an individual transaction.

2. BACKGROUND

A transaction record identifies a transfer from a source account s to a target account t . Representing this record as an edge $s \rightarrow t$ preserves which account sends and which account receives the funds. An individual transaction record and a graph edge are not necessarily the same object. Source data can contain several transfers with identical ordered endpoints. Keeping every record as a separate edge produces parallel edges and therefore a directed multigraph, a representation also considered in AML graph modeling [34]. Alternatively, the records from one source account to one target account can be consolidated into a single *directed account-pair edge*. The consolidated relation can still retain information such as the number and amount distribution of the underlying transactions [20]. Section 4.2 later specifies how the records are consolidated for this thesis.

The account-pair graph illustrated in Figure 2.1 can be written as

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad \mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}, \quad (2.1)$$

where each node in $\mathcal{V}$ represents an account and each edge in $\mathcal{E}$ represents a directed account-pair relation. Each account node v can additionally be described by a feature vector $\mathbf{x}_v$, which contains numerical information about that account and its activity.

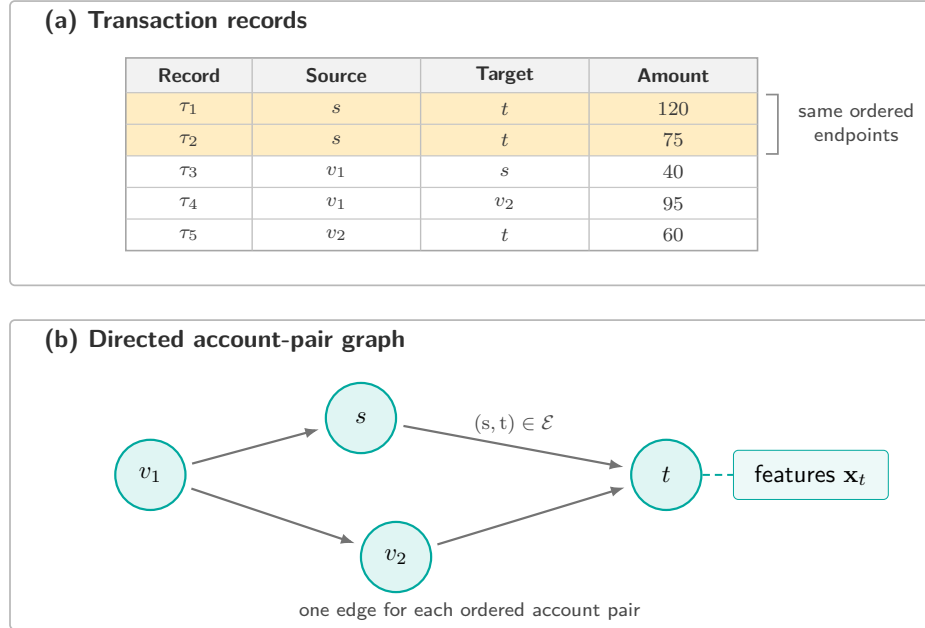


Figure 2.1: Transaction records and their directed account-pair representation. Panel (a) shows separate records, including two transfers with identical ordered endpoints. In Panel (b), accounts form nodes and each ordered account pair forms one directed edge.

In a multi-bank setting, the same transaction graph can be viewed at two different scopes. The *global graph* contains the account nodes and directed relations across all participating banks. A *local or client graph* contains the graph information available to one bank. Edges whose end-points belong to the same bank are within-bank edges, whereas cross-bank edges connect accounts at different banks. When a bank operates only on its local graph, cross-bank neighbors are not automatically available for local graph computation. This structural distinction becomes important when graph learning is later adapted to federated training.

With accounts represented as nodes, assigning a class to an account is a *node-classification task*. A node’s feature vector describes the account itself, while its graph neighborhood identifies connected accounts. For a target node t , the incoming neighborhood consists of the source nodes with directed edges into t . These neighbors provide relational context that a graph-learning model can combine with the target node’s own information. The following section explains how graph neural networks perform this combination.

2.3 Graph Neural Networks

2.3.1 Message Passing and Node Representations

A graph neural network (GNN) is a neural architecture designed to learn from graph-structured data [35]. Its input combines graph relations with numerical features attached to nodes or edges. Rather than processing these features independently, a GNN can update a node using information from its graph neighborhood.

Graph-learning tasks differ in what the model predicts. Node classification assigns a class to each node, such as whether an account exhibits suspicious activity. Edge-level tasks concern relationships between nodes: *edge classification* assigns labels to existing relations, whereas *link prediction* estimates whether a relation exists or is likely to form. *Graph-level prediction* assigns an outcome to an entire graph by combining information across its nodes [36].

A common way to describe GNN architectures is through the message-passing framework [37, 38]. At layer ℓ , each node v has a state $\mathbf{h}_v^{(\ell)}$. The initial state is its input feature vector, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$. A message-passing layer then updates this state through three steps. To describe them, consider a directed relation $s \rightarrow t$: s is the source node providing information and t is the target node being updated. We denote the set of direct incoming source nodes of t by $\mathcal{N}_{\text{in}}(t)$. First, each source node forms a message from its current state. Depending on the architecture, this message may also incorporate the current target state or information attached to the connecting edge. Second, an aggregation function combines the incoming

2. BACKGROUND

messages into a single message for t . Because the neighboring nodes form a set rather than an ordered sequence, the result must not depend on their ordering; common aggregation functions include the sum, mean, and maximum. Finally, an update function combines the aggregate with the previous target state to produce the next state.

Using $\mathbf{m}_{s \rightarrow t}^{(\ell)}$ for the message from s to t and $\mathbf{m}_t^{(\ell)}$ for the aggregated message, one layer can be written as

$$\begin{aligned}\mathbf{m}_{s \rightarrow t}^{(\ell)} &= \text{MSG}^{(\ell)}(\mathbf{h}_t^{(\ell)}, \mathbf{h}_s^{(\ell)}), \\ \mathbf{m}_t^{(\ell)} &= \text{AGG}^{(\ell)}\left(\left\{\mathbf{m}_{s \rightarrow t}^{(\ell)} \mid s \in \mathcal{N}_{\text{in}}(t)\right\}\right), \\ \mathbf{h}_t^{(\ell+1)} &= \text{UPDATE}^{(\ell)}(\mathbf{h}_t^{(\ell)}, \mathbf{m}_t^{(\ell)}).\end{aligned}\tag{2.2}$$

Message and update functions may be learned, while the same functions are applied across the graph. Their parameters are optimized together with the final predictor for the chosen task.

Figure 2.2 shows this process for one target node. The graph determines which source states can send messages, while the aggregation and update determine how those messages alter the target state.

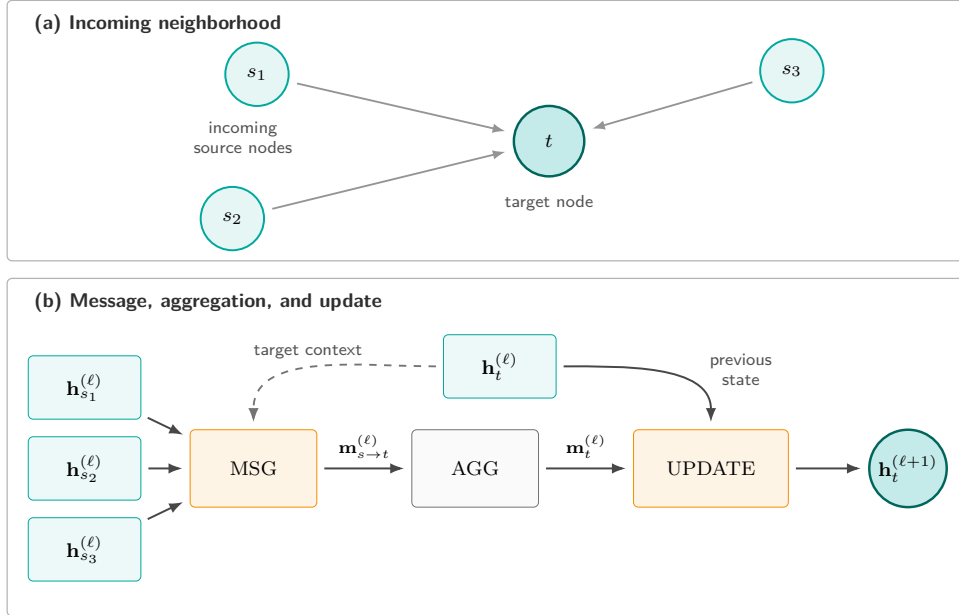


Figure 2.2: Message passing for target node t . Its incoming source nodes form messages, an order-independent aggregation combines them, and the update function produces the target state for the next layer.

Stacking message-passing layers expands the graph context available to each node. After one layer, the state of t can contain information from its direct incoming neighbors. After a second layer, it can also contain information that reached those neighbors from their own incoming neighbors, extending the potential context to two-hop paths. The portion of the graph that can potentially influence a node state is commonly called its *receptive field* [39]. After k layers, this field can extend across paths of up to k edges, although actual influence still depends on edge direction, graph structure, and the learned functions.

2.3.2 Graph Convolutional Networks

The graph convolutional network (GCN) formulation introduced by Kipf and Welling provides a specific architecture for semi-supervised node classification [14]. Whereas the general message-passing framework permits different message, aggregation, and update functions, this formulation combines degree-normalized aggregation with a shared learned transformation.

For $N = |\mathcal{V}|$ nodes, let $\mathbf{A} \in \mathbb{R}^{N \times N}$ be the adjacency matrix. The original formulation assumes an undirected graph. Here, we express propagation using a target-row, source-column convention, so $a_{t,s} > 0$ indicates a directed relation $s \rightarrow t$, consistent with the directed implementation of the GCN operator.¹ A GCN first adds a self-loop to every node, ensuring that the node’s current state remains part of its update.

The resulting normalized propagation matrix is

$$\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N, \quad \hat{\mathbf{D}} = \text{diag}(\hat{d}_1, \dots, \hat{d}_N), \quad \mathbf{P} = \hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2}, \quad (2.3)$$

where $\hat{d}_t = \sum_{s=1}^N \hat{a}_{t,s}$. For a relation $s \rightarrow t$, the normalized propagation coefficient is

$$P_{t,s} = \frac{\hat{a}_{t,s}}{\sqrt{\hat{d}_t \hat{d}_s}}. \quad (2.4)$$

The numerator retains the selected edge or self-loop weight, while the degree terms scale the contribution according to the connectivity of its source and target nodes. The self-loop therefore produces the corresponding coefficient $P_{t,t}$ for the target’s own state. This normalization moderates differences in scale that would arise from simply summing states across differently connected nodes.

Collecting the input feature vectors as rows of $\mathbf{X}$, we initialize $\mathbf{H}^{(0)} = \mathbf{X}$. A complete GCN layer is

$$\mathbf{H}^{(\ell+1)} = \phi\left(\mathbf{P}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}\right), \quad (2.5)$$

¹PyTorch Geometric 2.7.0’s `GCNConv` operator [40] supports directed source-to-target propagation; its [official documentation](#) specifies the normalized update.

where $\mathbf{H}^{(\ell)}$ contains the node states entering layer ℓ , $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ are trainable parameters, and ϕ is a nonlinear activation such as the rectified linear unit (ReLU), allowing stacked layers to model nonlinear relationships.

In terms of the message-passing framework, the source-node states provide the messages. The product $\mathbf{P}\mathbf{H}^{(\ell)}$ performs aggregation by weighting these states with the normalized propagation coefficients and summing them for each target node. The self-loop retains the target’s own state as part of the aggregated information. Multiplication by $\mathbf{W}^{(\ell)}$, addition of $\mathbf{b}^{(\ell)}$, and application of ϕ then form the update that produces the next node state. The propagation coefficients are determined by the graph, while the transformation parameters are learned and shared across nodes.

Stacking GCN layers produces a final state for every node. For binary node classification, the output layer produces one logit: an unconstrained real-valued score that the sigmoid function can convert into an estimated probability of the positive class. Comparing this probability with a decision threshold yields the class prediction.

GCNs can be trained in a transductive, semi-supervised setting. *Transductive* means that the nodes to be classified, together with their features and graph connections, are already available during training. In contrast, *inductive learning* aims to generalize to previously unseen nodes or graphs [36]. *Semi-supervised* means that only a subset of node labels supplies the training signal. Validation and test nodes can therefore participate in message passing even though their labels do not contribute to the training loss [14]. Validation labels support model selection, while test labels are reserved for the final evaluation.

Graph propagation requires the relevant source-node representations to be available during the forward pass. When one graph is divided among several data holders, some of those representations may lie outside a client’s local graph. The following section introduces federated learning before returning to this graph-specific difficulty.

2.4 Federated Learning

Federated learning (FL) allows several participants to train a shared model while retaining the training data in their local environments [22]. This enables collaborative learning without requiring the participants to pool their records. A participant that holds local data is called a *client*.

Two common FL settings differ in the type and number of participating clients [41]. *Cross-device FL* coordinates a large population of personal or edge devices, of which only a subset may be available for a given training round. *Cross-silo FL* instead connects a smaller and more stable group of organizations that retain separate institutional datasets.

A separate distinction concerns how data are partitioned across clients. In *horizontal FL*, clients hold different samples described by the same feature space. In *vertical FL*, clients hold different features for the same or overlapping sets of entities, such as customers represented in several organizations' datasets. Horizontal and vertical partitioning therefore describe the distribution of samples and features, rather than the type or number of participating clients [23].

The following explanation considers server-coordinated horizontal FL. The clients train local copies of a common model, while a coordinating server maintains the shared model state. Training alternates between local computation at the clients and model-state exchange with the server [22].

The shared model should perform well across the data held by all clients. Let $\mathcal{K}$ denote the set of clients, $\mathcal{D}_k$ the local training set of client k , and $n_k = |\mathcal{D}_k|$. For model parameters θ , the local objective is the average loss on that set:

$$f_k(\theta) = \frac{1}{n_k} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_k} \ell(\theta; \mathbf{x}_i, y_i). \quad (2.6)$$

Here, ℓ measures the model loss for one training example. The global objective combines the local objectives in proportion to the amount of training data held by each client:

$$\min_{\theta} f(\theta), \quad f(\theta) = \sum_{k \in \mathcal{K}} \frac{n_k}{n} f_k(\theta), \quad n = \sum_{k \in \mathcal{K}} n_k. \quad (2.7)$$

This formulation gives every training example the same influence on the shared objective even when the client datasets differ in size.

Training proceeds through a sequence of communication rounds. At the start of a round, the server distributes the current shared model state to the clients. Each client begins from this state and trains the model on its local data. One complete pass over the local training data is called a local epoch. A client may perform several local epochs before returning its updated model state to the server, which then combines the returned states to obtain the shared model for the next round.

The foundational federated averaging (FedAvg) algorithm performs this combination as a weighted average [22]. Let $\theta^{(r)}$ denote the shared model state at the start of round r and $\theta_k^{(r)}$ the state returned by client k . The next shared state is

$$\theta^{(r+1)} = \sum_{k \in \mathcal{K}} \alpha_k^{(r)} \theta_k^{(r)}, \quad \sum_{k \in \mathcal{K}} \alpha_k^{(r)} = 1, \quad (2.8)$$

where $\alpha_k^{(r)}$ controls the contribution of client k . These weights commonly reflect the amount of training data held by each client, so that larger local datasets contribute proportionally to the shared state.

The client datasets need not follow the same distribution. Client data are called non-IID when they do not satisfy the usual independent and identically distributed assumption; for example, class proportions and feature values can differ among clients. Because each client trains on its own distribution, the local model states may move in different directions, which can affect federated optimization and the resulting shared model [25]. Federated learning keeps raw training records at the clients, but the communicated model states can still disclose information about the data used to produce them [26]. Section 2.6 introduces relevant protection concepts. The following section turns to graph-structured data and introduces the accompanying challenges.

2.5 Federated Graph Learning

Federated graph learning applies the general collaborative training setting to graph-structured data [42]. The clients still train a shared model, but the graph data on which they operate can be distributed in different ways. In one arrangement, each client holds an independent graph and can perform message passing within that graph before its model state is aggregated with those of the other clients. A different situation arises when the nodes of one global graph are distributed among several clients. This is the arrangement relevant to a multi-bank transaction graph: each bank can operate on its local graph, while cross-bank transactions connect its account nodes to nodes held by other clients. When these distinct account nodes are described by a common feature schema, their distribution across banks corresponds to horizontal partitioning. Figure 2.3 contrasts these two arrangements.

The partitioned setting creates a graph-specific difficulty during local model computation. The propagation term $\mathbf{PH}^{(\ell)}$ in Equation (2.5) requires the current states of connected source nodes. In Panel (b), s_1 and s_2 illustrate two such source nodes connected to target node t . More generally, consider a cross-bank relation $s \rightarrow t$ in which the source and target nodes are held by different clients. The client holding t can propagate over its local graph, but without an additional exchange, the state of s is unavailable during its forward pass. Conventional FL therefore cannot use this cross-client relation for local message passing. Incorporating the missing neighborhood context requires graph-specific communication in addition to model-state exchange [24].

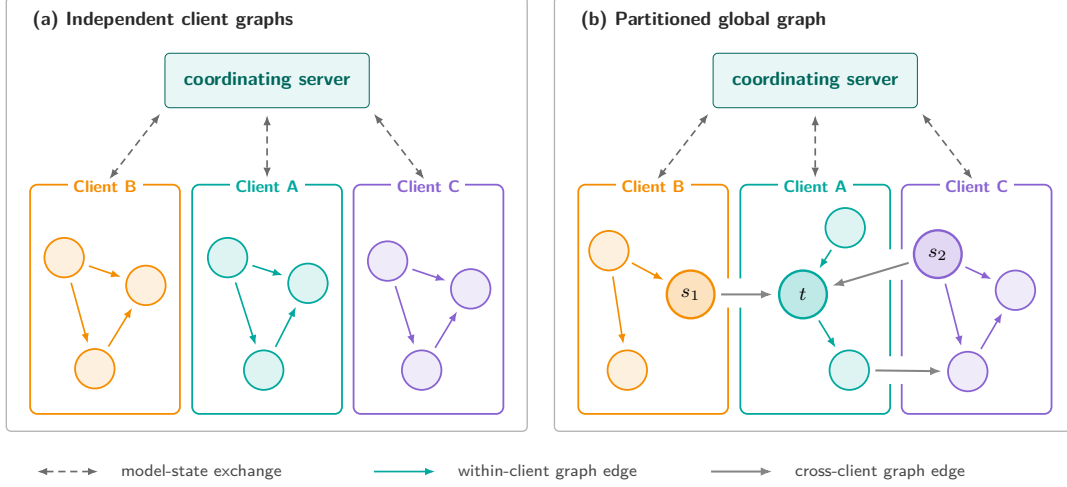


Figure 2.3: Independent and partitioned graph arrangements in federated graph learning. Panel (a) contains separate client graphs, whereas Panel (b) contains cross-client edges within one global graph.

FedGCN addresses this missing context by communicating accumulated neighbor-feature information in a single communication step before training [24]. For the first GCN layer, $\mathbf{H}^{(0)} = \mathbf{X}$, so its graph-dependent input is $\mathbf{P}\mathbf{X}$. Expressed using the GCN notation established above, the matrix of fixed feature aggregates and its row for target node t are

$$\mathbf{Z} = \mathbf{P}\mathbf{X}, \quad \mathbf{z}_t = \sum_{s \in \mathcal{V}} P_{t,s} \mathbf{x}_s. \quad (2.9)$$

The original formulation uses adjacency-weighted neighbor sums. Here, $\mathbf{P}$ expresses the same aggregation principle using the normalized propagation operator defined for the GCN. The row $\mathbf{z}_t$ is the *one-hop feature aggregate* of t : it combines the input features of its incoming source nodes and its own self-loop contribution. Both the normalized propagation matrix $\mathbf{P}$ and the input feature matrix $\mathbf{X}$ are fixed before model training, making this aggregate independent of the learned model parameters.

For each required target node, clients compute the parts of the feature aggregate contributed by the nodes they hold. The coordinating server adds these partial aggregates and returns the completed aggregate to the client that requires it [24]. We refer to this operation as *cross-client feature pre-aggregation*. FedGCN distinguishes its one-hop and two-hop settings by the cross-client context made available to each client. Expressed in the notation above, the one-hop setting supplies aggregate rows for the client’s target nodes. The two-hop setting additionally supplies aggregate rows for the one-hop neighbors of those targets, including $\mathbf{z}_s$ for cross-client source nodes s . Since each $\mathbf{z}_s$ already contains one-hop context around s ,

a subsequent local propagation step can carry that context into t . The two-hop label therefore describes the reach of the resulting cross-client context. The clients retain the received aggregates during local model optimization, so subsequent communication is limited to the model states used by FedAvg. This fixed-input argument does not apply to later hidden representations, which depend on the learned parameters. Section 4.4 adapts FedGCN’s one-hop and two-hop settings to the directed, bank-partitioned transaction graph and defines the contribution and recipient-routing protocol used in this thesis. Because the additional exchange exposes graph-derived information beyond ordinary model states, it also introduces a separate protection surface. The following section introduces privacy enhancements that limit what these exchanges reveal.

2.6 Privacy Enhancements

Local data retention provides a privacy-enhancing property of federated learning: participants can collaborate without pooling their training records. Additional protection addresses exposure through the model updates and graph-derived information that are still exchanged.

A threat model makes this scope explicit by specifying the participating actors, the information available to them, and the behavior under consideration. Under an *honest-but-curious* model, a party follows the prescribed protocol but may analyze the values it legitimately observes [43]. Privacy claims must therefore identify both the protected object and the observer from whom it is protected.

For example, a coordinating server may obtain information about local training data from the updates submitted by individual clients. Geiping et al. [44] demonstrate that an honest-but-curious server can reconstruct training images from client gradients and, in selected settings, from model updates after multiple local optimization steps. Their attacker observes these submissions without manipulating the training procedure. The feasibility of reconstruction depends on the information available to the server and the training configuration.

The mechanisms introduced below address different aspects of this exposure. Homomorphic encryption controls whether a party can read values while computing on them. Differential privacy constrains what a released result reveals about a defined protected input. Secure aggregation allows an aggregator to compute a combined result without seeing each individual input. The following subsections introduce their respective protection principles.

2.6.1 Homomorphic Encryption

Asymmetric encryption uses a public key to encrypt a value and a corresponding private key to decrypt it. With ordinary encrypted commu-

nication, the receiving party normally decrypts a value before using it in a computation. Homomorphic encryption (HE) instead permits selected operations to be performed directly on ciphertexts. Decrypting the resulting ciphertext then recovers the result of the corresponding plaintext operation [31].

Homomorphic encryption schemes differ in their supported operations and numerical representations. Paillier supports addition of encrypted integer messages and multiplication by known integer scalars, with results interpreted modulo the plaintext modulus [45]. The Brakerski–Fan–Vercauteren (BFV) and Brakerski–Gentry–Vaikuntanathan (BGV) schemes support exact modular arithmetic on encrypted integers [46, 47]. Implementations such as Microsoft SEAL also support batching, which packs multiple integer values for simultaneous computation [48]. Using these schemes for real-valued data requires a numerical encoding, such as scaled integers, whose precision and range must be chosen for the intended computation.

The Cheon–Kim–Kim–Song (CKKS) scheme instead provides approximate arithmetic on encrypted numerical data [31]. It can pack several real- or complex-valued numbers into one ciphertext and process them together through encrypted additions and multiplications. Encoding, rescaling, and ciphertext arithmetic introduce numerical approximation, so parameters must be selected for the intended computation to keep the decrypted result close to its plaintext counterpart. This combination of packed arithmetic and controlled approximation makes CKKS suitable for encrypted aggregation of vectors.

2.6.2 Differential Privacy

Differential privacy (DP) provides a formal way to limit what a released output reveals about one protected unit in its input. It defines this guarantee by comparing neighboring inputs, which differ in one such unit. For example, the protected unit may be one database record or, for graph data, one node or one edge. Each choice defines a different neighboring relation and therefore a different guarantee [49]. A DP claim can consequently be interpreted only after its protected unit and neighboring relation have been defined.

Let $\mathcal{M}$ be a randomized mechanism, let D and D' be neighboring inputs, and let S be a set of possible outputs. The mechanism satisfies (ϵ, δ) -differential privacy if

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad (2.10)$$

for every neighboring pair and output set [50]. Because $\mathcal{M}$ is randomized, the comparison concerns output distributions rather than equality between individual results. The parameter ϵ bounds their multiplicative separation, while δ permits a small additive deviation from that bound.

2. BACKGROUND

For a fixed neighboring relation, reducing either parameter while holding the other fixed strengthens the privacy requirement. The guarantee does not state that the released output contains no information about the protected input.

Constructing such a mechanism by adding noise requires bounding how much the underlying computation can change between neighboring inputs. This maximum change is called the *sensitivity*. For computations that aggregate vector-valued contributions, clipping each contribution to a fixed norm can help bound sensitivity. The bound must also account for how many contributions one protected unit can affect. Common approaches for numerical releases include the Laplace and Gaussian mechanisms, which add noise calibrated to this sensitivity [50].

The Gaussian mechanism releases a deterministic function f together with a random noise vector:

$$\mathcal{M}(D) = f(D) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}). \quad (2.11)$$

The noise scale σ is calibrated from the sensitivity and the selected privacy parameters. Under an otherwise fixed mechanism, greater sensitivity or stronger privacy requirements generally require more noise. The classical Gaussian mechanism uses a conservative calibration, while the analytic Gaussian mechanism evaluates the Gaussian distribution more directly and can require less noise for the same stated guarantee [50, 51]. Adding Gaussian noise without a defined neighboring relation, sensitivity bound, and calibration does not by itself establish differential privacy.

Once a DP output has been released, subsequent computation based only on that output preserves the guarantee; this property is known as *post-processing* [50]. Further access to the protected input or additional independently randomized releases may require separate privacy analysis.

2.6.3 Secure Aggregation

Plaintext FedAvg gives the coordinating server access to every participating client's returned model state before aggregation. Such model updates can reveal information about local training data even when the underlying records remain at the clients [26]. Secure aggregation addresses this server-side visibility by allowing the server to compute an aggregate, typically a sum, without learning the individual submitted vectors [52].

Pairwise additive masks form the algebraic core of the secure-aggregation construction considered here. For clients $k, q \in \mathcal{K}$, let $\mathbf{u}_k^{(r)}$ be the vector submitted by client k in round r , and let $\boldsymbol{\rho}_{kq}^{(r)}$ be a mask shared by k and q . Under a fixed client ordering, client k sends the masked vector

$$\mathbf{m}_k^{(r)} = \mathbf{u}_k^{(r)} + \sum_{q>k} \boldsymbol{\rho}_{kq}^{(r)} - \sum_{q<k} \boldsymbol{\rho}_{qk}^{(r)}. \quad (2.12)$$

When every client in $\mathcal{K}$ submits, each pairwise mask occurs once with a positive and once with a negative sign. The masks therefore cancel:

$$\sum_{k \in \mathcal{K}} \mathbf{m}_k^{(r)} = \sum_{k \in \mathcal{K}} \mathbf{u}_k^{(r)}. \quad (2.13)$$

Provided that the pairwise masks remain unavailable to the server, it can compute the unchanged aggregate without receiving the individual unmasked vectors. Their cancellation does not by itself constitute a complete secure-aggregation protocol. A deployable protocol must additionally establish pairwise secrets, authenticate the relevant exchanges, tolerate participating clients that drop out, and specify the adversarial and collusion assumptions under which individual submissions remain hidden [52].

These foundations distinguish three different forms of protection: computation without plaintext access, bounded disclosure through a randomized release, and aggregate-only visibility. The following chapter examines how prior graph and federated systems apply these ideas, while Chapter 4 defines how they are applied in this thesis.

3 Related Work

3.1 Synthetic AML Data and Evaluation Benchmarks

The limited availability of real transaction data with reliable labels has produced a varied AML benchmark landscape spanning synthetic simulators, bank-derived synthetic datasets, and public transaction networks. AMLSim, an open-source simulator developed by IBM researchers, generates accounts, transactions, and configured laundering patterns [13, 27]. Its open implementation and configurable scenarios have made it a useful basis for controlled AML experiments. AMLworld, developed by a later IBM Research team, advances this line of synthetic AML research by incorporating the laundering structures modeled by AMLSim into a broader virtual economy with richer entity relationships, a more comprehensive laundering lifecycle, and multiple currencies [53]. Its released datasets, collectively referred to as IBM AML, vary in scale and laundering rate and provide transaction-level ground truth based on the simulated movement of illicit funds. They have supported centralized GNN experiments [54] as well as interbank experiments in CE-FedGNN [55].

More recent generators have further expanded the synthetic AML data landscape. The 2026 Tide preprint combines customizable structural and temporal pattern definitions with programmatic injection of user-defined laundering schemes [56]. The peer-reviewed TransXion benchmark combines persistent entity profiles with conditional transaction behavior to generate illicit activity without relying solely on predefined laundering motifs [57]. Tide broadens structural and temporal configurability, whereas TransXion adds profile-aware generation. Their reported experiments remain centralized and do not evaluate federated GNN training over bank-owned graph partitions.

Other resources follow different approaches to the data-access problem. SynthAML generates synthetic AML alerts and their associated transaction histories using a probabilistic model fitted to confidential data from Spar Nord, a Danish bank [58]. Its validation compares alert-classification performance across synthetic and confidential source-bank data, but the released dataset does not provide a multi-bank graph partition. The Elliptic dataset instead represents Bitcoin activity as a temporal graph in which

transactions form the nodes, directed payment flows form the edges, and a subset of nodes is labeled as licit or illicit [17]. It has become an established benchmark for graph-based AML and has also been used in partitioned federated graph experiments [59]. Its cryptocurrency setting nevertheless differs from the institutional banking setting considered in this thesis. The later Elliptic2 benchmark extends this public cryptocurrency-data lineage with 122,000 labeled Bitcoin subgraphs embedded in a much larger background transaction graph [60].

Against this literature, the selection and construction of the multi-bank data used in this thesis form part of the experimental design. The chosen approach and the reasons for it are reported in Section 4.1. With the benchmark landscape established, the next section considers how prior AML methods represent and learn from transaction data.

3.2 Graph-Based Anti-Money Laundering

Graph-based AML research differs substantially in how it represents financial activity and defines the prediction task. Representative studies of semi-supervised node classification provide evidence for scalable GCN-based AML. Weber et al. [13] examined learning on an account-node transaction graph generated with AMLSim and provided an initial large-scale feasibility result under synthetic data and centralized computation. The Elliptic benchmark instead provides a transaction-node classification setting. In its original evaluation, GCN and Skip-GCN outperformed logistic regression, but random forest achieved the strongest overall result. Adding GCN-derived node embeddings to its full feature set produced only a marginal further improvement, showing that relational information was useful without making the GCN uniformly superior [17].

Karim et al. [16] provide a broader cross-dataset comparison of GCN models and configurations that supply graph embeddings to ensemble classifiers. On AMLSim and Elliptic, the strongest GCN variant outperforms all evaluated embedding-based ensemble configurations. On IBM AML and SynthAML, however, enriching dynamic graph transformer (DGT) embeddings with original and topological features in XGBoost enables the combined models to outperform the GCN variants. The results therefore support scalable GCN-based AML while showing that the preferred formulation depends on the dataset and available features.

Whereas the preceding studies classify graph nodes, Egressy et al. again represent accounts as nodes and transactions as edges, but formulate AML detection as an edge-classification task. Their Multi-GNN variants adapt standard GNNs through reverse message passing, port numbering, and ego identifiers, optionally combined with edge updates, to better capture transaction patterns [54].

Other work explicitly models collective laundering behavior. Cheng et

al.’s GAGNN combines a community-centric encoder with node-, edge-, and group-level objectives to detect organized laundering activity in private UnionPay data, thereby extending individual account and transaction predictions with a group-level objective [61].

LaundroGraph explores a complementary self-supervised setting. It represents customers and transactions as a bipartite graph and learns their representations through link prediction. The resulting anomaly scores identify unusual transactions in the context of a customer’s behavior and support analyst review without requiring suspicious-activity labels [62]. These studies show that the graph representation, prediction target, and available supervision jointly determine what information the model can exploit and what its reported utility means.

Across this literature, graph-based AML spans classification of account nodes, transaction nodes, and transaction edges, while other formulations introduce group-aware objectives or self-supervised representations for analyst support. The federated multi-bank setting examined in this thesis builds on account-level node classification.

3.3 Federated Graph Learning

Federated graph learning includes settings in which clients own independent graphs and settings in which they own partitions of one graph with edges crossing client boundaries. The latter setting is directly relevant to multi-bank AML because cross-bank transactions connect account nodes assigned to different clients. It creates a missing-neighbor problem when local message passing cannot directly use features held across a client boundary. Prior work addresses this problem through different forms of cross-client graph-information exchange and through mechanisms that protect the resulting information flows.

3.3.1 Cross-Client Graph Information

FedSage+ learns a missing-neighbor generator and uses the generated nodes to augment local subgraphs before federated GNN training [63]. This avoids direct exchange of the true remote neighborhood, but its utility depends on the quality of the generated information.

FedGCN provides the direct methodological basis for the cross-client feature pre-aggregation used in this thesis [24]. For each neighborhood that spans client boundaries, clients compute separate partial feature aggregates from the nodes they hold locally. A coordinating server sums these partial aggregates in one pre-training exchange, after which ordinary federated model training proceeds without further graph-data communication. FedGCN compares zero-, one-, and two-hop communication variants and proposes homomorphic encryption for aggregating the communicated features. The later FedGraph research library, presented in a

2024 preprint by an overlapping author team, incorporates FedGCN into a broader framework for distributed training and systems benchmarking [29].

Other methods instead exchange learned cross-client representations during training because these representations change as the model is optimized. OptimES performs boundary-embedding exchange in each federated round but reduces its cost by pruning remote neighborhoods, overlapping embedding transfer with local training, and retrieving only the embeddings needed by each client [64]. Fed2GNN supports deeper GNNs while reducing the cost of repeated direct exchange [65].

Two recent preprints apply cross-client representation exchange directly to financial transaction graphs. FedGraph-VASP treats graph partitions as virtual asset service providers and exchanges encrypted boundary-node embeddings between them. It evaluates this design for fraud detection on the Elliptic and Ethereum graphs [59]. Its results indicate that the benefit of embedding exchange depends on cross-client connectivity, while its privacy audit finds that authorized recipients can infer information from the exchanged embeddings. CE-FedGNN exchanges aggregated moving-average node representations less frequently than per-round approaches such as OptimES and evaluates both citation graphs and synthetic interbank AML data [55]. It combines communication analysis with a metric-DP guarantee for released representations.

Together, these methods recover missing cross-client context through generated neighbors, one-time feature pre-aggregation, or learned representations exchanged during training.

3.3.2 *Privacy-Preserving Federated Graph Learning*

Federated graph learning creates several points at which information becomes visible to another role. A coordinating server may process graph-derived values, participating clients may receive aggregate features or embeddings, and client updates are submitted for model aggregation. Consequently, prior work must be compared by which values it protects, from which observers, and at what stage of the protocol, rather than solely by whether it uses a privacy-enhancing technology.

Homomorphic encryption has been applied to graph computation under several ownership models. FedVGCN considers two data holders with complementary data for the same computation graph, assisted by a coordinating server. It uses Paillier encryption to exchange intermediate results during GCN training and assumes honest-but-curious parties, with no collusion between the server and either data holder [66]. CryptoGCN addresses GCN inference under a semi-honest cloud-server model. It uses CKKS to encrypt client node features while leaving the normalized adjacency matrix in plaintext [67]. Both protect graph computations, but

3. RELATED WORK

neither addresses recipient-specific aggregation across partitions connected by cross-client edges.

FedGCN and the FedGraph research library are more direct comparators because they support encrypted aggregation of cross-client feature information [24, 29]. In the accompanying FedGraph implementation, encrypted client tensors are combined into a global aggregate, while recipient selection occurs only after client-side decryption [30]. OptimES provides a recipient-aware exchange comparator rather than a cryptographic one. Its honest-but-curious embedding server stores shared embeddings, while clients retrieve only those needed by their sampled computation graphs. This restricts client-side delivery but does not hide the embeddings from the server or perform additive homomorphic aggregation of source contributions [64]. The relevant distinction is therefore not whether encryption or recipient-aware exchange exists, but which role observes each value and when recipient restriction occurs.

Model updates form a separate protection surface. Protocols for secure aggregation allow a server to recover an aggregate without learning each client’s update. Bonawitz et al. provide variants for honest-but-curious and active adversaries, including bounded collusion between the server and clients and client dropout [52]. Such protocols are established independently of graph learning and protect only the aggregation of model updates; they do not protect the separate graph-derived exchanges used to supply cross-client neighborhood information.

Differentially private graph learning varies with both the released object and the neighboring relation. FedGCN notes that aggregation no longer obscures an individual contribution when only one cross-client neighbor is present and suggests adding differential privacy in this case. It does not, however, specify an adjacency relation, instantiate a calibrated mechanism, or report a differential-privacy evaluation [24]. GAP perturbs graph aggregation and provides edge- and node-level guarantees for central GNN training [68]. These units protect changes to graph structure or a node and its incident edges, whereas a communicated representation can instead be protected with respect to distances in representation space. CE-FedGNN explicitly considers an honest-but-curious adversary that observes all exchanged messages and the publicly visible release cohort. It applies metric-DP to repeatedly released node representations and accounts for those releases through Rényi differential privacy composition [55].

The resulting guarantees are not interchangeable. Cryptographic protections depend on the observer and values covered by the protocol, while differential privacy depends on its neighboring relation and released object. Some systems combine protection for more than one surface, but the documented guarantees reviewed here do not establish end-to-end privacy across graph-derived exchanges, recipient-visible outputs, model updates, and graph topology.

3.4 Collaborative and Federated Anti-Money Laundering

Within AML, the institutional fragmentation of transaction data has motivated several forms of collaborative analysis. In a 2019 preprint, Suzumura et al. [69] combine graph-derived features with federated model training on data from a Financial Conduct Authority TechSprint. Institutions compute features from their local views, while a collaborative stage supplies broader graph information before federated training. The reported federated model is a small feed-forward network trained on transaction and graph-derived features, and it outperformed the corresponding local models in their evaluation. It therefore provides early evidence that cross-institutional graph information can complement federated model training.

Other collaborative systems perform protected graph algorithms directly rather than training a shared predictive model. Van Egmond et al. [70] introduce secure risk propagation over an interbank transaction network using a distributed Paillier-based secure multi-party computation (MPC) protocol. They evaluate its predictive value on real Dutch bank data and benchmark its runtime separately on generated graphs. The real-data experiment reports substantially higher precision at matched recall after risk propagation, while separate runtime measurements on generated graphs show approximately linear scaling for the Paillier-based implementation. Tian et al. [20] address secure discovery of laundering subgroups across two financial institutions. Their scatter-gather procedure represents cross-institution relations and is evaluated on a private dataset with more than 200 million accounts as well as balanced and unbalanced synthetic settings. Both studies demonstrate protected cross-bank graph computation in experimental settings, but they address different analytical tasks. Van Egmond et al. propagate predefined risk signals, while Tian et al. detect scatter-gather laundering subgroups; neither jointly trains a GCN for account classification.

Privacy-enhancing technologies have also been examined at a broader operational level. Project Aurora compares centralized and collaborative architectures for network-based AML analytics. Its proof of concept evaluates federated learning together with homomorphic encryption and local differential privacy, while also considering secure multi-party computation [71]. It reports improved detection and fewer false positives under collaborative analysis while identifying legal, technical, and operational barriers to deployment. A 2026 preprint on DPxFin studies reputation-guided adaptive differential privacy for model updates in federated, multi-layer perceptron (MLP)-based AML classification on tabular data. It reports modest accuracy gains over fixed-noise differential privacy together with lower TabLeak reconstruction accuracy [72].

These application-specific studies illustrate the range of analytical

tasks and protection goals already considered in collaborative AML, providing the final context for delimiting the combined gap addressed in this thesis.

3.5 Research Gap

The surveyed literature establishes the individual foundations required for federated graph-based AML. Synthetic generators and multi-bank datasets provide data for controlled evaluation, while collaborative AML systems demonstrate both the relevance and feasibility of cross-institutional analysis. Centralized graph-based AML covers several graph representations and prediction targets, and federated graph methods provide established ways to recover or approximate missing cross-client context. Within this design space, FedGCN is particularly relevant because it incorporates fixed cross-client neighborhood information through a single pre-training exchange before continuing with ordinary federated model training [24]. This retains an established GCN basis and allows the empirical focus to remain on the value of cross-bank graph information rather than on a new graph-learning architecture.

The original FedGCN study evaluates general graph benchmarks rather than AML data [24]. Across all AML-specific studies identified as part of this review, the recent CE-FedGNN preprint is the only one to report experiments with a baseline labelled FedGCN [55].¹ In that study, FedGCN serves as one comparator within a broader evaluation of GIN and PNA models. Although the paper reports common training settings, predictive results, and runtime measurements, FedGCN is not analyzed as a protocol in its own right or compared with centralized GCN-based detection. The experiments also do not isolate the predictive value of cross-bank information across controlled symmetric and asymmetric client configurations. Direct evidence for these questions therefore remains limited.

A further gap concerns how the one-shot exchange is realized and protected. FedGCN describes a contribution–server–recipient exchange at the paper level [24]. The examined reference implementation does not enforce this exchange, instead supplying clients with hop-expanded foreign graph context [28]. The later FedGraph implementation more closely reflects the paper-level aggregation, but its encrypted path forms a global aggregate and restricts recipient rows only after client-side decryption [30]. Existing implementations therefore do not establish how different privacy mechanisms compare once bank-owned contribution construction, server aggregation, and recipient-specific delivery are enforced as explicit protocol boundaries. It also remains insufficiently characterized how protecting the pre-training payload and model updates affects utility and systems

¹The preprint was submitted in May 2026, while the federated implementation presented in this thesis was already underway.

cost within the same setting. Formal privacy for the one-shot pre-training payload remains incompletely specified in prior work. FedGCN suggests adding differential privacy to communicated data when a node has only one cross-client neighbor, but does not instantiate or evaluate a mechanism [24]. FedGraph later reports a DP option for FedGCN and near-identical accuracy to plaintext training on Cora [29]. However, the paper does not identify the protected unit or neighboring relation, report the privacy parameters used in that experiment, or derive the sensitivity of the released aggregate. Its implementation clips each client’s complete feature-sum tensor at the server, perturbs the global sum, and configures the clipping norm and sensitivity independently [30]. Consequently, the surveyed work does not establish a formal guarantee for the FedGCN payload or evaluate such a guarantee for cross-client AML.

Against this background, this thesis uses FedGCN’s established one-shot aggregation as its methodological basis and refines its role and delivery boundaries for controlled multi-bank evaluation. Rather than introducing a new graph-learning architecture, it investigates utility relative to centralized learning, the value of cross-bank graph information across different data settings, and the privacy–utility–efficiency consequences of protecting the resulting information flows.

4

Methodology

This chapter presents the methodological approach, from synthetic multi-bank data generation and graph construction to centralized and federated learning, privacy enhancements, and evaluation. Figure 4.1 summarizes the components and their relationships.

4.1 Synthetic Multi-Bank Data Generation

Evaluating federated graph learning across banks requires more than a labeled transaction dataset. The data must support account-level graph learning within distinct bank partitions while providing cross-bank connections whose predictive value can be assessed through the federated comparisons. For this study, that required a small number of sufficiently large bank partitions, broad account participation in transactions, and within-bank connectivity to support local graph learning. Positive accounts associated with both bank-restricted and cross-bank laundering patterns were needed across the banks to support local training and global evaluation. No publicly available real-world dataset meeting these structural, scale, and labeling requirements could be identified.

Beyond observational suitability, the experimental design required control over client size, transaction topology, and client heterogeneity. A configurable synthetic generator thus provided the most suitable available basis for constructing controlled datasets. AMLWorld [53], a configurable simulator for synthetic financial transactions and money-laundering scenarios, could in principle have generated data tailored to these requirements, but its generator code was not publicly available. IBM instead provides several datasets generated with AMLWorld under the collective name IBM AML [53]. Among them, we audited HI-Large because its overall scale appeared most compatible with the intended experiments.

HI-Large contains 2,126,855 accounts, but these are distributed across 122,333 institutions. Its overall scale therefore reflects a highly fragmented institutional distribution rather than the intended setting of a few large banks. Even the ten institutions with the highest transaction participation contain only 26,532 accounts in total, with at most 3,800 accounts at any selected institution. These client populations are substantially smaller than intended for repeated local node-classification training. Re-

4.1. Synthetic Multi-Bank Data Generation

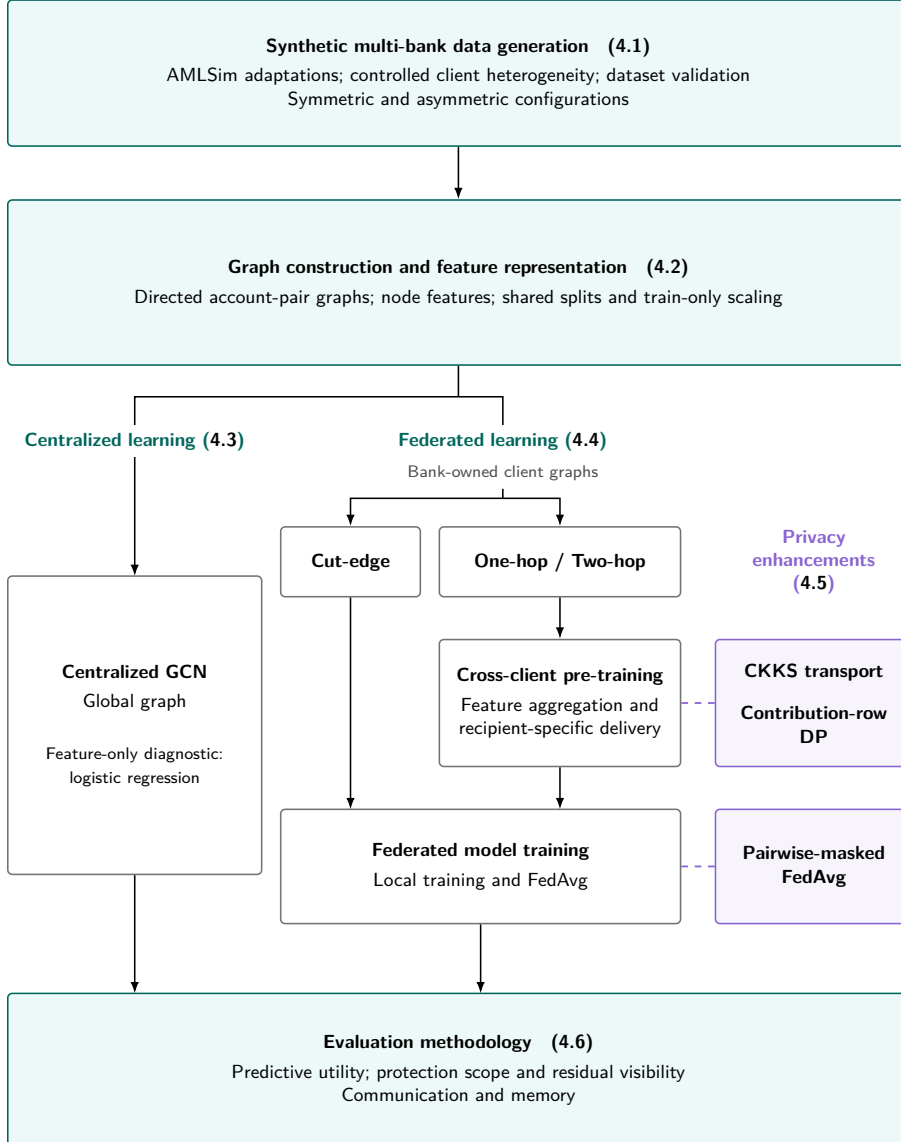


Figure 4.1: Overview of the methodological approach. Shared data-generation and graph-construction procedures support centralized and federated learning configurations. Section numbers identify the corresponding methodological descriptions.

stricting the data to transactions between the selected institutions retains only 958,331 of 179.7 million transaction records, or approximately 0.53%. The subset may be computationally manageable, but the combination of relatively small client populations and severe structural truncation makes it unsuitable for the federated experiments conducted in this study. Additional bank-level audit results are reported in Appendix A.1.1.

We therefore selected AMLSim [13], an earlier open-source simulator developed by IBM researchers and a conceptual predecessor to AMLWorld [53]. AMLSim generates account and transaction data from parameterized normal and laundering patterns. This configurability, together with access to its source code [27], supported the systematic calibration of dataset scale, bank partitions, cross-bank connectivity, and client heterogeneity. We also considered two complementary candidate datasets: SynthAML, a semi-synthetic banking dataset [58], and Elliptic [17], a labeled Bitcoin transaction graph.¹ Elliptic was excluded because its cryptocurrency transaction graph does not provide the bank ownership and account-level node structure required for the federated setting. SynthAML could have provided cross-source robustness, but incorporating it would have required a separate preprocessing and validation workflow that fell outside the final experimental scope. The final design therefore prioritized controlled comparisons across datasets produced by the same generator over cross-source robustness. Consequently, the empirical findings are grounded in one synthetic generation framework, and their generalizability to other synthetic or real-world transaction data remains to be established.

AMLSim thus provided the common generation basis for the symmetric and asymmetric datasets developed in this study. The following subsections describe the generation workflow, the thesis-specific adaptations, the controlled variation of client distributions, and the procedure used to validate the generated outputs. Section 5.1 presents the retained dataset configurations and their experimental roles.

4.1.1 AMLSim Pipeline and Generator Adaptations

AMLSim separates dataset generation into three stages [27]. First, graph and model construction establishes accounts, a directed base graph, normal transaction models, and generated laundering patterns from parameter files. Transaction simulation then realizes account activity over a configured sequence of time steps. Finally, simulator-log conversion produces tables containing accounts, individual transaction records, and pattern memberships. AMLSim can generate both SAR-designated and false-

¹The more recent Tide and TransXion data resources discussed in Section 3.1 appeared after the AMLSim-based generation approach had been selected and the corresponding data-generation pipeline was already under development. They were therefore not part of the original dataset-selection process [56, 57].

alert patterns. We configured the retained datasets to use SAR-designated patterns only, and membership in these generated patterns defines the ground-truth positive class. We refer to their members as SAR-labeled accounts and subsequently as positive accounts. We packaged the AMLSim generation pipeline and its dependencies in a containerized environment to support repeatable preliminary runs and full dataset generation from fixed configurations. Figure 4.2 summarizes the AMLSim generation workflow used in this study and highlights the stages at which adaptations were introduced.

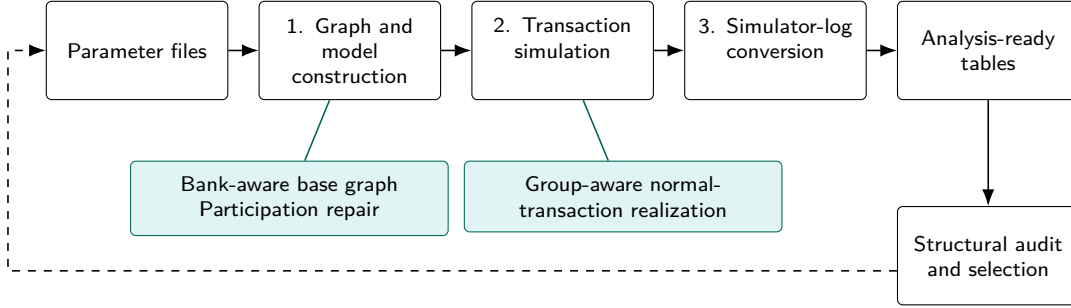


Figure 4.2: AMLSim generation and calibration pipeline. Black boxes show the inputs, three generator stages, outputs, and structural audit. Green boxes identify where adaptations were made. Structural audits feed back into the parameterization and graph and model construction before a configuration is retained for full-scale generation.

The first adaptation modifies base-graph construction. At this stage, AMLSim constructs a directed base graph that establishes potential account-to-account relationships before the transaction simulation generates individual transaction records. In the original generator, this graph was constructed by randomly matching prescribed incoming and outgoing degrees without considering the banks to which accounts belonged [27]. Bank affiliation was therefore recorded for each account but did not provide control over the proportion of within-bank and cross-bank edges. We introduced a bank-aware construction procedure that first creates within-bank edges toward a configurable global target and then assigns the remaining degree requirements preferentially across banks. The procedure uses the same global incoming and outgoing degree sequence as its basis while making the allocation between within-bank and cross-bank edges controllable. The configured within-bank share remains a target for the base graph rather than a guarantee for the transaction records subsequently realized by the simulator.

The adaptation to the transaction simulation modifies how normal transactions are realized. In AMLSim, normal transactions form the base-

line transaction activity generated by normal transaction models, while laundering activity is introduced through separately configured patterns. The normal transaction models assign groups of accounts to transfer structures such as fan-in, fan-out, and periodic transfers. In the original implementation, however, a model could select sender or recipient accounts from the complete incoming or outgoing neighbor lists of the active account, including accounts outside its assigned group [27]. Consequently, the group structure defined during graph and model construction was not consistently reflected in the realized transaction records. This mismatch weakened the correspondence between the configured normal transaction models and the generated transaction structure. We modified the six normal transaction model types to prefer eligible members of their assigned account group. Group membership was introduced as a probabilistic preference rather than a hard restriction, preserving the models’ ability to select other connected accounts. The adaptation therefore makes realized normal transactions follow the configured model groups more closely without confining all baseline activity to those groups.

A further adaptation to graph and model construction addresses account participation in the generated transactions. AMLSim executes each normal transaction model through a designated main account. Within that model, the other group members participate only if selected as sender or recipient accounts [27]. Membership in an account group therefore does not ensure participation in a realized transaction. In an early preliminary generation with 100,000 accounts, 59,984 accounts appeared in at least one transaction, while 40,016 never appeared and consequently formed isolated nodes in the resulting transaction graph. Because these nodes provide no relational information for graph learning, we introduced a coverage pass for accounts without a main-account role. The pass assigns each such account its own one-shot normal model, meaning a model in which that account acts as the main account for one scheduled transaction. For that transaction, the pass preferentially selects a same-bank partner with few existing model assignments and low graph degree, adding a directed fallback edge only if no suitable outgoing connection exists. With the transaction-simulation settings held constant, the coverage pass increased participation to 99,968 accounts and reduced the isolated count to 32.

Although the coverage pass established near-complete participation, 45,798 accounts still appeared in exactly one transaction and hence received only minimal relational context. Rather than increasing activity globally, we added a targeted top-up for accounts with very low base-graph degree. The top-up assigns a limited number of additional one-shot main roles to these accounts. In the resulting preliminary generation, 99,991 accounts participated in at least one transaction, only nine remained isolated, and the number appearing in exactly one transaction decreased to 8,563.

Taken together, the adaptations modify specific account-assignment

and selection steps rather than replacing AMLSim’s broader simulation framework. The existing normal transaction models and laundering typologies, together with the temporal simulation and output stages, remain in use. This preserves the generator’s broader design while providing the structural control and account coverage needed to produce datasets suitable for the multi-bank graph-learning experiments.

Dataset selection followed an iterative calibration procedure in which candidate configurations were evaluated through preliminary generations of 100,000 accounts and corresponding audits. These evaluations informed decisions about generation with one million accounts. Among other properties, the audits assessed overall account participation, connectivity, within-bank and cross-bank transaction structure, the prevalence of low-activity accounts, and the share of SAR-labeled accounts. Configurations that did not jointly satisfy the intended structural targets and practical requirements were adjusted and evaluated again. The calibration progressed from establishing controlled bank structure to resolving widespread account inactivity and then refining the bank-flow balance and low-activity tail. The principal preliminary calibration stages are summarized in Appendix A.1.2. Calibration continued after a structurally suitable configuration had been obtained, but subsequent targeted changes did not improve the overall audit profile sufficiently to justify replacing it. The primary symmetric three-bank dataset presented in Section 5.1.1 was therefore generated from the configuration that best satisfied the combined selection criteria, rather than from the last candidate evaluated.

Beyond this structural calibration, AMLSim’s parameters permit controlled variation of client distributions. The available parameters allow variation in client populations, initial-balance ranges, laundering-pattern allocations and amounts, and the global balance between within-bank and cross-bank edges. They do not support bank-specific degree distributions, normal-model mixes, normal-transaction amount or timing regimes, or bank-specific within- and cross-bank routing preferences. This permits joint bank-level heterogeneity under a shared normal-transaction process rather than independent bank-specific simulations.

4.1.2 Dataset Validation and Acceptance

Completing a simulation did not by itself establish that its output was suitable for the experimental setting. The realized transaction properties could deviate from their configured targets, and converting the resulting data into learning inputs presented a separate risk of inconsistency. We therefore used a staged acceptance process in which preliminary outputs informed generator calibration, while each retained full-scale output was audited first as generated account and transaction data and again in the graph representation used for model training.

Defining the account labels required a specific choice between two AMLSim outputs. The pattern-membership table records all accounts assigned to generated alert patterns and indicates whether each pattern is SAR-designated. We configured AMLSim to generate only SAR-designated patterns, so the complete membership recorded in this table defines the ground-truth positive class. The transaction-derived SAR roster is reconstructed from laundering transactions realized during simulation and can omit a designated member when none of that account’s laundering transactions appear in the output. We therefore used the complete SAR-designated pattern membership as the label source and the transaction-derived roster as a consistency check.

Table 4.3 organizes the remaining audit around the questions that determined whether an output could support the later experiments. Short-cut diagnostics use the area under the precision–recall curve (AUPR).

The validation checks were interpreted jointly rather than reduced to one fixed acceptance threshold. Account participation and connectivity established whether most accounts received relational context, while the comparison between transaction records and unique directed account pairs distinguished broad graph coverage from repeated transfers over the same relations. Directional bank flows and laundering-pattern composition then showed whether this structure extended across banks. Configurations were revised or rejected when these checks exposed substantial isolation, fragmented graphs, or bank flows that did not support the intended multi-bank setting.

At the learning-pipeline stage, validation covered the graph-construction, feature-generation, and data-splitting procedures described in Sections 4.2.1, 4.2.2, and 4.2.3. We verified that preprocessing preserved account and label correspondence, produced well-formed feature matrices, and kept the evaluation partitions disjoint. For the asymmetric datasets, we screened for simple shortcuts by using each bank’s training-positive rate and individual feature values, in both increasing and decreasing order, as prediction scores. We compared their AUPR with the observed centralized GCN performance.

Appendix A.2 records the audit coverage and selected acceptance indicators for the primary dataset. Taken together, these checks establish the retained datasets’ internal consistency and suitability for the intended experiments without implying structural correspondence to real banking networks.

Table 4.3: Validation framework applied during dataset calibration and final acceptance.

Dimension	Validation question	Principal evidence
Output integrity	Are accounts, transactions, bank assignments, and labels internally consistent?	Unique account identifiers, resolvable transaction endpoints, and consistent bank assignments
Relational structure	Does the output provide sufficient graph context across the account population?	Account participation, low-activity counts, repeated interactions, and weak connectivity; comparative improvements in account coverage rather than transaction volume alone
Multi-bank structure	Does the network retain the intended structure within and across banks?	Per-bank populations compared with configured bank sizes; directional flows and realized within-/cross-bank transaction shares compared with the intended interaction pattern
Laundering structure	Are the positive class and generated laundering patterns represented across the banks?	Positive-account rates per bank and bank-restricted/cross-bank pattern composition checked for consistency with configured pattern allocations and sizes
Learning-pipeline integrity	Is the source structure preserved during preprocessing?	One-to-one account-to-node correspondence, exact source-label agreement, valid graph edges, feature-shape and value checks, and zero split overlap with complete node coverage
Shortcut diagnostics	Is either bank identity or an individual feature strongly predictive of the labels?	Bank-only and individual-feature AUPR, complemented by binary-feature agreement with labels

4.2 Graph Construction and Feature Representation

4.2.1 Nodes, Directed Account-Pair Edges, and Labels

Prior AML research has used graph representations to detect illicit activity in transaction data [13]. For the account-level prediction objective considered here, we converted each accepted dataset into a directed transaction graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each account corresponds to exactly one node in $\mathcal{V}$, and $N = |\mathcal{V}|$ denotes the number of account nodes.

Multiple transactions may occur from the same sender to the same recipient. To retain this repeated-interaction information without constructing parallel edges, we consolidate all transaction records with the

same ordered pair of accounts into one directed account-pair edge. For $u, v \in \mathcal{V}$, let n_{uv} denote the number of individual transaction records from account u to account v . We distinguish between repeated-transaction and unit edge weighting:

$$\begin{aligned} \mathcal{E} &= \{(u, v) \in \mathcal{V} \times \mathcal{V} \mid n_{uv} > 0\}, \\ w_{uv} &= \begin{cases} \log(1 + n_{uv}), & \text{repeated-transaction weighting,} \\ 1, & \text{unit weighting.} \end{cases} \end{aligned} \quad (4.1)$$

Direction remains explicit, so $u \rightarrow v$ and $v \rightarrow u$ are distinct relations. The logarithmic formulation compresses the range of transaction counts while preserving their order, whereas unit weighting retains the same directed topology without varying edge influence by transaction frequency. Transaction amounts are not used as pair-level edge weights; they instead contribute to the node-level features described in the following subsection.

Following the label semantics established in Section 4.1.2, the resulting learning problem is binary node classification. Nodes representing SAR-labeled accounts form the positive class, while all remaining account nodes form the negative class. Let $\mathbf{y} \in \{0, 1\}^N$ denote these node labels, where $y_i = 1$ for a SAR-labeled account and $y_i = 0$ otherwise.

4.2.2 Node Features

For node-classification training and inference, each account node requires a fixed-length numerical representation as model input. This representation consists of node features that describe selected characteristics of the account and its transaction context. We construct 36 features, organized into eight groups according to the information they capture.

The first four groups cover account and transaction behavior through the initial balance, transaction activity, counterparty breadth, and flow balance. Transaction-activity features use transaction counts and amount distributions to characterize the volume and variability of incoming and outgoing transfers. Counterparty-breadth features describe how widely an account interacts, while flow-balance features compare incoming and outgoing activity and the net movement of funds. Incoming and outgoing transactions are treated separately throughout to preserve differences between receiving and sending behavior.

The remaining four groups derive from each account’s relationships in the directed account-pair graph. Pair topology captures relationship breadth and preserves information about repeated transactions. Measures of repeat interaction distinguish activity concentrated among recurring counterparties from activity spread more broadly. Beyond individual counterparties, bank mixing captures the extent of cross-bank activity. Finally, neighbor activity describes how active and well connected an ac-

count’s direct counterparties are. Appendix A.3 provides the complete feature definitions.

To prevent direct target leakage, the feature set excludes SAR flags and fields identifying alerts or laundering patterns. Bank assignments define client membership and allow the bank-mixing features to distinguish within-bank from cross-bank activity.

Feature values that were unavailable because an account had no relevant activity were set to zero. Selected count, amount, topology, and intensity features were transformed as $x \leftarrow \log(1 + x)$ to compress large values while preserving their order. The resulting values form the matrix $\mathbf{X}^{\text{eng}} \in \mathbb{R}^{N \times 36}$, with one row per account node and one column per feature. This feature set supports the centralized and initial federated analyses. Section 4.5 later revisits which features remain available under stricter feature-observability assumptions. The next subsection defines the node partitions and the train-only standardization applied to $\mathbf{X}^{\text{eng}}$.

4.2.3 Data Splits and Feature Scaling

To separate model fitting, model selection, and final evaluation while preserving representation of the minority positive class at each stage, we partitioned the account nodes into training, validation, and test sets using a class-stratified random split. Positive and negative node indices were shuffled and divided separately into approximately 60% training, 20% validation, and 20% test nodes before the class-specific subsets were recombined. The resulting partitions are disjoint and cover the complete node set:

$$\mathcal{V} = \mathcal{V}_{\text{tr}} \dot{\cup} \mathcal{V}_{\text{val}} \dot{\cup} \mathcal{V}_{\text{te}}. \quad (4.2)$$

The split itself removes neither nodes nor edges. Each model still operates on the graph constructed for its configuration, while the partitions determine how node labels are used. Only training labels contribute to model fitting, validation labels support model selection, and test labels are used for reporting. This follows the transductive setting described in Section 2.3.2.

Because the engineered features span substantially different numerical ranges, we standardized them to have zero mean and unit variance over the training nodes. Centering and scaling place the inputs on comparable numerical scales and support more efficient gradient-based optimization [73]. To prevent validation and test data from influencing this preprocessing step, the required statistics were estimated exclusively from the training nodes [74]. For feature f , let μ_f^{tr} and σ_f^{tr} denote the training-node mean and population standard deviation. The standardized model input is

$$x_{if} = \frac{x_{if}^{\text{eng}} - \mu_f^{\text{tr}}}{\sigma_f^{\text{tr}}}. \quad (4.3)$$

If $\sigma_f^{\text{tr}} = 0$, it is replaced by one. Applying the fitted transformation to every node yields the standardized feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$, where F denotes the number of active features. Validation and test values are thereby placed on the training-derived scale without contributing to its estimation.

Splitting and standardization are applied globally before accounts are assigned to bank clients. Centralized and federated configurations therefore use the same node assignments and feature scale, while each federated client trains only on the training nodes it owns. This common preprocessing keeps the model comparisons aligned. The resulting graph, standardized features, labels, and partitions form the common input to the graph-learning models described next.

4.3 Centralized Graph-Learning Model

The centralized model provides the full-graph reference for the federated comparisons summarized in Figure 4.1. It receives the standardized node features and global directed transaction graph defined in Section 4.2 and treats that graph as a static representation of activity over the generated observation period. This permits propagation across bank boundaries but does not model transaction order. GCN-family models have previously been evaluated for node-level AML detection on real transaction graphs [17], while later work reported effective graph-learning results across several real and synthetic AML datasets [16]. Together, these studies provide domain precedent for learning jointly from node attributes and transaction relationships.

We select a standard GCN as a controlled model base for two related reasons. Its neighborhood aggregation directly represents relational information among connected accounts, while the same propagation structure provides a common mathematical basis for the FedGCN-inspired treatment of cross-client edges [24]. Keeping this model base fixed allows the comparison to focus on changes in graph access and federation rather than differences between model families. As a diagnostic, we also use logistic regression, which applies a sigmoid function to a learned linear combination of the input features plus a bias term. It uses the same standardized features and node partitions but performs no adjacency-based propagation. This non-propagating feature baseline distinguishes predictive information already encoded in the engineered features from the additional value obtained through message passing.

We use the GCN propagation rule described in Section 2.3.2 [14]. The principal centralized references use the unit edge weighting defined in Equation (4.1), while matched runs using repeated-transaction weighting are retained for comparison. Under the target-row, source-column convention, the selected adjacency matrix has entries $a_{v,u} = w_{uv}$ for a

directed edge $u \rightarrow v$ and zero otherwise. Applying the normalization in Equation (2.3) yields the propagation matrix $\mathbf{P}$.

We initialize $\mathbf{H}^{(0)} = \mathbf{X}$ and apply the GCN layer update given in Equation (2.5). In the hidden layers, $\phi = \text{ReLU}$ is followed by dropout, which randomly suppresses activations during training to reduce co-adaptation and limit overfitting [75]. The output layer applies neither ReLU nor dropout and returns one logit o_i for each account node.

To support controlled calibration, the model implementation parameterizes graph-convolution depth, hidden width, dropout, learning rate, weight decay, and training duration. We tuned these settings against validation performance rather than retaining generic implementation defaults. This process selected three layers and hidden width 128 as the common architecture across the principal comparisons, yielding $F \rightarrow 128 \rightarrow 128 \rightarrow 1$. The primary-dataset reference uses dropout 0.2, whereas the references for the moderately and strongly asymmetric datasets use dropout 0.1. All three use learning rate 10^{-2} and weight decay 10^{-4} . The corresponding calibration evidence and final training budgets are reported in Section 5.2.

Training is performed in full batches so that every forward pass uses all nodes and edges of the global graph. At each epoch, the model computes logits for all nodes, restricts the objective to $\mathcal{V}_{\text{tr}}$, backpropagates the resulting gradients, and updates its parameters. Because SAR-labeled accounts form the minority class, an unweighted objective would allow negative accounts to dominate the loss. We therefore derive the positive-class weight

$$\rho^+ = \frac{\sum_{i \in \mathcal{V}_{\text{tr}}} (1 - y_i)}{\sum_{i \in \mathcal{V}_{\text{tr}}} y_i} \quad (4.4)$$

from the training partition and minimize the class-weighted binary cross-entropy

$$\begin{aligned} \mathcal{L} &= -\frac{1}{|\mathcal{V}_{\text{tr}}|} \sum_{i \in \mathcal{V}_{\text{tr}}} [\rho^+ y_i \log p_i + (1 - y_i) \log(1 - p_i)], \\ p_i &= \text{sig}(o_i) = \frac{1}{1 + \exp(-o_i)}. \end{aligned} \quad (4.5)$$

The implementation evaluates the numerically stable logit form of this objective.² Optimization uses Adam [77], which adapts parameter updates using running estimates of the first and second gradient moments. Weight decay regularizes the trainable parameters during these updates. The diagnostic logistic-regression model likewise uses inverse-frequency class weights. Section 4.6.2 defines checkpoint selection and the subsequent conversion of logits into reported predictions.

The resulting centralized model assumes access to the global graph, standardized feature matrix, and training labels. It therefore provides

²Implemented using PyTorch 2.5.1’s `BCEWithLogitsLoss` [76]; the [versioned official documentation](#) describes the combined sigmoid and binary cross-entropy formulation.

the reference against which the effects of bank ownership and restricted graph access can be assessed. The following section retains the prediction task and common three-layer model base while replacing centralized graph access and optimization with bank-owned client graphs, local training, model aggregation, and controlled cross-client information exchange.

4.4 Federated Graph Learning

4.4.1 Federated Setting and Cut-Edge Baseline

As outlined in Section 2.4, federated learning allows multiple clients to train a shared model while keeping their respective data local. In the multi-bank instance considered here, each bank acts as one client, and the account nodes of the global transaction graph are partitioned by bank affiliation. With $\mathcal{K} = \{1, \dots, K\}$ denoting the set of clients, $\mathcal{V}_k$ contains the accounts at bank k . The client-owned node sets form a partition of the global node set:

$$\mathcal{V} = \bigcup_{k \in \mathcal{K}} \mathcal{V}_k. \quad (4.6)$$

This bank-based ownership remains fixed across all federated comparisons. From each ownership set, we construct the induced client graph

$$\mathcal{G}_k = \mathcal{G}[\mathcal{V}_k] = (\mathcal{V}_k, \mathcal{E}_k), \quad \mathcal{E}_k = \{(u, v) \in \mathcal{E} \mid u, v \in \mathcal{V}_k\}. \quad (4.7)$$

It contains every account at bank k and precisely those directed account-pair edges whose endpoints both belong to that bank. This graph provides the local topology available without a cross-client exchange of graph information.

As in conventional federated learning, the clients interact through a coordinating server that maintains the shared model and orchestrates its iterative optimization.

Within this common client-server structure, we consider three configurations that differ in the cross-bank information available during local model computation. The first configuration is the cut-edge baseline, which most closely follows conventional federated learning: clients exchange model states but no node features or neighborhood aggregates for cross-bank propagation. In this baseline, each client applies the GCN architecture introduced for the centralized reference to its local graph $\mathcal{G}_k$. The round-based local optimization, model-state aggregation, and global evaluation described below form the common training procedure for all three configurations. The other two configurations additionally precede these rounds with a pre-training phase that exchanges aggregated feature information across bank boundaries. The following subsections define this phase and distinguish the cross-bank context provided by its two variants.

All three federated configurations use the unit edge weighting introduced in Section 4.2.1. This matches the edge-weighting formulation of the principal centralized references and holds edge weighting fixed, while the available cross-client graph information varies.

Training proceeds in rounds using federated averaging (FedAvg) [22]. At the start of round r , the server sends the current global parameters $\theta^{(r)}$ to every client. Each client initializes its model from this state and performs the configured number of full-batch local epochs on its client graph. The class-weighted objective from Equation (4.5) is evaluated only over the client’s training accounts, from which the positive-class weight is also derived. Local optimization again uses Adam, with a fresh optimizer state initialized in each round. The server combines the returned client states according to Equation (2.8), using weights derived from the number of training accounts each client owns. For client k , let $\mathcal{V}_{k,\text{tr}} = \mathcal{V}_k \cap \mathcal{V}_{\text{tr}}$ denote these accounts. The aggregation weight is

$$\alpha_k^{(r)} = \frac{|\mathcal{V}_{k,\text{tr}}|}{\sum_{q \in \mathcal{K}} |\mathcal{V}_{q,\text{tr}}|}. \quad (4.8)$$

This weighting gives each client influence in proportion to its number of training accounts rather than weighting differently sized clients equally. To support controlled comparisons, the federated implementation parameterizes the number of rounds and local epochs. The retained settings are reported with the corresponding experiments in Chapter 5, while Section 4.6.2 defines checkpoint selection.

The shared model is evaluated after every round. Each client predicts only its validation and test accounts, and the evaluator combines these predictions by their global node identities. Metrics are computed once over the combined predictions rather than averaged from separate client-level scores. This preserves the global evaluation objective used for the centralized model.

The cut-edge baseline makes the graph-specific limitation described in Section 1.2 explicit. The following protocol addresses this missing cross-client information before federated training begins.

4.4.2 Cross-Client Feature Pre-Aggregation

The difference between the cut-edge baseline and the centralized GCN becomes concrete during the first feature-propagation step. While the centralized model propagates information over the global graph, each client is restricted to its local graph and therefore misses contributions across bank boundaries.

The operation can be expressed using the feature matrix $\mathbf{X}$ and propagation operator $\mathbf{P}$. For a directed edge $s \rightarrow t$, feature information flows from s to t ; we therefore call s the source node and t the target node. Row t of $\mathbf{P}$ assigns a normalized coefficient $P_{t,s}$ to every direct incoming

source node, including the target itself through the added self-loop. Multiplication with $\mathbf{X}$ applies these coefficients to the feature row $\mathbf{x}_s$ of each source node and sums the weighted rows for every target simultaneously:

$$\mathbf{Z} = \mathbf{P}\mathbf{X} \in \mathbb{R}^{N \times F}, \quad \mathbf{z}_t = \sum_{s \in \mathcal{V}} P_{t,s} \mathbf{x}_s. \quad (4.9)$$

Each row $\mathbf{z}_t$ is therefore the one-hop feature aggregate for target node t . Within each graph-convolution layer, the cut-edge model still performs one-hop aggregation on its local graph, combining the target’s own features with those of its direct incoming same-bank neighbors. It cannot include feature contributions from accounts at other banks and therefore does not reproduce the corresponding global aggregate row. Crucially, $\mathbf{P}$ and $\mathbf{X}$ remain fixed during model training. The global aggregate rows therefore do not depend on learned parameters and can be constructed once before the federated rounds.

The FedGCN paper formalizes this idea as a dedicated pre-training communication round [24]. Each client sends its accumulated local feature contributions to the central server. The server sums these contributions and returns the aggregate rows required by each client. After this pre-training step, no further graph-data exchange is required, and subsequent communication is limited to the model states exchanged through FedAvg.

The examined FedGCN reference implementation does not realize this paper-level protocol. Instead, it expands each client’s local computation graph to the selected hop distance and supplies the corresponding adjacency and raw feature rows, including those of foreign nodes. This data path is incompatible with our bank-local ownership boundary: a client must not receive raw features of foreign accounts or topology beyond the source–target relations required for its own contribution [28].³

The later FedGraph library, developed by an overlapping author team as a broader federated graph-learning framework, provides a closer implementation of the paper-level design [29]. Its clients nevertheless receive hop-expanded graph structure and construct contributions in the global node space. The server forms the complete feature aggregate before selecting the rows required by each recipient. Uploads therefore scale with the global graph, lack recipient-specific scope, and require clients to receive more topology than is needed to construct their feature contributions [30].⁴

³This implementation comparison is based on commit [378438d](#) of the official FedGCN repository. Lines 201–243 of `src/utils.py` expand each client partition and construct its induced adjacency. In lines 200–215 of `src/fedgc_n_run.py`, the resulting adjacency and raw feature slice are passed to the trainer.

⁴This implementation comparison refers to commit [9bea6d9](#) of the official FedGraph repository. Lines 314–344 of `fedgraph/trainer_class.py` construct feature contributions in the global node space; lines 402–444 of `fedgraph/federated_methods.py` aggregate these contributions and select recipient rows.

Our implementation responds to these limitations through a more explicit division between the information retained by the coordinating server and the information made available to bank clients.⁵

The resulting pre-training exchange distinguishes two client roles. A client acts as a *contributing client* when features from its accounts contribute to an aggregate row required by a recipient client; it acts as a *recipient client* when its local model requires an aggregate row containing feature contributions from accounts at another bank. These roles are not mutually exclusive. In our experiments, every client performs both because each bank has incoming and outgoing cross-bank relationships.

Compared with its role in the cut-edge baseline, the coordinator assumes additional responsibilities during this pre-training phase. Figure 4.4 shows the resulting pre-training sequence.

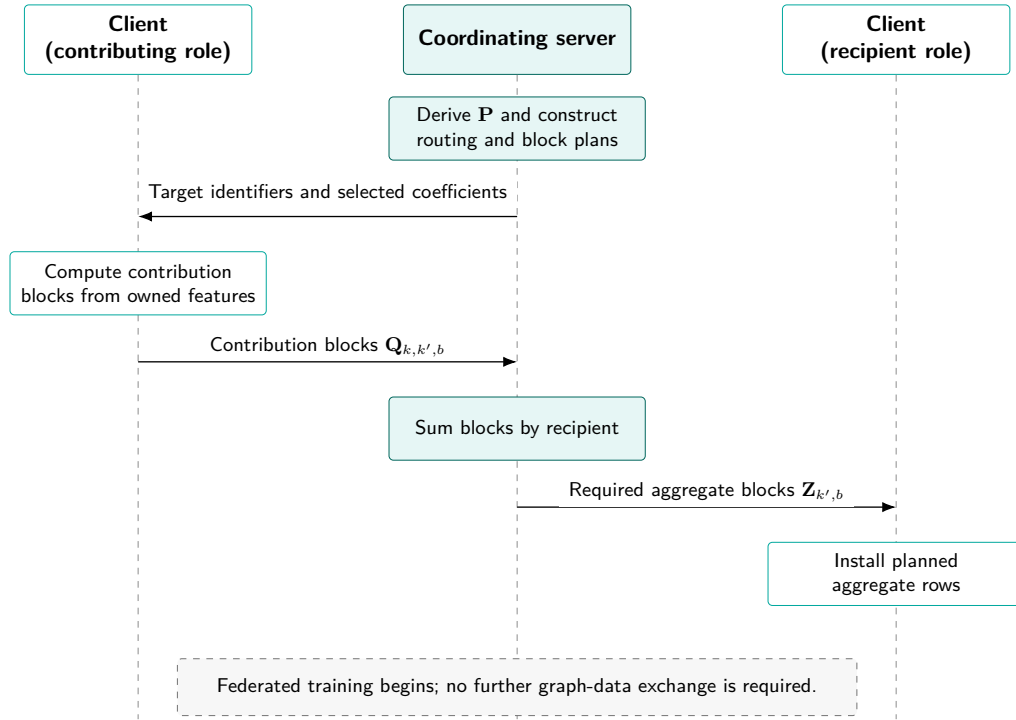


Figure 4.4: Sequence of the pre-training protocol. The contributor and recipient lanes represent roles that can be performed by the same client. Federated training begins after the planned aggregate rows have been delivered.

Within this sequence, the coordinator derives the normalized propagation operator from the global adjacency and determines which aggregate

⁵The protocol was implemented independently; no code from either reference repository was imported.

rows each recipient requires. It then provides each contributing client only with the target identifiers and propagation coefficients needed to form its part of those rows. The one-shot aggregation principle is thereby implemented as an explicit recipient-routed protocol that limits every role to the inputs and outputs required for its assigned computation.

Formally, Equation (4.9) expresses each aggregate row as a sum over its source nodes. Because these source nodes can be distributed across banks, we decompose the sum according to client ownership. For contributing client k and target node t , $\mathbf{c}_{k,t}$ denotes the *contribution row* formed from t 's source nodes in $\mathcal{V}_k$:

$$\mathbf{c}_{k,t} = \sum_{s \in \mathcal{V}_k} P_{t,s} \mathbf{x}_s, \quad \mathbf{z}_t = \sum_{k \in \mathcal{K}} \mathbf{c}_{k,t}. \quad (4.10)$$

Summing the contribution rows across all clients reconstructs the required aggregate row and preserves the global normalization from Equation (2.3).

Reconstructing an aggregate row does not yet determine who should receive it. Different clients require different rows for their local computation. Sending the complete matrix $\mathbf{Z}$ to every client would expose unrelated aggregates and make every download scale with the global graph. The coordinator therefore records the aggregate rows needed by each recipient k' in a *row plan* $\mathcal{R}_{k'}$. Whether this plan contains only owned target rows or additional context rows depends on the one-hop or two-hop configuration defined in the following subsection.

To bound the number of rows handled in one message, the coordinator divides each ordered row plan into disjoint blocks $\mathcal{B}_{k',b}$:

$$\mathcal{R}_{k'} = \bigcup_b \mathcal{B}_{k',b}. \quad (4.11)$$

Each block contains a bounded set of target rows, and every row in the plan occurs in exactly one block. For block b , contributing client k combines its owned feature rows using the corresponding propagation coefficients:

$$\mathbf{Q}_{k,k',b} = \mathbf{P}[\mathcal{B}_{k',b}, \mathcal{V}_k] \mathbf{X}[\mathcal{V}_k, :]. \quad (4.12)$$

Here, $\mathbf{P}[\mathcal{B}_{k',b}, \mathcal{V}_k]$ selects the coefficients from accounts owned by k to the target rows in the block, while $\mathbf{X}[\mathcal{V}_k, :]$ selects the corresponding owned feature rows. The resulting matrix $\mathbf{Q}_{k,k',b}$ contains client k 's contribution to every target row in that recipient block.

The server combines the blocks associated with the same recipient and block identifier:

$$\mathbf{Z}_{k',b} = \sum_{k \in \mathcal{K}} \mathbf{Q}_{k,k',b} = \mathbf{Z}[\mathcal{B}_{k',b}, :]. \quad (4.13)$$

In the final term, $\mathbf{Z}[\mathcal{B}_{k',b}, :]$ denotes the rows of $\mathbf{Z}$ indexed by the block. The server returns this aggregate block only to recipient k' . Blocking

changes the message framing but not the aggregate values. If several recipients require the same target row, its contribution is uploaded separately for each recipient; this duplication remains part of the communication accounting.

Taken together, contribution construction, block aggregation, and *recipient-specific routing* define the information available to each role during the pre-training exchange. The coordinator holds the global topology, uses it to derive the routing plans, provides contributing clients with the selected coefficients required for their computations, and receives their contribution blocks. Each contributing client aggregates features from its owned nodes into the requested contribution blocks and returns them to the coordinator. The coordinator sends each recipient only the aggregate rows in its plan. Bank clients therefore receive neither raw feature rows from another bank nor aggregates unrelated to their local computation. Without additional protection, this arrangement places greater trust in the coordinator, which receives each client’s partial aggregate contributions before they are combined. These values are not raw node features; however, the coordinator also holds the topology and propagation coefficients needed to attempt inference about their inputs from the partial aggregates.

Compared with the examined FedGCN and FedGraph implementations [28, 30], recipient-specific routing and role-specific messages turn the paper-level exchange into a protocol with explicit information boundaries. These interfaces prevent bank clients from receiving raw foreign features or dense global-sized payloads and make the permitted information flows directly auditable. The current implementation enforces these boundaries in a local simulation rather than across separately deployed services. It therefore does not establish deployment-level process isolation.

Recipient-specific routing narrows the aggregate information disclosed to each bank. It does not, however, make the rows within that permitted view inherently private. The rows remain deterministic functions of source-node features and propagation coefficients. In particular, a row containing only one or a few source-node contributions provides little aggregation and may expose information about those inputs. OptimES warns that directly communicated FedGCN aggregates can leak private data. FedGCN itself identifies the extreme case of a node with only one cross-client neighbor and suggests adding differential privacy to the communicated data [24, 64]. The paper does not instantiate a calibrated mechanism for this case. Section 4.5 therefore introduces separate protections for pre-training transport and contribution-row release.

The following subsection defines how the one-hop and two-hop configurations select and use these routed aggregate rows.

4.4.3 One-Hop and Two-Hop Graph Context

The rows installed before training determine how cross-bank context enters each client’s local model. The one-hop and two-hop configurations request different subsets of $\mathbf{Z} = \mathbf{P}\mathbf{X}$ and, consequently, provide different cross-client context during local computation. Together with the cut-edge baseline, these configurations adapt FedGCN’s zero-hop, one-hop, and two-hop settings to the bank-partitioned transaction graph [24]. The hop labels describe the cross-client information delivered before training, not the total depth of subsequent local propagation. We use the full names in prose, with h1 and h2 serving as compact labels in equations, tables, and comparisons.

Recall that $\mathcal{R}_{k'}$ denotes the aggregate rows delivered to recipient k' . Since these rows already contain the initial graph propagation, the model does not repeat that step at its input. Its first trainable layer instead performs the linear transformation $\mathbf{Z}[\mathcal{R}_{k'}, :] \mathbf{W}^{(0)} + \mathbf{b}^{(0)}$ on the installed row matrix, after which ReLU and dropout are applied. Under the three-layer architecture used throughout the principal experiments, two graph-convolution layers remain. The graph on which these layers operate depends on whether one-hop or two-hop context is selected.

The one-hop configuration applies the delivery protocol defined in the preceding subsection to the nodes owned by recipient k' . Its row plan therefore requests one aggregate row for every node in $\mathcal{V}_{k'}$:

$$\text{h1:} \quad \mathcal{R}_{k'} = \mathcal{V}_{k'}. \quad (4.14)$$

The client graph retains the same node and edge structure as in the cut-edge baseline: it contains the owned nodes and the edges among them. What changes is their model input. The client installs $\mathbf{Z}[\mathcal{V}_{k'}, :]$, so each owned node is represented by the one-hop feature aggregate of itself and its direct incoming neighbors. When these neighbors include accounts at other banks, their feature contributions are retained without adding the foreign nodes to the local graph. The installed input rows therefore incorporate direct cross-bank context before training. The two remaining graph convolutions then propagate these enriched representations over the edges of the local graph.

The two-hop configuration extends the local computation graph with one additional row for each foreign node connected by a directed edge into an owned node. Let $\mathcal{I}_{k'}$ denote the owned nodes together with these direct cross-bank source nodes:

$$\begin{aligned} \mathcal{I}_{k'} &= \mathcal{V}_{k'} \cup \{s \in \mathcal{V} \setminus \mathcal{V}_{k'} \mid \exists t \in \mathcal{V}_{k'} : (s, t) \in \mathcal{E}\}, \\ \text{h2:} \quad \mathcal{R}_{k'} &= \mathcal{I}_{k'}. \end{aligned} \quad (4.15)$$

We term these additional rows *aggregate placeholders* because each stands in for a foreign source node s during local propagation while containing its

precomputed aggregate $\mathbf{z}_s$, rather than its raw features or label. Since $\mathbf{z}_s$ includes feature contributions from s and its direct incoming neighbors, each placeholder already represents one propagation step. Pre-training therefore still delivers rows of the one-hop aggregate $\mathbf{Z} = \mathbf{P}\mathbf{X}$. The additional two-hop context arises when the local model propagates a delivered placeholder row into an owned target. Aggregate placeholders have no active training, validation, or test mask and never enter local loss computation or the owned prediction population.

Figure 4.5 contrasts these two client inputs in an illustrative three-bank graph.

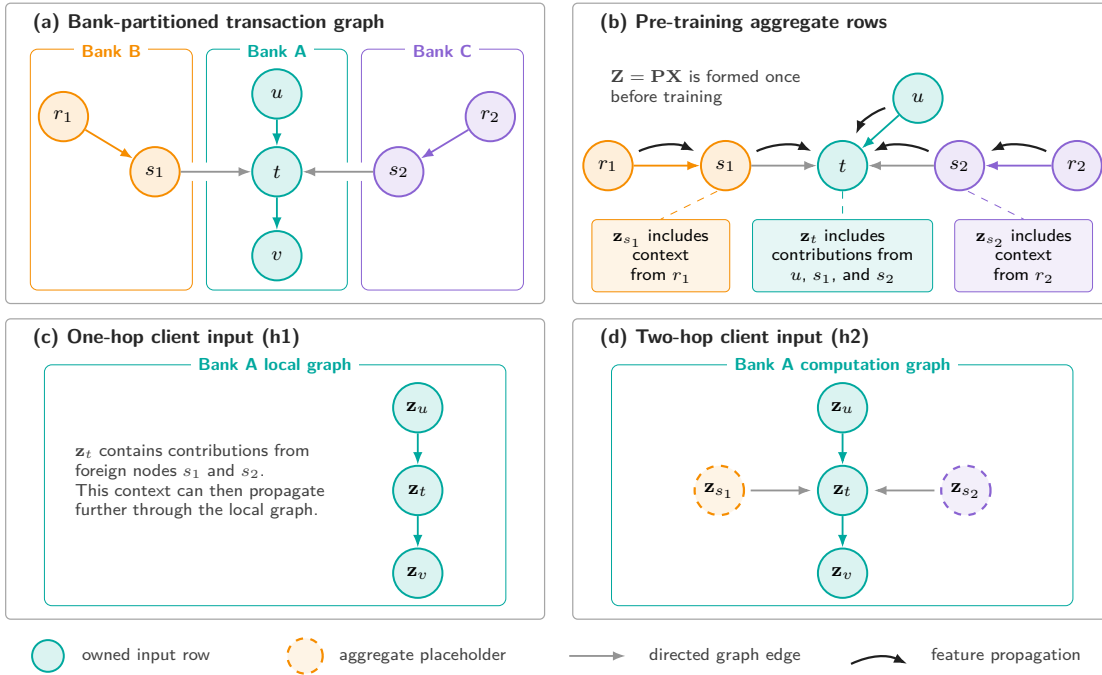


Figure 4.5: One-hop and two-hop client inputs for an illustrative three-bank graph. Pre-training forms $\mathbf{Z} = \mathbf{P}\mathbf{X}$ from the bank-partitioned graph. The one-hop configuration installs aggregate rows only for Bank A's owned nodes, whereas the two-hop configuration also installs aggregate rows of foreign source nodes as placeholders.

Table 4.6 summarizes the resulting distinction among all three federated configurations.

The hop labels refer to the depth of cross-client context available during training, not to the depth of the model's local receptive field. The centralized three-layer GCN performs all three propagation steps on the global graph. In the federated two-hop configuration, the installed aggregate rows provide the cross-client context defined above, after which

Table 4.6: Graph information and model computation in the three federated configurations.

Configuration	Delivered aggregate rows	Computation graph	Cross-client context
Cut-edge	None	Owned nodes	None; propagation remains within the bank
One-hop (h1)	$\mathbf{Z}[\mathcal{V}_{k'}, :]$	Owned nodes	Direct cross-bank neighbors
Two-hop (h2)	$\mathbf{Z}[\mathcal{I}_{k'}, :]$	Owned nodes and aggregate placeholders	Direct cross-bank neighbors and their incoming neighbors

two graph-convolution layers propagate over the client computation graph. This allows imported context to travel over local edges, but does not reconstruct the complete three-hop neighborhood of the global graph because aggregate placeholders do not receive deeper foreign hidden-state updates.

We nevertheless retain the common three-layer architecture because it outperformed the two-layer alternative for the centralized model as well as in each of the three federated configurations. Sections 5.2 and 5.3 report the corresponding model comparisons.

The one-hop and two-hop configurations thus provide clients with different depths of cross-client context through the routed aggregate rows. The following section addresses this exchange as one of several protection surfaces and introduces privacy mechanisms that build on the ownership and recipient-specific routing structure established here.

4.5 Privacy Enhancements

4.5.1 Threat Model and Protection Surfaces

The preceding section established the bank clients and coordinating server that participate in federated training. In the cut-edge baseline, clients exchange only model states through FedAvg; no additional graph information is shared across banks. The one-hop and two-hop configurations add a pre-training exchange of aggregate feature rows. For the privacy analysis, we distinguish the two communication lanes highlighted in Figure 4.1: pre-training aggregation and federated model training.

Figure 4.7 shows the plain protocol and establishes the common visual structure retained throughout this section. Each arrow represents an exchange between one bank client and the coordinating server; the same exchange occurs separately for every participating bank.

To evaluate the privacy–utility–efficiency trade-offs posed in RQ3 without prescribing one operational deployment, we implement independently configurable protections for selected exposures in each lane. Accordingly,

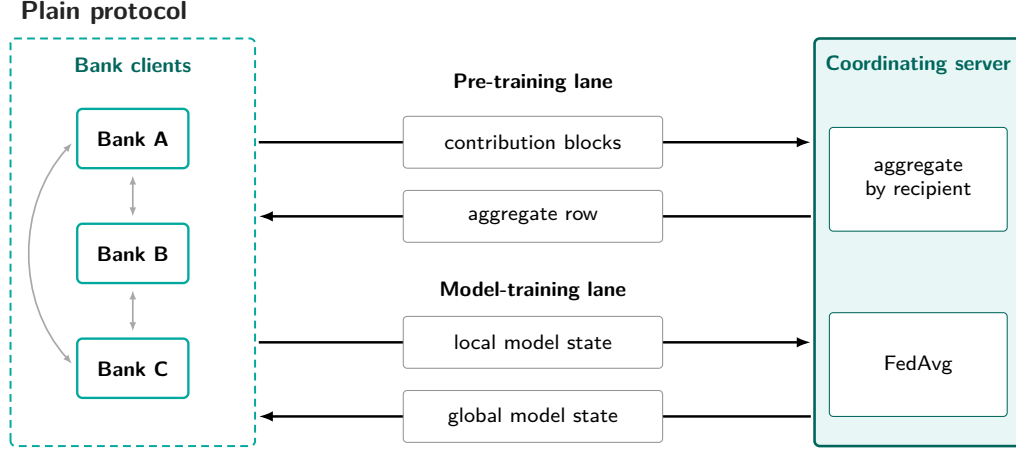


Figure 4.7: Plain communication across the pre-training and model-training lanes. The arrows summarize exchanges performed separately between each participating bank and the coordinating server.

we use homomorphic encryption to hide pre-training contribution values from the coordinator and differential privacy to protect individual contributions within aggregate rows released to recipients. Pairwise masking separately hides individual plaintext model updates from the coordinator.

The common behavioral assumption is that all modeled parties follow the prescribed protocol. Bank clients retain their raw account data and are legitimate observers of the aggregate rows routed to them. The coordinating server is trusted to hold the global topology and execute the protocol correctly. From this topology, it derives the normalized propagation coefficients and routing plans. Trusting the coordinator with these tasks does not require granting it plaintext access to every payload it processes. It is therefore treated as honest-but-curious regarding the payload values visible in its role.

This model places substantial trust in the coordinator. Whether that role could be assigned to a regulator or another jointly trusted institution, and which combination of mechanisms remains operationally viable under the measured costs, is considered in Chapter 6. The methodology therefore defines the evaluated protection boundaries without prescribing a production deployment.

The pre-training lane follows the recipient-specific routing protocol from Section 4.4.2, which limits each bank to the rows required for its local computation. This restriction reduces direct disclosure, but it does not make the routed rows inherently private. Each row remains a deterministic function of the contributing features and propagation coefficients, and an aggregate formed from only a few foreign contributions may re-

4. METHODOLOGY

Table 4.8: Protection surfaces in the pre-training and model-training information flows. Visibility refers to the logical protocol roles in the evaluated local simulation.

Information object	Plain-protocol visibility	Applied mechanism	Remaining visibility
Contribution blocks	Plaintext values at the coordinator	Recipient-routed CKKS; local contribution-row DP	Block metadata, dimensions, and routes remain visible
Aggregate feature rows	Plaintext values at the coordinator and routed recipients	Local or central contribution-row DP	Intended recipients inspect the released rows
Individual model updates	Plaintext values at the coordinator	Pairwise-masked FedAvg	Masked updates and the aggregate update remain visible
Aggregate model update	Visible to the coordinator	None	Remains visible and is not protected by model-update DP

veal information about its inputs [24, 64]. Although recipients receive no raw foreign features, they may attempt to infer or reconstruct underlying feature information from the released aggregates. The release of these aggregates therefore forms one protection surface. Server-side aggregation forms another because the plain configuration allows the coordinator to inspect individual contribution blocks and relate their values to the known topology and propagation coefficients. Exact feature reconstruction is not guaranteed, but this combined visibility can support inference when only a few source nodes contribute to a row.

The model-training lane introduces a third protection surface. Each client trains on its local graph and submits the resulting model state for FedAvg. Without an additional mechanism, the coordinator receives every client state in plaintext before computing the aggregate state used in the next round. Pairwise masking changes this server-side visibility while leaving the aggregate model update available for continued training.

Table 4.8 summarizes the principal information objects, their visibility in the plain protocol, and the mechanism applied to each surface.

The privacy experiments also use a more restricted feature set based on stricter assumptions about the information available to each bank. From the 36 features defined in Section 4.2.2, we remove four neighbor-activity features whose construction requires centrally derived statistics about connected accounts. We refer to the resulting 32-feature representation as the *boundary-compliant feature set*. Appendix A.3 identifies the four excluded features.

The implementation enforces the stated ownership, message, and visibility boundaries through explicit protocol interfaces within a local sim-

ulation. It does not establish deployment-level process isolation or cover malicious behavior, collusion, network security, production key management, or client-dropout recovery. Within this scope, the first protected surface is the transport and server-side aggregation of pre-training contribution blocks.

4.5.2 Recipient-Routed CKKS Aggregation

In the plain pre-training exchange, each bank sends its contribution blocks to the coordinating server. Because these blocks are transmitted in plaintext, the server can inspect their values before combining them into the aggregate blocks returned to recipients. Homomorphic encryption allows the same aggregation to be performed directly on encrypted values without having to decrypt them.

We implement this path using CKKS through TenSEAL, which provides encrypted operations on real-valued vectors [78]. The contribution blocks already contain the locally weighted feature sums, so the encrypted computation requires only their element-wise addition. CKKS provides a packed numerical representation suited to this workload, with approximate rather than exact recovery of the aggregate values [31]. The integer-based alternatives discussed in Section 2.6.1 could also support this additive computation, but would require a suitable encoding of the real-valued contributions and control of their numerical range. Our choice therefore combines approximate real-vector arithmetic with the vector operations available in the selected implementation library.

Figure 4.9 shows how CKKS changes the pre-training lane while leaving model-training communication unchanged.

The bounded recipient-aligned blocks introduced in Section 4.4.2 become especially useful under encryption. Each block can be packed and processed as an independent CKKS vector. This bounds the number of scalar values encoded in each payload and provides a common unit for encryption, routing, and communication accounting. The block decomposition does not itself alter the underlying sum; small numerical differences can arise from CKKS encoding and arithmetic.

At the beginning of each run, a CKKS context containing the secret key is created for client-role operations, while the server receives the corresponding public context. Each contributing client encrypts and serializes its recipient-aligned blocks. The server reconstructs the ciphertexts under the public context, adds blocks with matching recipient and block identifiers, and routes each encrypted aggregate to its intended recipient. Client-role decryption then recovers the aggregate rows required for local model computation.

CKKS changes the representation in which the block aggregation from Equation (4.13) is performed. Instead of summing plaintext contribution

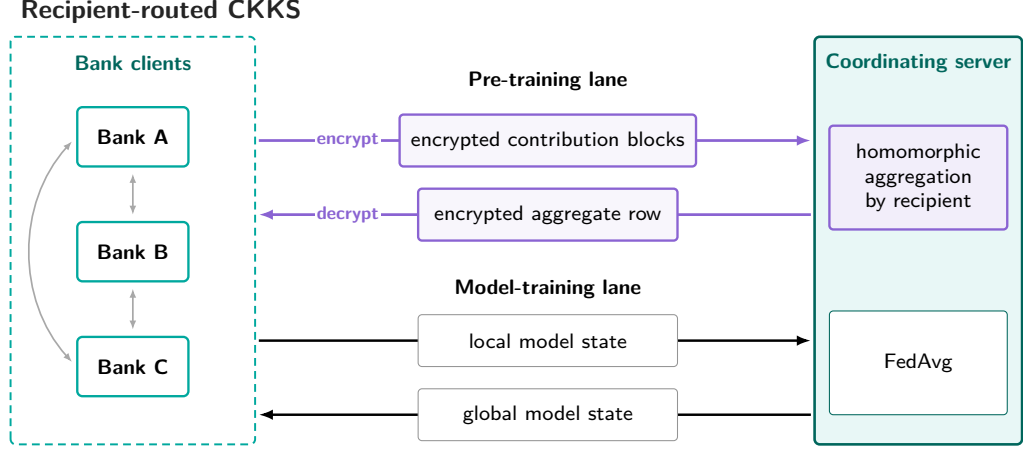


Figure 4.9: Recipient-routed CKKS aggregation. Contribution blocks are encrypted before upload, and aggregate rows remain encrypted until recipient-side decryption.

blocks, the server adds their ciphertexts:

$$\llbracket \mathbf{Z}_{k',b} \rrbracket = \sum_{k \in \mathcal{K}} \llbracket \mathbf{Q}_{k,k',b} \rrbracket, \quad \text{Dec}(\llbracket \mathbf{Z}_{k',b} \rrbracket) \approx \mathbf{Z}_{k',b}. \quad (4.16)$$

Here, $\llbracket \cdot \rrbracket$ denotes a CKKS ciphertext and Dec denotes client-role decryption using the secret key. The decrypted value approximates the plaintext aggregate because CKKS encodes real-valued vectors with finite numerical precision and may introduce small rounding errors during encrypted arithmetic.

We use one fixed CKKS configuration throughout the comparisons so that cryptographic parameter tuning does not become an additional experimental variable. Table A.10 in Appendix A.4 records the TenSEAL version, encryption parameters, and block size. Sections 4.6.3 and 4.6.4 define the approximation-error, memory, and ciphertext-traffic accounting, while the corresponding outcomes are reported in Chapter 5. Within the evaluated role boundary, the server has no secret key and cannot decrypt contribution values. The local simulation uses one shared client-role secret context per run and provides the server only with the corresponding public context. This arrangement supports evaluation of encrypted server-side aggregation, but does not model separate per-bank key provisioning or deployment-level process isolation. Separation among recipient banks therefore remains enforced by the routing protocol. Once an intended recipient decrypts an aggregate row, CKKS no longer protects its contents. Limiting what the released row can reveal requires a separate mechanism.

4.5.3 Contribution-Row Differential Privacy

Differential privacy protects a selected input by requiring statistically similar results when that input is modified. The parameters ε and δ specify the strength of this guarantee: ε bounds the relative change in output probabilities, while δ permits a small additive deviation from that bound. Smaller values provide stronger privacy [50]. In graph learning, the choice of protected input is consequential because neighborhood aggregation makes node representations interdependent: modifying one node or edge can affect several propagated values [68]. A formal claim must therefore define its protected input precisely. Our mechanism focuses on the additional data exchange introduced by pre-training, while model-update visibility is treated separately in the following subsection. The protected unit is one contribution row $\mathbf{c}_{k,t}$ from Equation (4.10), representing client k 's contribution to the aggregate feature row for target node t . Under *substitution adjacency*, two inputs are neighboring when one client's numerical contribution to a given target is replaced by another possible contribution. Only this value changes; all other contributions, the graph, and the communication plan are treated as fixed.

To bound the influence of one contribution and calibrate the required noise, every contribution row is first clipped to an L_2 -norm bound $C_{\text{clip}} > 0$:

$$\text{clip}_{C_{\text{clip}}}(\mathbf{c}) = \frac{\mathbf{c}}{\max(1, \|\mathbf{c}\|_2/C_{\text{clip}})}, \quad \bar{\mathbf{c}}_{k,t} = \text{clip}_{C_{\text{clip}}}(\mathbf{c}_{k,t}). \quad (4.17)$$

Rows below the bound remain unchanged, while larger rows are scaled to norm C_{clip} . Under substitution adjacency, the change between any neighboring pair of clipped contribution rows satisfies

$$\|\bar{\mathbf{c}}_{k,t} - \bar{\mathbf{c}}'_{k,t}\|_2 \leq \|\bar{\mathbf{c}}_{k,t}\|_2 + \|\bar{\mathbf{c}}'_{k,t}\|_2 \leq 2C_{\text{clip}}. \quad (4.18)$$

The implementation uses the upper bound $\Delta_2 = 2C_{\text{clip}}$ because either clipped row may be replaced by another.

The implementation uses the conventional classical Gaussian calibration [50]:

$$\sigma = \sqrt{2 \ln(1.25/\delta)} \frac{\Delta_2}{\varepsilon}, \quad 0 < \varepsilon \leq 1, \quad 0 < \delta < 1. \quad (4.19)$$

For settings outside the classical calibration range, we use the analytic Gaussian mechanism of Balle and Wang [51]. Its calibrated noise scale is denoted by

$$\sigma = \sigma_{\text{AGM}}(\varepsilon, \delta, \Delta_2). \quad (4.20)$$

The numerical calibration procedure is recorded in Appendix A.4.2. In both cases, each noise vector follows $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_F)$.

We evaluate the mechanism at two positions in the protocol. In the local variant, a contributing client clips and perturbs each row before block

construction and transport. In the central variant, clients clip their rows before transport, the server combines them, and one noise vector is added to the completed target row:

$$\mathcal{M}_{\text{local}}(\mathbf{c}_{k,t}) = \bar{\mathbf{c}}_{k,t} + \boldsymbol{\eta}_{k,t}, \quad \mathcal{M}_{\text{central}}(t) = \sum_{k \in \mathcal{K}} \bar{\mathbf{c}}_{k,t} + \boldsymbol{\eta}_t. \quad (4.21)$$

The same sensitivity bound applies at either position because replacing one contribution row changes only one clipped term in the completed aggregate. In either variant, perturbation preserves the row shape, so routing and local model computation remain unchanged. In the local variant, perturbed contribution rows enter the existing block aggregation. Noise is therefore added before transport, and its contribution-row guarantee covers both the server-visible payload and the aggregate later released to recipients. In the central variant, the server first aggregates clipped contributions and then perturbs the completed rows before release. This protects the recipient-facing release, while clipped contributions remain visible to the server under plaintext transport. Our implementation supports combining CKKS transport with either the local or central row-DP variant. In the local combination, clients perturb their contribution rows before encryption. In the central combination, the sampled noise is encrypted under the public context and added to the encrypted aggregate without server-side decryption.

Figure 4.10 shows where local and central perturbation enter the protocol under plaintext transport and when combined with CKKS.

When two-hop routing requires the same target row for multiple recipients, the local variant reuses each perturbed contribution row, while the central variant reuses each perturbed aggregate row. Noise is not resampled for each recipient.

Overall, the mechanism protects the information released through the pre-training exchange at the contribution-row level. Each released aggregate satisfies (ε, δ) -differential privacy with respect to substituting one bounded contribution. The parameters therefore bound how much the distribution of the released aggregate can differ between two such contributions, while the aggregate remains available for graph learning. Subsequent routing and model computation preserve this guarantee under the post-processing property of differential privacy [50]. Extending it to full node- or edge-level differential privacy for the underlying graph would require separate adjacency definitions and mechanisms and therefore falls outside the scope of this thesis.

Experimental Evaluation additionally uses diagnostic Gaussian perturbation as a noise-tolerance diagnostic without attaching a formal (ε, δ) guarantee. With the pre-training protection mechanisms defined, the following subsection turns to the individual model updates exposed during FedAvg.

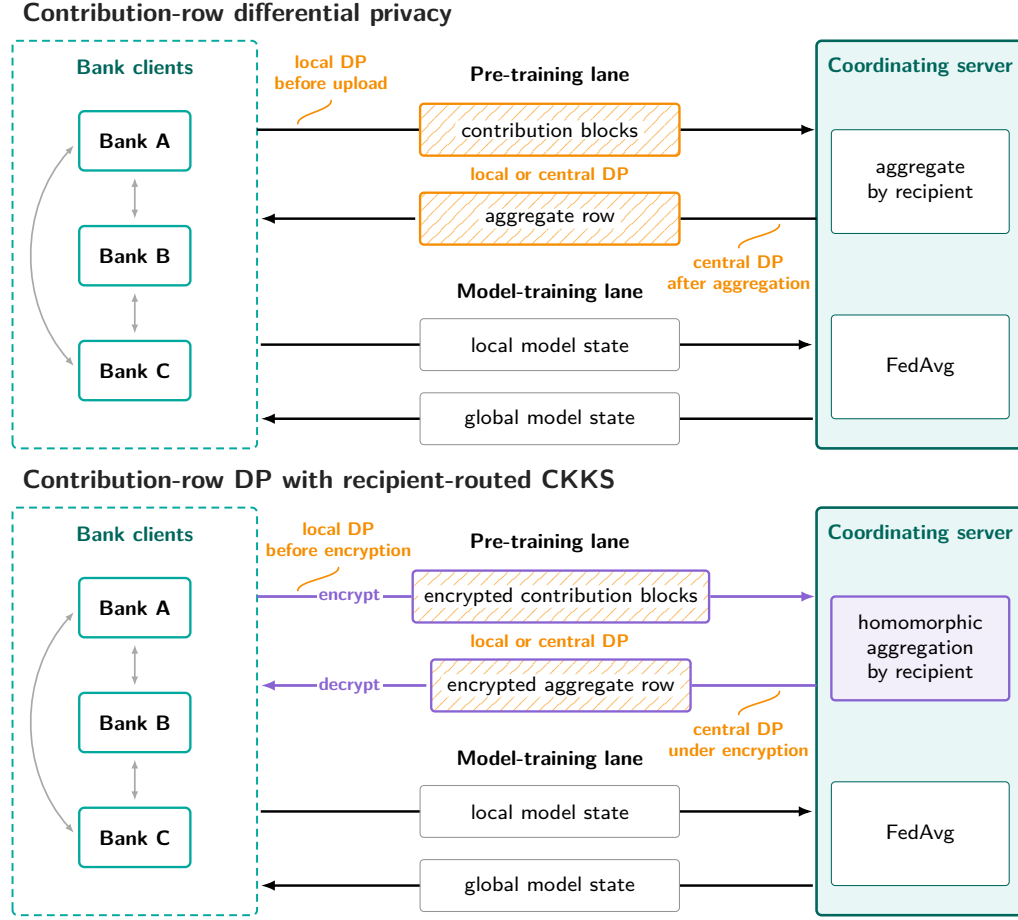


Figure 4.10: Contribution-row differential privacy under two transport configurations. The upper diagram shows differential privacy with plaintext transport, while the lower diagram combines the same mechanism with recipient-routed CKKS. Local and central perturbation are alternative placements and are not applied simultaneously.

4.5.4 Pairwise-Masked Federated Averaging

In the plaintext implementation of the FedAvg operation in Equation (2.8), the coordinator receives every participating client's returned model state before averaging and can derive its update from the current global state. Keeping raw training records local does not make these updates private: as discussed in Section 2.6, exchanged model updates can reveal unintended properties of the underlying client data [26]. We therefore implement pairwise masking for the model-training lane. Pairwise masking adds coordinated masks with opposite signs to the client messages so that the masks cancel during aggregation. Here, we apply this construction to the

weighted model deltas used by our FedAvg implementation.

Figure 4.11 isolates pairwise masking within the model-training lane. The mechanism can also be combined with the pre-training protections introduced above.

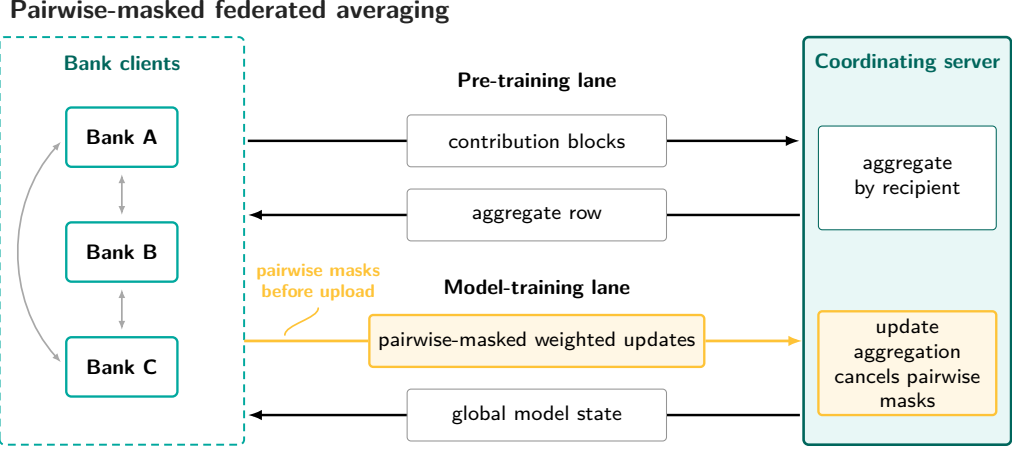


Figure 4.11: Pairwise-masked federated averaging in the model-training lane. It is shown separately to make its effect visible, although it can be combined with the pre-training protection mechanisms.

To apply masking without changing the train-node-weighted FedAvg result, we first express the state average through weighted model deltas. For client $k \in \mathcal{K}$, we define

$$\begin{aligned} \mathbf{d}_k^{(r)} &= \boldsymbol{\theta}_k^{(r)} - \boldsymbol{\theta}^{(r)}, & \mathbf{u}_k^{(r)} &= \alpha_k^{(r)} \mathbf{d}_k^{(r)}, \\ \boldsymbol{\theta}^{(r+1)} &= \boldsymbol{\theta}^{(r)} + \sum_{k \in \mathcal{K}} \mathbf{u}_k^{(r)} = \sum_{k \in \mathcal{K}} \alpha_k^{(r)} \boldsymbol{\theta}_k^{(r)}. \end{aligned} \quad (4.22)$$

The training-account weights defined in Equation (4.8) sum to one, so this delta form is equivalent to the state average in Equation (2.8). Each client masks its weighted delta $\mathbf{u}_k^{(r)}$ before transmission. The coordinator receives these masked vectors, whose pairwise masks cancel in the sum, and reconstructs the same aggregate update as under plaintext FedAvg. The aggregate update remains visible, but individual plaintext floating-point model states are absent from the server-facing message interface. Since the masks add no deliberate perturbation, the aggregated model remains mathematically unchanged.

Exact cancellation requires every expected client to submit its message; the evaluated implementation fails the round if one is missing. It is a local simulation of pairwise mask cancellation and does not implement cryptographic pairwise key establishment, dropout recovery, or malicious

security. Clients are assumed to follow the protocol and not collude with the coordinator to reconstruct another client's masks. We therefore refer to the mechanism as pairwise-masked FedAvg rather than as a production secure-aggregation protocol [52].

Figure 4.12 brings the three independently configurable mechanisms together over the common protocol. Their placement is complementary, but it does not establish a complete-system privacy guarantee. CKKS and contribution-row differential privacy address the pre-training lane, while pairwise masking addresses model training.

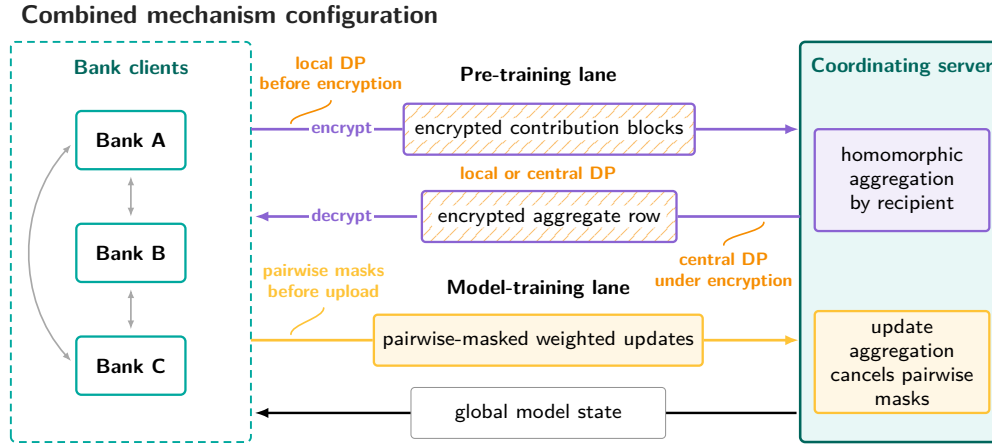


Figure 4.12: Combined mechanism configuration across both communication lanes. Each overlay retains the protection scope and assumptions defined for its respective mechanism.

With these mechanism-specific protection boundaries established, Section 4.6 next defines the broader evaluation design for the centralized, federated, and privacy experiments, including model selection, utility, protection, and implementation-cost assessment.

4.6 Evaluation Methodology

The evaluation design brings together the methodological branches shown in Figure 4.1 and connects their assessment to the three research questions. It first establishes how the centralized and federated configurations are compared, then defines a common assessment of predictive utility, mechanism-specific protection, and implementation cost. This separation is necessary because no single measurement captures whether cross-bank information improves detection, what a privacy mechanism conceals, and what computational or communication cost that protection introduces.

The exact experimental configurations and their measured outcomes are reported in Chapter 5.

4.6.1 Comparative Research Design

To distinguish the value of engineered node features, graph propagation, federated model aggregation, and cross-client graph information, the utility evaluation follows a staged comparison. Logistic regression first measures what can be predicted from the node features without learned graph propagation. The centralized GCN then supplies the full-graph utility reference. Moving to federation, the cut-edge baseline introduces model aggregation while excluding cross-client graph propagation. The one-hop and two-hop configurations subsequently add the two levels of cross-client feature pre-aggregation defined in Sections 4.4.2 and 4.4.3. The federated part of this progression mirrors FedGCN’s comparison of centralized, zero-hop, and pre-aggregated federated GCN variants [24].

The cut-edge, one-hop, and two-hop configurations are evaluated on the symmetric and moderately asymmetric datasets. The symmetric dataset supplies the main comparison, while the moderately asymmetric dataset tests whether findings persist under moderately asymmetric client distributions and lower positive-class prevalence. Within each dataset, the compared methods reuse the node partitions and train-fitted feature transformation from Section 4.2.3.

The privacy comparisons build on these configurations using the primary symmetric dataset. Paired privacy comparisons additionally retain the same graph semantics, model configuration, and training budget wherever possible, changing only the mechanism under examination. Because the training procedures differ, Chapter 5 reports their respective training budgets and communication costs alongside predictive utility.

Federated utility follows the global owned-node evaluation defined in Section 4.4.1, under which each account contributes exactly once. This preserves the same validation and test objective used for centralized evaluation.

4.6.2 Utility Metrics and Model Selection

Predictive utility is assessed through threshold-independent and threshold-dependent measures. The primary metric is the area under the precision–recall curve (AUPR), computed as average precision [79]. For precision P_m and recall R_m at successive score thresholds, average precision is

$$\text{AUPR} = \sum_m (R_m - R_{m-1}) P_m. \quad (4.23)$$

AUPR emphasizes retrieval of the minority positive class and is therefore the principal utility measure for the imbalanced classification task.

Its baseline depends on positive-class prevalence, so cross-dataset differences are interpreted together with the corresponding class rates. The area under the receiver operating characteristic curve (AUROC) supplies a supplementary view of ranking across both classes [80].

A decision threshold converts each predicted probability into a positive or negative classification. At this operating point, precision describes the correctness of positive predictions and recall describes positive-class coverage. F1 summarizes their balance. The Matthews correlation coefficient (MCC) complements these measures by incorporating all four outcomes of the confusion matrix, while the corresponding counts preserve the absolute scale of classification errors [81]. This combination of ranking and operating-point measures is consistent with established AML evaluation practice [16, 17, 53].

Model-state and decision-threshold selection remain separate. During iterative GCN training, the centralized and federated procedures retain the model state with the highest validation AUPR. Selecting the checkpoint on validation data avoids selecting it directly against the test objective [82].

After restoring the selected checkpoint, the decision threshold is chosen from $\mathcal{T} = \{0.05, 0.10, \dots, 0.95\}$. A validation account is assigned to the positive class when its predicted probability is at least τ . The selection rule is

$$\tau^* = \arg \max_{\tau \in \mathcal{T}}^{\text{lex}} (\text{F1}_{\text{val}}(\tau), \text{MCC}_{\text{val}}(\tau)). \quad (4.24)$$

Validation F1 is maximized first and validation MCC breaks an F1 tie. The same τ^* is subsequently applied to test probabilities for precision, recall, F1, MCC, and the confusion counts. Threshold selection does not alter AUPR or AUROC and does not participate in checkpoint selection. It provides a consistent research operating point rather than a deployment threshold optimized for a bank’s investigation capacity or relative error costs.

Although automated selection used only validation data, test metrics were logged during development and were therefore not fully blinded. The resulting evidential limitation is considered in Chapter 6.

4.6.3 Privacy and Information-Exposure Assessment

The evaluated mechanisms provide different forms of protection and therefore cannot be summarized by one privacy score. We first identify the protected information flow and the observer from whom it is hidden. We then assess the supporting evidence, required assumptions, and residual visibility. Section 4.5 defines these boundaries; the evaluation applies this common structure to each mechanism.

The feature-observability boundary is assessed through aligned utility comparisons between the original 36-feature representation and the

boundary-compliant 32-feature set. Cryptographic and masking mechanisms are additionally checked for computational correctness against their plaintext counterparts. Recipient-routed CKKS aggregation is evaluated through the numerical error between decrypted aggregate rows and an in-memory reference, together with a matched comparison of predictive utility. Pairwise-masked FedAvg is evaluated through mask cancellation, aggregate-update equivalence, and predictive utility. These checks establish whether the protected execution preserves the intended computation; they do not independently establish privacy.

Contribution-row differential privacy is evaluated as the formal mechanism using the privacy definition and calibration established in Section 4.5.3. We additionally evaluate empirical Gaussian perturbation to examine how predictive utility changes with perturbation magnitude. These supplementary comparisons carry no formal differential-privacy claim.

These results are interpreted within the protection boundaries established in Section 4.5.1. Their operational adequacy is considered further in Chapter 6.

4.6.4 *Memory and Communication Accounting*

Efficiency is evaluated together with utility because cross-client graph information and its protection can substantially change both communication and computation. Communication–utility comparisons are also central to the evaluation of FedGCN-style pre-aggregation [24], while recent collaborative AML work reports runtime and transferred data to assess multi-institution feasibility [20]. We report peak memory and communication separately because their measurement scopes differ.

For selected cluster jobs, scheduler records provide peak resident memory usage. These measurements remain hardware-dependent and are compared only between runs with aligned hardware and software configurations.

To separate protocol volume from machine performance, communication during cross-client pre-training is counted in rows, blocks, and feature elements for both upload and download. Plaintext payload volume is calculated from these feature-element counts using their float32 representation. Under CKKS transport, upload and download volume is measured from the serialized ciphertext payloads. Repeated two-hop uploads are additionally characterized by comparing total uploaded row occurrences with the number of unique contribution rows. For CKKS, serialized ciphertext payloads and cryptographic contexts are recorded separately.

Model-update communication is estimated from the model-state tensors, the number of participating clients, and the executed rounds. Plaintext upload and global-model broadcast estimates use the tensor element sizes. The pairwise-masked path uses the size of its flattened float64

update vector, together with any non-floating state. These estimates cover the implemented masking exchange. Additional communication that would be required by a production secure-aggregation protocol falls outside this accounting.

The resulting measurements support relative implementation comparisons within the reported experimental environment rather than representing end-to-end network performance. Their implications beyond this environment are considered in Chapter 6.

Experimental Evaluation

5

5.1 Experimental Setup

The preceding chapter established the common data, model, and evaluation procedures. Building on this foundation, the experiments examine how predictive utility changes as graph access moves from the global setting to bank-owned client graphs and as protection is added to the resulting information flows. Evaluation begins with the centralized baseline, continues with the federated configurations, and then examines additional privacy enhancements. Together, these comparisons address federated adaptation and utility retention relative to centralized learning (RQ1), the value of cross-bank graph information under differing degrees of client heterogeneity (RQ2), and privacy–utility–efficiency trade-offs (RQ3).

The experiments use three datasets generated through the construction and validation procedures described in Section 4.1. We first establish their configurations and experimental roles, then specify the common model, evaluation, and computing settings.

5.1.1 *Primary Symmetric Three-Bank Dataset*

The selected configuration generated one million accounts distributed almost equally across three banks. Bank A contains 333,334 accounts, while Banks B and C each contain 333,333. The three-bank design provides a compact multi-party setting that moves beyond bilateral interaction while keeping the resulting comparisons straightforward to inspect and interpret. We refer to this output as the primary symmetric three-bank dataset. Here, symmetric describes the configured client setup: bank populations are nearly equal and bank-specific generation inputs are shared. It does not imply class balance, independent and identically distributed (IID) data, equal transaction activity, or identical realized properties across banks.

Base-graph construction targeted 66% within-bank and 34% cross-bank edges. Each bank was assigned 3,600 generated laundering patterns whose members were restricted to that bank, alongside 3,000 patterns configured to span banks. The resulting design contains both bank-restricted and cross-bank laundering patterns, providing the basis for later experi-

ments that determine whether cross-bank information contributes predictive value.

The output contains 68.5 million individual transaction records. Of these, 45,718,769 (66.74%) are within-bank and 22,788,500 (33.26%) are cross-bank records. Membership in the generated laundering patterns yields 100,525 SAR-labeled accounts, corresponding to 10.05% of all accounts. This rate provides sufficient positive examples at each bank for local training and evaluation while retaining an imbalanced classification task. A 10% positive-account rate has also been used as the baseline in prior account-level AML experimentation, although such an experimental class distribution does not estimate real-world laundering prevalence [83]. The output also includes 90,685 transaction records generated within laundering patterns, corresponding to 0.132% of all transaction records. This transaction-level share is close to the 0.102–0.124% range reported for AMLWorld’s higher-illicit-ratio datasets [53]. Appendix A.1.3 reports the corresponding per-bank profile.

These properties make the dataset the reference for the centralized, federated, and privacy comparisons. Its nearly symmetric bank populations and positive-account rates provide a stable basis for comparison, even though transaction activity remains uneven across banks. Its cross-bank connectivity permits later experiments to test the predictive value of cross-bank graph information. The dataset supports controlled comparisons rather than modeling the distributions of real banking systems. The following subsection introduces deliberate departures from this symmetric setting.

5.1.2 Asymmetric Dataset Variants

The primary symmetric dataset provides a controlled reference, but federated clients may differ substantially in size and data composition. We therefore generated two parameter-controlled asymmetric variants to examine federated learning as client heterogeneity increases. The terms symmetric and asymmetric describe the configured client setup, whereas *client heterogeneity* refers to realized structural and statistical differences among client datasets. The asymmetric variants deliberately amplify this heterogeneity; the symmetric dataset is not assumed to be IID or homogeneous. The principal comparison uses the symmetric and moderately asymmetric datasets. The strongly asymmetric stress dataset has a narrower supporting role.

Both variants preserve the one-million-account scale, three-bank setting, 1,440-step simulation horizon, global degree sequence, normal-model allocation, and transaction-type distribution. They apply the parameter-based variation described in Section 4.1.1 without further generator-code changes.

5. EXPERIMENTAL EVALUATION

The moderately asymmetric dataset assigns the largest bank the lowest initial balances and fewest bank-restricted laundering patterns, while the smallest bank receives the highest balances and most patterns. This gradual ordering amplifies client heterogeneity and lowers positive-class prevalence relative to the primary dataset, while every bank retains a substantial account population. The moderately asymmetric dataset therefore serves as the principal robustness setting for assessing whether findings from the primary dataset persist under these conditions.

Table 5.1 compares selected properties of the symmetric and moderately asymmetric settings, including their account allocation, balance ranges, laundering-pattern distribution, and SAR-labeled account composition.

Table 5.1: Selected properties of the symmetric and moderately asymmetric three-bank datasets. Initial-balance amounts are in USD.

Property	Symmetric	Moderate asymmetry
Accounts by bank	A: 333 334 B: 333 333 C: 333 333	A: 400 000 B: 350 000 C: 250 000
Initial-balance range by bank	A: 50 000–100 000 B: 50 000–100 000 C: 50 000–100 000	A: 30 000–80 000 B: 50 000–125 000 C: 100 000–300 000
Bank-restricted laundering patterns	A: 3 600 B: 3 600 C: 3 600	A: 1 350 B: 2 100 C: 3 000
Cross-bank laundering patterns	3 000	1 500
SAR-labeled accounts, total	100 525 (10.05%)	57 508 (5.75%)
SAR-labeled account rate by bank	A: 10.16% B: 9.99% C: 10.01%	A: 3.43% B: 5.34% C: 10.03%

The strongly asymmetric dataset intensifies the same ordering and uses more widely separated laundering-pattern amount settings. Although its overall SAR-labeled account rate of 6.08% is slightly higher than the moderately asymmetric dataset’s rate, its bank-level rates span 1.97–17.69%, compared with 3.43–10.03% under moderate asymmetry. This wider dispersion, together with the stronger configured differences, provides an amplified multi-bank stress setting for selected analyses rather than a third principal comparison setting. Appendix A.1.4 reports its detailed properties.

5.1.3 Common Experimental Configuration

Across the principal centralized and federated utility comparisons, we retain unit directed account-pair edges and the common three-layer GCN with hidden width 128. Centralized training uses an 80 000-epoch cap, whereas the federated configurations run for 5 000 rounds with 16 local epochs per round. These settings reflect the retained calibration results. Because centralized epochs and federated rounds with local training do not provide a common unit of computation, the two budgets are not treated as exactly matched. Further dataset- and configuration-specific settings are reported with the corresponding experiments.

Utility comparisons initially use the 36-feature representation. For the privacy evaluation, we first measure the effect of moving to the boundary-compliant 32-feature representation and then retain this feature set when evaluating the mechanisms introduced in Section 4.5. Memory and communication results accompany the experiments to which they apply.

Following the model-selection, reporting, and accounting procedures in Section 4.6, AUPR serves as the primary utility measure. At the validation-selected operating point, F1 and MCC provide the main supporting measures. AUROC supplements them with a ranking measure where available.

Principal model-training experiments ran on RWTH Aachen University’s CLAIX-2023 high-performance computing cluster [84] using one NVIDIA H100 GPU and eight allocated cores of its Intel Xeon Platinum 8468 processors.¹ Memory is compared only between runs with aligned hardware environments and otherwise comparable experimental configurations, while communication values retain the measured or estimated scope defined in Section 4.6.4. Against this common setup, the following section establishes the centralized reference performance.

5.2 Centralized Reference Performance

5.2.1 Model Calibration

Alongside the first centralized GCN implementation, we trained a logistic regression model on the same features and node partitions as a non-propagating diagnostic. It reached test AUPR 0.3487, whereas a preliminary two-layer GCN trained under a 600-epoch cap reached 0.5315. The later calibrated GCN widened this separation further. Although the comparison does not isolate message passing from differences in model capacity and optimization, the substantial gap provides evidence that adjacency-based propagation contributes useful signal beyond the engineered features. On this basis, we proceeded to calibrate the GCN.

¹Further details on the computing environment are available in the RWTH Aachen University IT Center’s description of the [CLAIX-2023 compute nodes](#).

5. EXPERIMENTAL EVALUATION

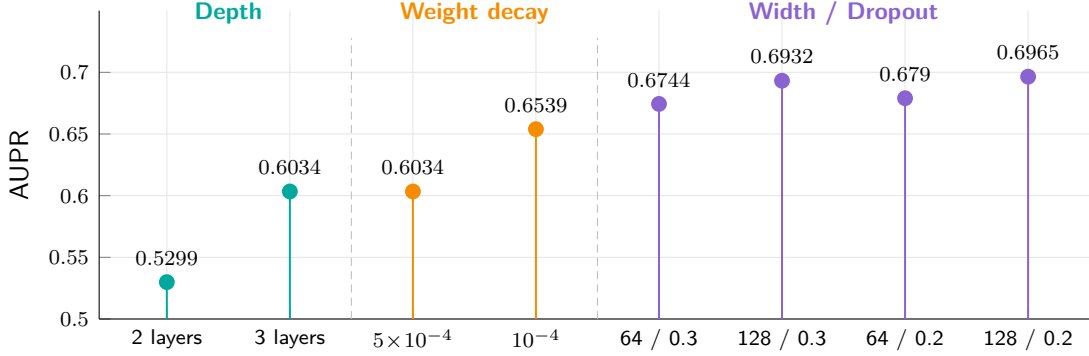


Figure 5.2: Selected validation-based checks used to calibrate the centralized GCN on the primary dataset. The depth and weight-decay comparisons use 600-epoch caps; the width/dropout comparison uses an 800-epoch cap and reports settings as hidden width / dropout. The groups show the successive stages through which the retained configuration was established.

Starting from the two-layer implementation, we refined the model through a sequence of comparisons using validation AUPR. At the 600-epoch cap, adding a third graph-convolution layer increased AUPR from 0.5299 to 0.6034. Reducing weight decay from 5×10^{-4} to 10^{-4} then raised it to 0.6539. In a subsequent 800-epoch comparison, hidden width 128 and dropout 0.2 together produced the strongest candidate, with AUPR 0.6965. Figure 5.2 summarizes these successive checks, which established the retained configuration.

The selected primary configuration therefore uses three layers, hidden width 128, and dropout 0.2. We then examined how well this configuration transferred to the moderately asymmetric dataset through a targeted screen of depth, learning rate, and dropout. Three layers remained preferable. For this dataset, a matched 2,500-epoch comparison favored dropout 0.1 over 0.2, with AUPR 0.7235 rather than 0.7105. Learning rate 10^{-2} and weight decay 10^{-4} remain unchanged across both settings.

Architecture selection alone did not settle the required training duration. The retained primary configuration continued to select checkpoints at or near the cap when training was extended from 800 to 1,000 epochs. When the first federated results later approached this working reference, we revisited whether the centralized model remained undertrained. Further runs at 2,500 and 5,000 epochs continued to improve, with checkpoints selected at epoch 2,500 and 4,780, respectively.

The final centralized campaign extended this progression to an 80,000-epoch cap. For orientation, the later federated experiments use 5,000 rounds with 16 local epochs per round, giving each client a nominal total

of 80,000 local training epochs. This aligns the nominal training horizon across the two settings. A centralized epoch and a client-local epoch do not perform equivalent computation, so the alignment is not an exact compute match. Across the extended horizon, validation AUPR gradually leveled off near its strongest values, with the highest observed value determining the retained checkpoint reported below.

5.2.2 Retained Centralized References

During primary calibration, repeated-transaction weights outperformed unit edges by 0.0030 test AUPR in a 1,000-epoch comparison. Because performance continued to improve as the training cap increased, we held the selected architecture fixed and repeated this comparison under the final 80,000-epoch cap rather than carrying the earlier ordering forward. Table 5.3 reports the matched long-cap results. Unit edges improve test AUPR by 0.0292 on the primary dataset and by 0.0123 under moderate asymmetry. Both matched long-cap comparisons therefore favor unit edges.

Table 5.3: Matched centralized edge-semantics comparison. Values are test AUPR from the retained 80,000-epoch runs. Weighted edges use repeated-transaction weights.

Dataset design	Weighted edges	Unit edges	Difference
Symmetric	0.7725	0.8017	+0.0292
Moderate asymmetry	0.7482	0.7606	+0.0123

The reversal indicates that the observed effect of edge weighting depends on the training budget and does not establish a universal ordering between the two representations.

On the primary dataset, the retained unit-edge run reached its highest logged validation AUPR of 0.8044 at epoch 59,780. This checkpoint was retained for evaluation. The corresponding test AUPR is 0.8017, with F1 0.7378 and MCC 0.7105 at the validation-selected operating point. Figure A.11 in the appendix shows the validation trajectory and selected checkpoint.

Table 5.4 places this principal reference alongside the corresponding result for the moderately asymmetric dataset.

Table 5.4: Retained centralized references.

Dataset design	Epoch	AUPR	F1	MCC
Symmetric	59 780	0.8017	0.7378	0.7105
Moderate asymmetry	61 580	0.7606	0.7193	0.7037

Under moderate asymmetry, validation AUPR selected epoch 61,580; the corresponding test AUPR is 0.7606. Although the positive-account rate falls from 10.05% to 5.75%, all three measures remain comparatively close to the primary reference: AUPR decreases by 0.0411, while F1 and MCC decrease by 0.0185 and 0.0068, respectively. The centralized GCN therefore retains substantial predictive utility under this setting, indicating that the modeling approach remains effective across both dataset conditions despite the lower positive rate and uneven bank-level distributions. These full-graph results now provide the empirical references for evaluating bank-owned federated training under different levels of access to cross-bank graph information.

5.3 Federated Utility and Cross-Bank Graph Information

The centralized references show the utility available when one model can use the global transaction graph. The federated experiments now examine what remains when this graph is divided among bank clients and whether the additional cross-bank context provided by the pre-aggregation introduced in Section 4.4.2 improves predictive utility. Before comparing the retained configurations, we first establish the model and local-training settings used throughout these experiments.

5.3.1 Federated Configuration Calibration

The federated campaign began from the centralized three-layer, width-128 model base. We then checked whether this architecture remained suitable after partitioning the graph and used targeted learning-rate and dropout screens to adapt the optimization settings to each dataset. A matched two-versus-three-layer comparison confirmed that three layers remained preferable for the cut-edge, one-hop, and two-hop configurations. Supporting depth and optimization-screen results are reported in Appendix A.5.2. Retaining the third layer does not alter the cross-client depth represented by the one-hop and two-hop labels. It adds another propagation step within the client graph, as described in Section 4.4.3.

Local epochs determine how much optimization each client performs before the next model aggregation. Their number is a standard tunable parameter in FedAvg, with foundational experiments considering values up to 20 local epochs [22]. In this campaign, we screened a range of 1, 2, 4, 8, and 16 local epochs. Figure 5.5 reports validation AUPR after 2,500 rounds for every federated configuration on both principal datasets.

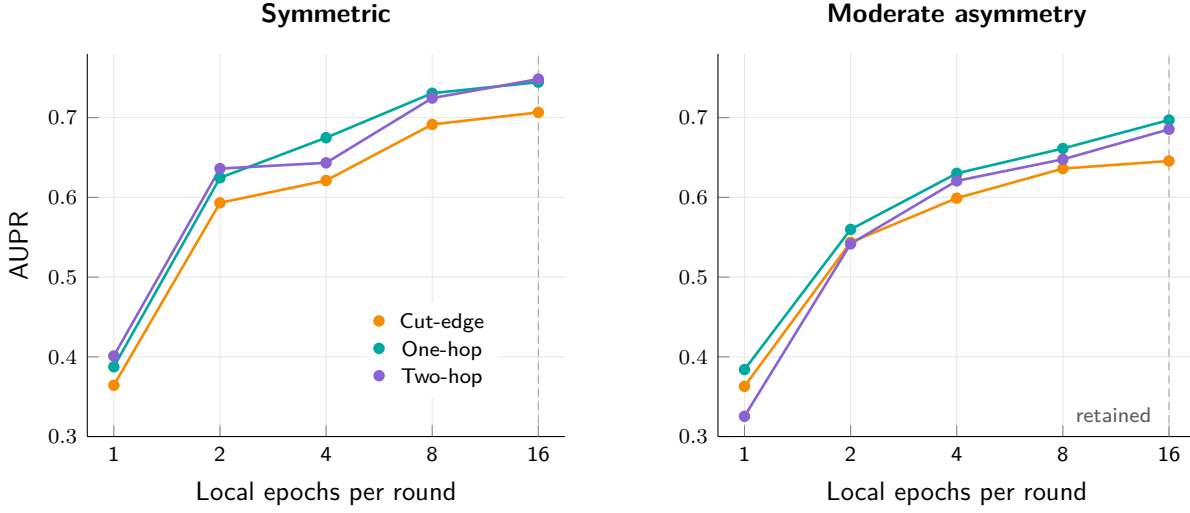


Figure 5.5: Federated local-work calibration after 2,500 rounds. Each panel reports validation AUPR as the number of local epochs per round is increased while the remaining dataset-specific settings are held fixed. The dashed line marks the retained setting of 16 local epochs.

Across all six comparisons, AUPR rises over the tested range and is highest at 16 local epochs. We therefore carried 16 local epochs into the final 5,000-round campaign as the strongest tested common setting. This schedule allows substantial optimization on each client graph while continuing to alternate local training with shared-model aggregation; it is not treated as a universal optimum.

Since every point uses 2,500 rounds, increasing the number of local epochs also adds optimizer work. The screen therefore selects a common local-training setting rather than comparing equal computational budgets. The final runs combine this setting with three layers, hidden width 128, unit edges, and train-node-weighted federated averaging. Targeted learning-rate comparisons selected 2×10^{-3} for the primary configurations and 3×10^{-3} under moderate asymmetry. An additional dropout comparison for the moderately asymmetric dataset retained 0.2 for cut-edge and 0.1 for the one-hop and two-hop configurations; the primary configurations use dropout 0.2.

The final checkpoint positions provide additional context for the 5,000-round budget. Selected checkpoints ranged from round 4,131 to 4,843 on the primary dataset and from round 4,843 to 4,957 under moderate asymmetry. The longer horizon therefore remained relevant in both settings, with late selection particularly consistent under moderate asymmetry. Because validation performance fluctuates within a late plateau, these rounds should not be interpreted as exact convergence points.

5. EXPERIMENTAL EVALUATION

5.3.2 Cut-Edge Federated Utility

The cut-edge baseline provides the most direct starting point for the federated comparison. It retains the shared model and federated averaging, but each bank performs graph propagation only on its local graph. Table 5.6 reports the exact values, while Figure 5.7 visualizes the resulting gaps across the three measures.

Table 5.6: Predictive utility of the centralized reference and cut-edge federated baseline on both principal datasets. Difference rows report cut-edge minus centralized utility.

Dataset design	Configuration	AUPR	F1	MCC
Symmetric	Centralized	0.8017	0.7378	0.7105
	Cut-edge	0.7194	0.6729	0.6356
	Difference	−0.0822	−0.0649	−0.0749
Moderate asymmetry	Centralized	0.7606	0.7193	0.7037
	Cut-edge	0.6529	0.6428	0.6209
	Difference	−0.1077	−0.0765	−0.0829

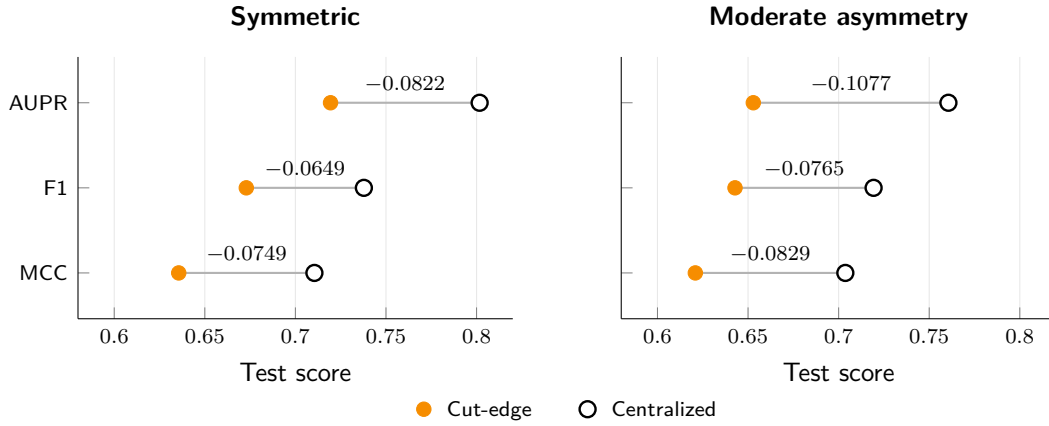


Figure 5.7: Predictive utility of the centralized reference and cut-edge federated baseline on both principal datasets. Labels report cut-edge minus centralized utility.

On the primary dataset, the cut-edge baseline reaches test AUPR 0.7194, F1 0.6729, and MCC 0.6356. The shared model therefore retains substantial predictive utility when each client is restricted to its local graph. At the same time, a clear utility gap remains: centralized training exceeds cut-edge by 0.0822 AUPR, 0.0649 F1, and 0.0749 MCC.

The consistent deficit across ranking and operating-point measures establishes the performance gap that the one-hop and two-hop configurations are intended to narrow.

The result for the moderately asymmetric dataset follows the same ordering at a lower overall level. Cut-edge training reaches AUPR 0.6529, F1 0.6428, and MCC 0.6209, while the centralized reference reaches 0.7606, 0.7193, and 0.7037. Relative to the primary dataset, centralized AUPR decreases by 0.0411, whereas cut-edge AUPR decreases by 0.0666. The gap between them consequently widens from 0.0822 to 0.1077. F1 and MCC preserve this pattern, with larger centralized-to-cut-edge differences under moderate asymmetry than on the primary dataset.

These results indicate that conventional federated averaging over local client graphs remains viable. Its separation from full-graph training is larger under moderate asymmetry, where bank sizes, positive rates, and activity distributions vary jointly. Federated model aggregation therefore retains a useful shared model, but leaves a material predictive gap when graph propagation is restricted to local client graphs. This gap provides the practical reference for the next experiment, which asks how much cross-client pre-aggregation can narrow it.

5.3.3 Federated Learning With Cross-Bank Information

One-hop and two-hop pre-aggregation make progressively broader cross-bank information available to the federated clients. Figure 5.8 compares the resulting test AUPR of both cross-client configurations with the centralized and cut-edge references. Exact AUPR, F1, and MCC values are reported in Table 5.9.

Both forms of cross-client pre-aggregation improve predictive performance over cut-edge training on both principal datasets. On the primary dataset, one-hop reaches AUPR 0.7506 and two-hop reaches 0.7599, corresponding to gains of 0.0312 and 0.0405. Under moderate asymmetry, one-hop reaches 0.6981 and two-hop reaches 0.6836, improving on cut-edge by 0.0452 and 0.0307. The main result is therefore not simply that a broader cross-client scope is always better, but instead that two-hop is strongest on the primary dataset, whereas one-hop is strongest under moderate asymmetry.

F1 and MCC show the same broad result: both cross-client configurations improve on cut-edge in each dataset. All three measures favor two-hop on the primary dataset and one-hop under moderate asymmetry. In the latter setting, one-hop leads two-hop by 0.0145 AUPR, while their F1 and MCC differ by only 0.0051 and 0.0038. One-hop therefore provides the clearer improvement in the precision-recall trade-off across thresholds, whereas both configurations still achieve similar classification quality at the selected operating point.

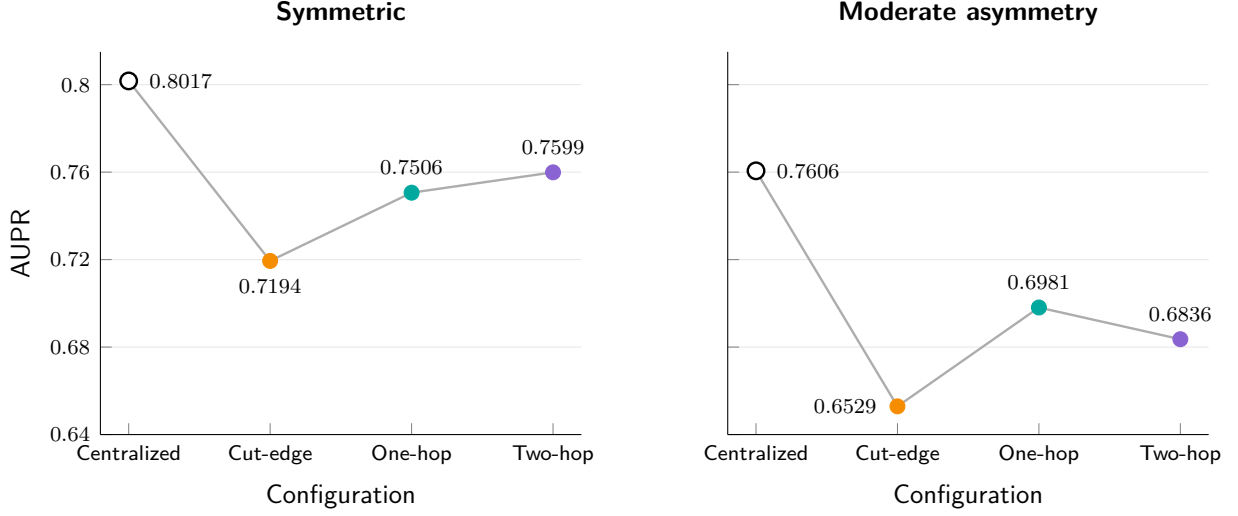


Figure 5.8: Test AUPR of the retained centralized and federated configurations on the symmetric and moderately asymmetric datasets.

Table 5.9: Predictive utility of the retained federated configurations on both principal datasets. Gain columns report one-hop or two-hop minus cut-edge within the same dataset.

Configuration	Test utility			Gain over cut-edge		
	AUPR	F1	MCC	AUPR	F1	MCC
<i>Symmetric</i>						
Cut-edge	0.7194	0.6729	0.6356	–	–	–
One-hop	0.7506	0.7052	0.6732	0.0312	0.0323	0.0376
Two-hop	0.7599	0.7174	0.6864	0.0405	0.0445	0.0508
<i>Moderate asymmetry</i>						
Cut-edge	0.6529	0.6428	0.6209	–	–	–
One-hop	0.6981	0.6806	0.6608	0.0452	0.0378	0.0400
Two-hop	0.6836	0.6755	0.6570	0.0307	0.0327	0.0362

To assess whether these findings depended on the selected random seed, we repeated the comparison with four alternative seeds. The supporting runs reinforce the cross-bank results. Across every tested seed and both datasets, one-hop and two-hop consistently outperform cut-edge, with modest seed-to-seed variation in the reported metrics. Appendix A.5.3 reports the complete results and supporting evaluation budget.

Cross-bank information reduces the gap to the centralized reference, but does not eliminate it. The strongest primary result, two-hop, remains

0.0417 AUPR below centralized training, while the strongest result under moderate asymmetry, one-hop, leaves a gap of 0.0625. Relative to the observed gap between cut-edge and centralized training, the best federated mode closes 49.3% on the primary dataset and 42.0% under moderate asymmetry. F1 and MCC show the same overall pattern: the best cross-client configuration moves closer to centralized training without matching it. Figure 5.10 shows the corresponding gaps across all three utility measures.

Pre-aggregation therefore recovers part of the observed utility gap without reproducing centralized graph access or optimization, leaving a smaller but still material difference.

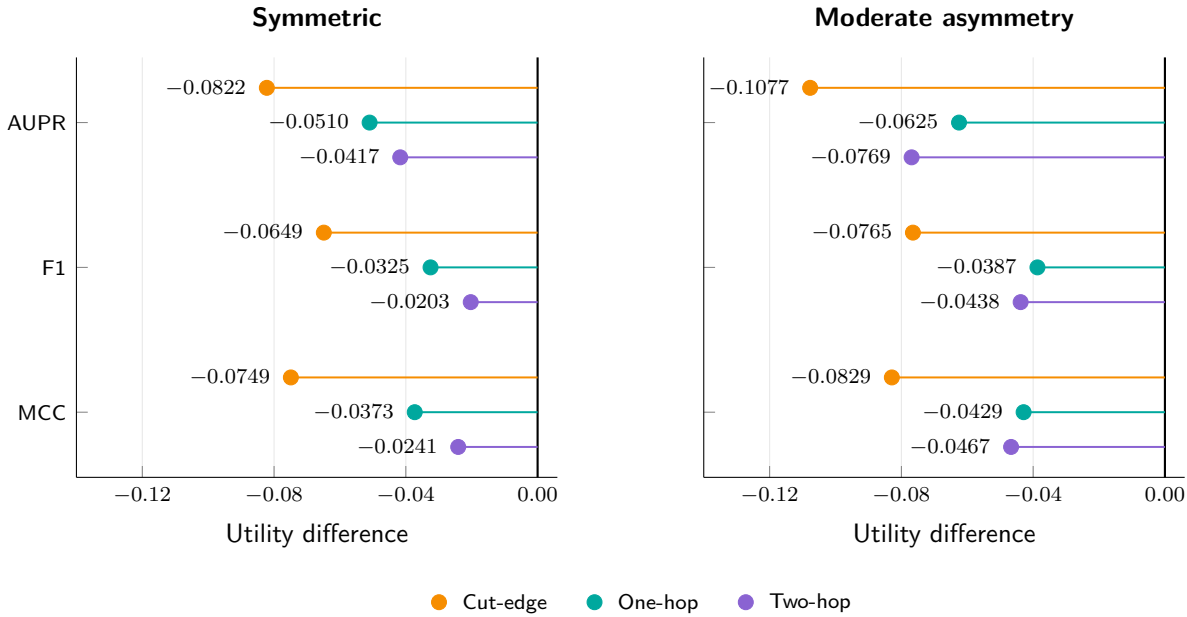


Figure 5.10: Difference between each retained federated configuration and the corresponding centralized reference across AUPR, F1, and MCC.

At this point, the strongly asymmetric dataset provides additional evidence on whether the one-hop advantage persists under a second, more asymmetric client partition. One-hop again performs best, reaching AUPR 0.7289 compared with 0.6974 for cut-edge and 0.7041 for two-hop. Its gain over cut-edge is therefore 0.0315, whereas the two-hop gain contracts to 0.0067. Figure 5.11 places these gains alongside the principal results. The complete metric panel for the strongly asymmetric dataset, including the centralized reference, is reported in Appendix A.5.3.

5. EXPERIMENTAL EVALUATION

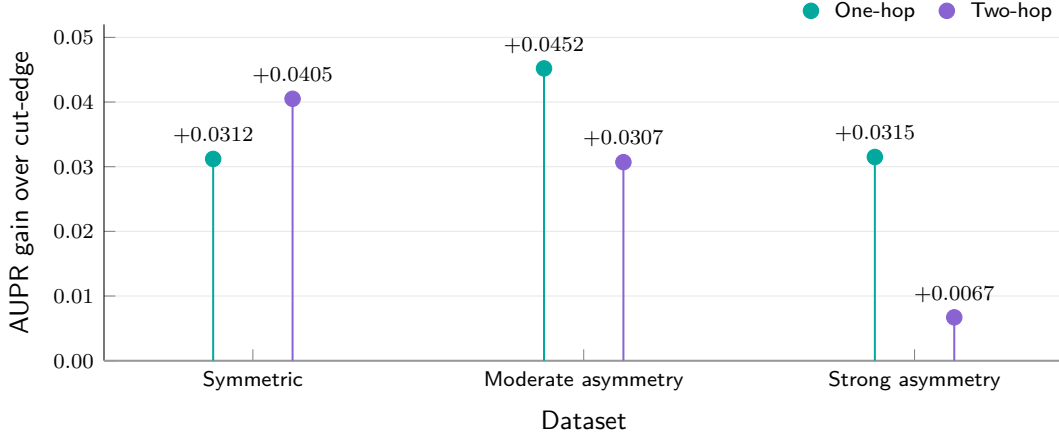


Figure 5.11: Test AUPR gained by one-hop and two-hop pre-aggregation over cut-edge training within each dataset.

The one-hop and two-hop configurations also differ in the amount of pre-training data delivered to clients. One-hop installs one aggregate row per owned account. Two-hop additionally delivers the placeholder rows required by the expanded computation graphs, increasing the recipient row count by 42.6% on the primary dataset and 45.5% under moderate asymmetry. This wider delivery produces a small additional utility gain on the primary dataset, but not in either asymmetric setting. Because the placeholder rows are graph-derived aggregates for foreign nodes, they also enlarge the set of information released to each recipient. Chapter 6 considers this combined utility, communication, and exposure trade-off. The row counts describe the one-shot recipient download rather than complete protocol traffic, which is evaluated in the following section.

Overall, these experiments support the use of cross-bank aggregate context in federated graph learning, but do not identify one universally preferable depth. Two-hop is strongest on the primary dataset, while one-hop combines the lower recipient payload with stronger utility in both asymmetric settings. The next section applies the feature boundary and protection mechanisms defined in Section 4.5 to pre-training and model-update communication.

5.4 Evaluation of Privacy Enhancements

The preceding experiments established that cross-client pre-aggregation can recover part of the utility lost when federated training restricts each bank to its local graph. Achieving these gains requires the pre-training exchange of graph-derived information, which adds new protection surfaces alongside the model updates already communicated through FedAvg. Sec-

tion 4.5 defines these surfaces and the mechanisms applied to them. Using the primary symmetric dataset, we now examine whether protected execution preserves the intended computation and predictive utility, and which systems overheads it introduces. We begin with the feature representation retained throughout the privacy experiments.

5.4.1 *Boundary-Compliant Features*

The utility comparisons in Section 5.3 use the original 36-feature representation. Four of these features summarize neighboring-account activity and can incorporate statistics derived from nodes at other banks. Before evaluating the privacy mechanisms, we therefore remove these features and adopt the boundary-compliant 32-feature set defined in Section 4.5.1. This restriction is not itself a privacy mechanism. Instead, it applies a stricter bank-level observability assumption to the comparisons that follow.

Removing the four summaries reduces AUPR, F1, and MCC across the federated training paths. The principal federated result nevertheless remains unchanged: under the 32-feature boundary, both one-hop and two-hop continue to outperform cut-edge training. One-hop reaches AUPR 0.7248, while two-hop reaches 0.7268, compared with 0.6868 for cut-edge. The observed cross-client benefit therefore does not depend on the excluded summaries. All subsequent mechanism comparisons retain the boundary-compliant feature set. Appendix A.5.4 reports the complete matched results.

5.4.2 *CKKS Transport and Pairwise-Masked FedAvg*

The first two mechanisms address information observed by the coordinating server in different phases of the federated protocol. CKKS encrypts the contribution blocks combined during pre-training, whereas pairwise masking replaces the individual model updates received during FedAvg with masked vectors. We compare their combined use with the canonical plain one-hop and two-hop configurations established above.

CKKS enables approximate arithmetic over encrypted vectors [31]. As expected, the decrypted aggregate rows in the evaluated runs closely match their plaintext counterparts. Across all delivered payloads, the maximum absolute error introduced by CKKS is 3.81×10^{-6} , while mean absolute errors remain on the order of 10^{-9} . For each client pair, pairwise masking adds the same random mask to one update and subtracts it from the other. These terms cancel when all expected clients participate, as in secure-aggregation protocols [52]. Tests likewise confirm that the implemented masks reconstruct the same aggregate update, which remains visible to the server.

Consistent with this numerical fidelity and the tested mask cancellation, predictive utility remains close to the canonical plain references. With CKKS transport and pairwise-masked FedAvg, one-hop AUPR is lower by 0.0044, while two-hop remains unchanged at the reported precision. Across these two comparisons, F1 and MCC decrease by at most 0.0072. CKKS-specific stability is additionally supported by a four-seed comparison of plain and encrypted transport for both one-hop and two-hop, in which all paired differences remain small under a shorter training schedule. The complete values and seed results are reported in Appendix A.5.5.

The more visible effect is the systems cost of CKKS. Table 5.12 reports these costs for the one-million-node primary dataset and a reduced-scale 100,000-node version. On the primary dataset, the one-hop path contains 0.512 GB of plaintext feature payload but produces 20.88 GB of serialized CKKS traffic and reaches 26.2 GiB peak memory. For two-hop, the corresponding values rise to 0.730 GB, 29.78 GB, and 34.8 GiB. Encryption therefore expands the serialized representation by a factor of approximately 40.8 under the fixed CKKS configuration. Two-hop remains more expensive because its recipient plans contain additional aggregate placeholder rows.

Serialized CKKS traffic scales with the number of encrypted contribution blocks by construction. To place these costs in scale context, we measure the same one-hop and two-hop CKKS paths on the reduced-scale version with aligned bank proportions and generation settings.

Table 5.12: Systems cost of CKKS pre-training at two graph scales. Upload and download block counts and serialized CKKS volume are measured; plaintext payload volume is calculated from the number of transmitted feature values. Communication volumes use decimal GB, while peak memory uses binary GiB.

Mode	Nodes	Blocks (up/down)	Plaintext payload (GB)	Serialized CKKS (GB)	Peak memory (GiB)
One-hop	100,000	4 686/1 563	0.051	2.09	3.1
One-hop	1,000,000	46 833/15 627	0.512	20.88	26.2
Two-hop	100,000	6 712/2 239	0.073	2.99	4.1
Two-hop	1,000,000	66 797/22 283	0.730	29.78	34.8

As expected, the tenfold increase in account nodes produces between 9.95 and 10.00 times as many transmitted blocks and the same increase in serialized traffic. Peak memory rises by a smaller factor of approximately 8.4.

Because every account requires an aggregate target row, pre-training communication grows with account count even when graph structure remains stable. The multiplier can exceed this baseline when larger de-

ployments also contain more cross-bank neighbors or participating banks, since additional rows may be required by several recipients.

Over the complete training run, pairwise masking increases client-update traffic from 1.2519 GB to 2.5038 GB. This doubling results from representing the masked vectors in float64 rather than the float32 format of the model state; it is not an inherent requirement of pairwise masking. The global-model broadcast is unchanged. A complete secure-aggregation protocol would introduce additional coordination and recovery traffic that is not modeled in the local simulation.

CKKS and pairwise masking therefore alter two server-visible information flows without a material utility loss in the evaluated runs. Neither mechanism changes the aggregate rows ultimately released to recipient banks. The next subsection evaluates perturbation of those released values, where protection directly changes the model input and creates a different utility question.

5.4.3 *Contribution-Row DP*

After decryption, the routed aggregate rows are directly visible to the recipient that uses them for local model computation. Contribution-row differential privacy is the evaluated mechanism that places a formal guarantee on the information exposed through this release. We evaluate it alongside CKKS transport and pairwise-masked FedAvg in the two-hop setting, allowing all three privacy mechanisms to be assessed in combination. Without row DP, this configuration provides the strongest retained boundary-compliant federated reference at AUPR 0.7268.

Following the mechanism defined in Section 4.5.3, every formal run uses contribution-row substitution adjacency, clipping bound $C_{\text{clip}} = 5$, sensitivity $\Delta_2 = 10$, and $\delta = 10^{-5}$. We hold δ fixed throughout the panel. We evaluate two placements that protect the same released aggregate row but introduce noise at different points in the aggregation path. Local placement clips and perturbs each contribution before its encrypted upload. Under central placement, the clipped contributions are first aggregated under CKKS and one encrypted noise vector is then added to the completed target row. The two placements therefore differ in where noise enters the computation and whether independently perturbed client contributions are combined. Under either placement, the released aggregate satisfies (ϵ, δ) -differential privacy with respect to substituting one clipped contribution row, as defined in Section 4.5.3.

We begin with the classical Gaussian mechanism, which provides the standard conservative calibration for $0 < \epsilon \leq 1$ [50]. At $\epsilon = 1$, the fixed δ and sensitivity require $\sigma = 48.448$. Table 5.13 shows that neither perturbation placement retains useful predictive performance under this calibration.

5. EXPERIMENTAL EVALUATION

Table 5.13: Predictive utility under classical contribution-row DP at $\varepsilon = 1$, $\delta = 10^{-5}$, and $\sigma = 48.448$.

Placement	AUPR	F1	MCC
Local	0.2380	0.3144	0.2253
Central	0.2415	0.3081	0.2212

Central placement has the slightly higher AUPR, whereas local placement has the slightly higher F1 and MCC. These small differences do not establish a preferred placement. More importantly, both results remain far below the unperturbed two-hop reference at AUPR 0.7268 and the boundary-compliant cut-edge baseline at 0.6868.

The classical bound is conservative and restricted to $\varepsilon \leq 1$. Balle and Wang derive an analytic calibration that can be applied beyond this range and requires less noise for the same (ε, δ) guarantee [51]. We evaluate this mechanism for $\varepsilon \in \{1, 2, 4, 8, 16, 32, 64\}$. Figure 5.14 presents the resulting sweep; the complete metric panel is reported in Appendix A.5.6.

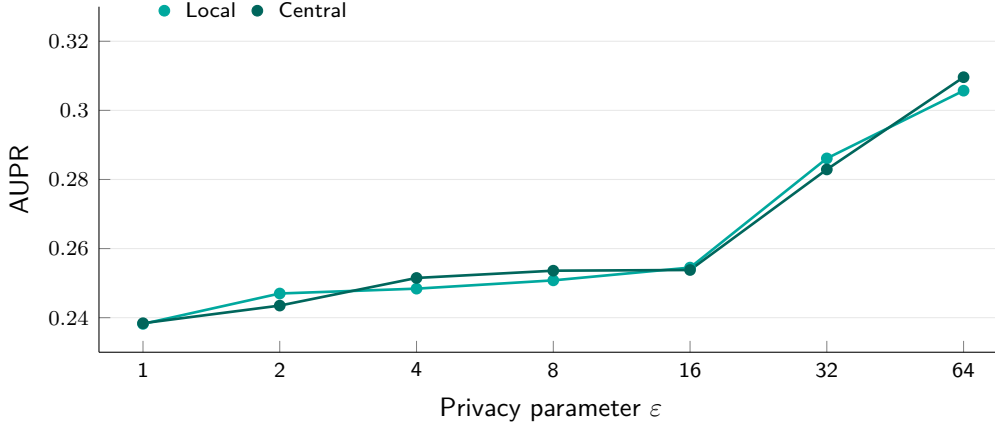


Figure 5.14: Test AUPR under analytic contribution-row DP for local and central perturbation across the evaluated privacy parameters.

Interpreting this wider sweep requires separating formal validity from the practical strength of the guarantee. Larger ε permits greater differences between neighboring output distributions and therefore weakens privacy. Balle and Wang characterize increasing ε as movement toward the low-privacy regime, while guidance from the U.S. National Institute of Standards and Technology (NIST) cautions that large values may not provide meaningful privacy and require contextual risk assessment beyond $\varepsilon = 1$ [51, 85]. Accordingly, $\varepsilon = 64$ remains a formally calibrated but weak contribution-row guarantee.

Despite this relaxation, the analytic result remains poor. AUPR stays below 0.27 through $\varepsilon = 16$, rises to 0.2861 and 0.2829 for local and central placement at $\varepsilon = 32$, and reaches only 0.3057 and 0.3096 at $\varepsilon = 64$. The latter central run is the strongest formal point, yet it remains 0.4172 below the unperturbed two-hop reference and 0.3772 below the boundary-compliant cut-edge baseline. Formal contribution-row DP therefore eliminates the cross-client utility advantage in every tested setting.

F1 and MCC confirm the same outcome. At the strongest formal point they reach 0.3670 and 0.2878, compared with 0.6911 and 0.6572 without row DP and 0.6444 and 0.6077 for cut-edge. Local and central placement exchange order across the sweep, and neither consistently dominates the reported metrics. These small differences do not alter the common result. At $\varepsilon = 1$, analytic calibration reduces σ from the classical value 48.448 to 37.306, but both calibration choices remain near 0.24 AUPR.

Because even the weakest formal setting removes most utility, we additionally evaluate lower-noise Gaussian perturbations as utility diagnostics. These runs use the same clipping bound but do not carry an (ε, δ) claim. At noise multiplier 0.005, central placement reaches AUPR 0.6875 and approximately matches the cut-edge reference, while local placement reaches 0.6799 and remains slightly below it. Increasing the multiplier to 0.010 lowers them to 0.6472 and 0.6432, respectively. Even these smaller perturbations therefore produce a visible utility loss.

The classical $\varepsilon = 1$ calibration has an equivalent multiplier of 9.690, and the analytic $\varepsilon = 64$ endpoint still uses 0.253. The diagnostic settings use only 0.005 and 0.010, corresponding to standard deviations 0.025 and 0.050. Even the least noisy formal setting is therefore approximately 25 times noisier than the diagnostic 0.010 setting and 51 times noisier than the 0.005 setting. The region in which the aggregate rows return to the cut-edge utility range therefore requires noise levels far below the least noisy formally calibrated setting evaluated here.

Figure 5.15 brings the selected endpoints together. The classical result uses central placement at $\varepsilon = 1$, the analytic endpoint uses central placement at $\varepsilon = 64$, and the Gaussian diagnostic uses central perturbation at multiplier 0.005.

5. EXPERIMENTAL EVALUATION

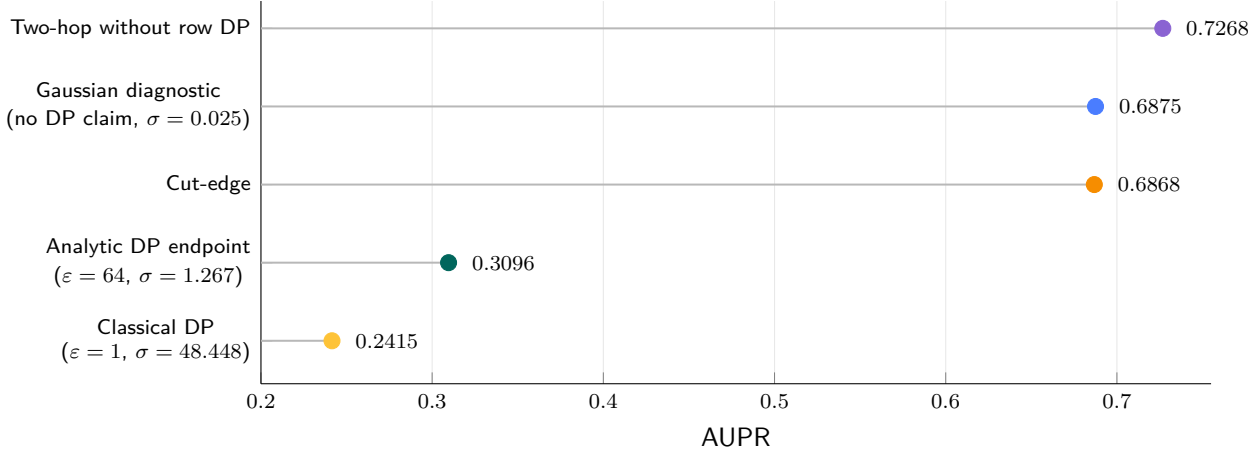


Figure 5.15: Selected contribution-row privacy and noise-diagnostic results in relation to the unperturbed two-hop and cut-edge references. The diagnostic result carries no formal differential-privacy guarantee.

Performance returns to the cut-edge range only at the diagnostic noise level for which no formal privacy claim was established.

Row DP changes the released values without changing the recipient plans or ciphertext layout. The formal runs therefore retain the same measured CKKS traffic as the unperturbed two-hop path: 22.33 GB uploaded and 7.45 GB downloaded. Its decisive cost is the complete loss of the cross-client utility advantage rather than additional communication. Appendix A.5.6 reports the complete formal and supplementary metric panels. The following section brings this result together with the utility-preserving mechanisms and the federated utility findings.

5.5 Synthesis of Experimental Findings

The reported findings rest on an experimental campaign broader than the retained comparisons alone. The campaign comprised 502 completed cluster jobs spanning synthetic-data generation, model development and calibration, principal model comparisons, and privacy-mechanism evaluation. Collectively, these jobs account for approximately 961 GPU-hours and 8183 allocated CPU-core-hours.² The synthesis below organizes the presented results into three broader findings.

First, centralized training provides the strongest retained reference, while the federated adaptation remains viable with each bank operating on its own graph. On the primary dataset, test AUPR decreases from

²These values are cumulative Slurm allocation totals for completed jobs through August 20, 2026.

0.8017 for the centralized GCN to 0.7194 for the cut-edge configuration. The shared federated model therefore retains substantial predictive utility, but model aggregation without cross-client graph propagation still leaves a clear gap.

Second, pre-aggregation improves on cut-edge training across all evaluated datasets. The result also persists with boundary-compliant features. Two-hop provides the strongest result on the primary symmetric dataset, whereas one-hop performs best in both asymmetric settings, where client heterogeneity is amplified. Cross-bank graph information is therefore consistently useful, although the preferable depth depends on the evaluated data distribution.

Third, the privacy mechanisms produce two distinct empirical outcomes. CKKS and pairwise masking preserve the intended aggregate computations and introduce little or no material utility change, although CKKS substantially increases communication and memory requirements. Formal contribution-row DP provides its stated guarantee for aggregate-row release, but its best-performing evaluated setting reaches only 0.3096 AUPR, compared with 0.6868 for the boundary-compliant cut-edge baseline. Lower diagnostic noise recovers utility only in configurations without a formal DP claim. Figure 5.16 organizes these findings by research question and indicates the broader issues to which they lead.

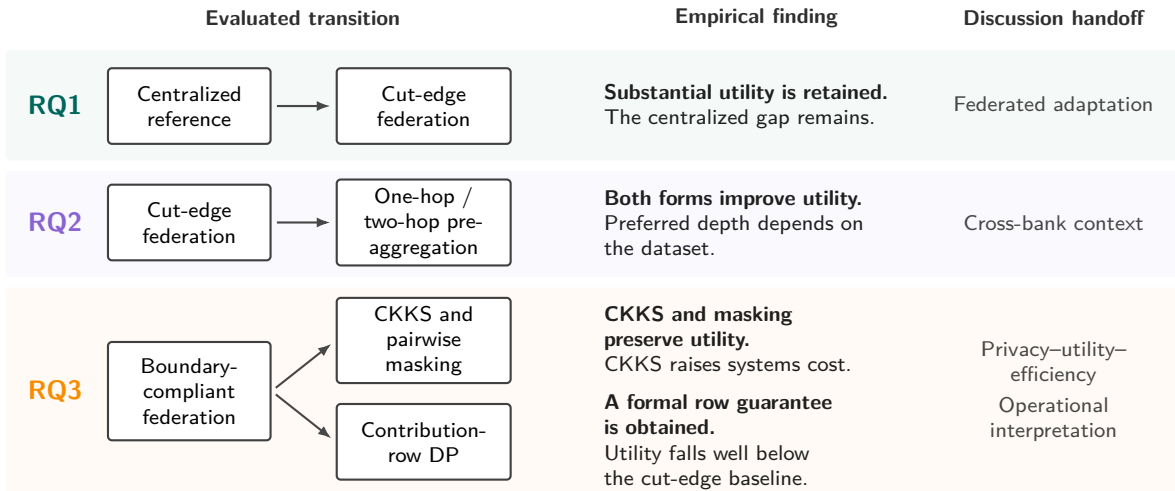


Figure 5.16: Principal experimental findings across the three research questions. The lanes represent distinct comparisons rather than a continuously modified configuration.

These findings form the empirical basis for RQ1–RQ3, not their complete answers. Chapter 6 combines this numerical evidence with the broader argument, implications, and limitations.

6 Discussion

6.1 Federated Adaptation and Cross-Bank Context

The cut-edge results establish that GCN-based AML remains viable when training is divided among bank clients, yet a clear performance gap to centralized training remains. In this configuration, each bank learns from its local graph while FedAvg combines the resulting model updates. Missing cross-bank propagation is therefore one contributor to the observed gap, although its effect cannot be isolated completely because federation also changes the optimization process. The one-hop and two-hop comparisons identify the contribution of cross-bank context more directly. Both improve on cut-edge training across the principal datasets, and the supporting robustness checks preserve this result. Useful graph signal can therefore be recovered through selected aggregate rows without reproducing the complete cross-bank graph at each client. This agrees with the motivation behind FedGCN’s one-shot feature aggregation [24], while establishing the benefit specifically for the bank-owned AML setting considered here.

Cross-client scope is not, however, a monotonic proxy for utility. Two-hop leads on the primary dataset, whereas one-hop leads in both asymmetric settings. Measured against the corresponding centralized references, the AUPR reduction increases from 6.4% to 8.2% for one-hop and from 5.2% to 10.1% for two-hop between the symmetric and moderately asymmetric settings. Because these reductions are computed within each dataset, the pattern is not merely a consequence of the lower positive rate and lower overall AUPR under moderate asymmetry. The matched-reference comparison in Figure 6.1 therefore strengthens the evidence that heterogeneous bank partitions impose an additional penalty on federated training.

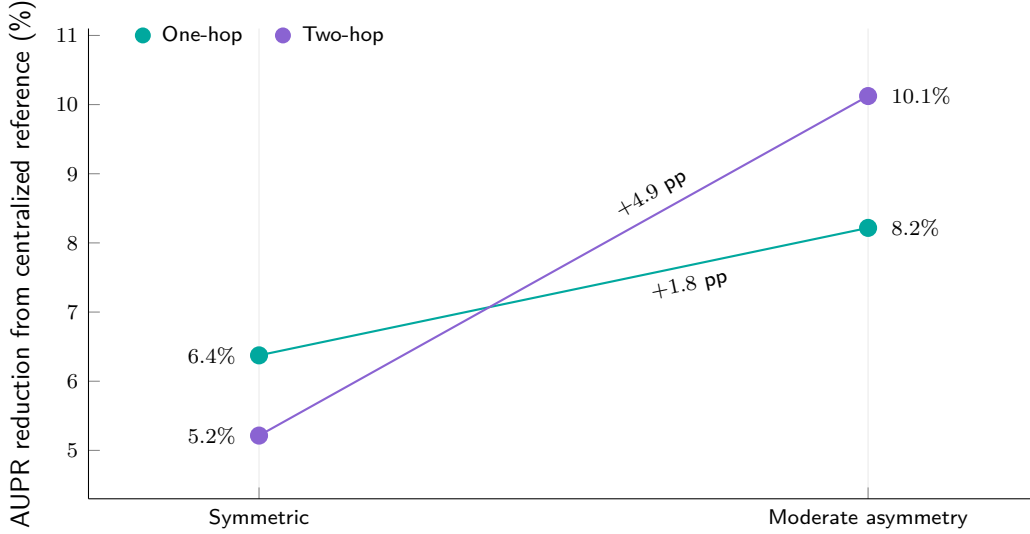


Figure 6.1: Relative reduction in AUPR for one-hop and two-hop compared with the corresponding centralized reference. Line labels report the increase from the primary symmetric to the moderately asymmetric dataset in percentage points.

The larger reduction under moderate asymmetry may also provide context for the change in the one-hop/two-hop ordering. When bank distributions are comparatively aligned, aggregate representations constructed around foreign nodes may reinforce predictive structures that are also useful in the local graph. As those distributions diverge, the same representations may reflect different local objectives. One-hop may be less affected because it retains direct cross-bank context without extending as far into the foreign neighborhood as two-hop. This interpretation is consistent with broader federated learning evidence that distributional heterogeneity can separate local from global optimization objectives [25]. In the graph setting, FED-PUB similarly treats subgraph heterogeneity as a challenge for uniform model aggregation and instead weights client models according to their functional similarity [86]. Together, these studies make the observed sensitivity to bank heterogeneity plausible and motivate cross-client scopes that adapt to the participating subgraphs. Representation compatibility therefore remains a plausible, but not independently tested, explanation for both the larger percentage reduction from centralized performance under moderate asymmetry and the changed one-hop/two-hop ordering.

This dataset dependence makes cross-client depth a joint utility, exposure, and efficiency choice. Both configurations recover only part of the gap to centralized training, while two-hop delivers 43–46% more recipient

rows than one-hop in the two principal datasets. Its wider scope produces an additional utility gain on the primary dataset, but not in either asymmetric setting. Communication depth should therefore be selected for the expected bank partition and exposure budget rather than treating two-hop as the default. In this comparison, depth describes the extent of the cross-client exchange. The retained three-layer model performs an additional propagation step only within the client graph and therefore does not reproduce the receptive field of a centralized three-layer model.

A larger row set is not by itself a measure of privacy loss, and the delivered aggregates are not raw foreign feature rows. It does, however, widen the set of graph-derived values available to recipient banks. The same exchange that recovers predictive utility therefore creates information surfaces at the coordinator during aggregation and at the recipients after release. This shifts the discussion from whether cross-bank context is useful to which of these surfaces can be protected without surrendering the recovered utility or making the protocol impractical.

6.2 Privacy–Utility–Efficiency Trade-offs

Plain one-hop and two-hop pre-aggregation occupy an intermediate position between keeping foreign information local and providing a formally protected release. Recipient banks do not receive raw foreign feature rows. They do, however, receive deterministic aggregate rows derived from foreign features and propagation coefficients. The recipient does not know their complete weighted decomposition: coefficients for foreign source nodes depend in part on the degrees of those nodes, which are not disclosed to it, while the corresponding feature values also remain unknown. Recipient-specific routing limits those rows to the local computations that require them. This reduces the scope of disclosure, but it does not constrain what a bank may infer from the rows it legitimately receives.

The resulting inference surface differs between the two cross-client depths. Aggregation may make one contribution difficult to isolate when several unknown inputs are mixed, but that practical concealment depends on the recipient’s knowledge and the number of contributors. One-hop supplies aggregate rows centered on a recipient’s owned accounts. Since the bank knows its local inputs, it may be able to remove locally reproducible components and narrow the remaining foreign aggregate. This becomes more revealing when only one or a few foreign contributions remain. Two-hop additionally supplies aggregate rows centered on selected foreign nodes as placeholders. These rows remain aggregates rather than raw features. Their placeholder role does not by itself imply that an individual row is easier to reconstruct; the concern is the broader set of foreign representations available for joint analysis. Recovering underlying infor-

mation would require a deliberate inference or reconstruction attempt, but not necessarily a deviation from the protocol. FedGCN identifies the single-cross-client-neighbor case as problematic, and OptimES warns that directly communicated FedGCN aggregates can leak private data [24, 64]. For the related release of learned boundary embeddings, the FedGraph-VASP preprint reports partial feature reconstruction: its inversion model achieves a coefficient of determination $R^2 = 0.32$, while membership inference reaches an AUC of 0.95 [59]. These findings establish a credible recipient-side risk, but not exact reconstruction of the aggregate rows used here. Since the thesis does not evaluate such attacks, routing should be treated as reducing exposure rather than establishing privacy.

This unresolved recipient exposure clarifies the narrower role of the two computation-preserving mechanisms. CKKS hides plaintext contribution values from the coordinator during pre-training aggregation, but does not hide the global topology required for that role. Pairwise masking hides each individual plaintext model update from the coordinator while still allowing it to compute the aggregate model update. Both mechanisms preserve their respective aggregate computations while leaving the routed aggregate rows visible to their recipients. They therefore preserve computational fidelity, not recipient privacy.

Utility preservation does not make the encrypted path operationally free. On the primary graph, serialized CKKS traffic totals 20.88 GB for one-hop and 29.78 GB for two-hop, with peak memory of 26.2 GiB and 34.8 GiB. Relative to the plaintext feature payload, encryption expands the serialized representation by approximately a factor of 40.8 under the fixed configuration. The broader two-hop release is consequently also more expensive to protect. Additional banks or denser cross-bank connectivity can increase the number of rows required by several recipients and amplify the operational burden further. Account count therefore provides only the baseline growth: if larger graphs also introduce more cross-bank neighborhoods or more recipients for the same rows, communication can grow faster than account count alone.

Pairwise masking presents a different systems question because it does not inherently enlarge the model-update vector. The doubling of upload bytes follows from the prototype’s float64 representation rather than from the mask construction itself. Conversely, a complete secure-aggregation protocol would require cryptographic setup, dropout recovery, and stronger adversary handling absent from the prototype. Bonawitz et al. implement such a failure-tolerant construction through pairwise key agreement, masking, and secret sharing. For 1,024 participants, each contributing approximately one million 16-bit values, they report a 73% communication increase over plaintext aggregation [52]. At a scale closer to an institutional banking collaboration, DHSA implements a cross-silo construction combining multi-key homomorphic encryption with pseudorandom masking. With ten clients, it reports approximately 50% higher traffic than plain

FedAvg [87]. Together, these results show that fuller protocols introduce measurable communication overhead, although its magnitude depends on the construction and deployment setting. The present experiment therefore establishes mask cancellation and the intended coordinator view, not the cost or security of a production secure-aggregation deployment.

Contribution-row DP instead addresses the values released to recipient banks. It also exposes the central privacy–utility conflict because it perturbs the same representation through which cross-bank information benefits the local model. The protected object is simultaneously the useful model input and the numerical release whose dependence on one source contribution is being bounded. Even at the weak endpoint $\varepsilon = 64$, the strongest formal result reaches only 0.3096 AUPR. Its calibrated noise remains approximately 51 times larger than the diagnostic setting at which central perturbation returns to the cut-edge utility range. The formally protected release is therefore far outside the utility-preserving noise regime observed in this model path.

This bounded negative result does not imply that formal graph privacy must eliminate predictive utility. GAP reports useful performance under edge- and node-level DP in centralized graph learning. The CE-FedGNN preprint reports limited utility loss at moderate embedding-noise levels and separately derives a metric-DP guarantee for the released representations [55, 68]. Their protected units, release schedules, architectures, and datasets differ from the contribution-row mechanism, so their numerical results and guarantees are not directly comparable. They nevertheless show that the choice of representation, privacy unit, and perturbation point can move the utility frontier. A useful formal release mechanism for the present protocol may therefore require a redesigned representation or protected unit rather than only different parameter values.

Taken together, the evaluated configurations leave an important part of the privacy frontier unoccupied. Plain one-hop and two-hop preserve useful cross-bank signal but leave recipient inference unresolved. CKKS and pairwise masking reduce coordinator visibility in the two communication lanes without materially changing utility, but leave the recipient view unchanged. Formal contribution-row DP protects the release only at a utility level below cut-edge training throughout the tested range. No evaluated configuration therefore combines useful cross-bank utility with formal recipient-release privacy. The systems implications of filling that gap remain open. CKKS shows that utility-preserving protection can remain executable at the evaluated scale despite a considerable increase in traffic and memory, but a utility-preserving recipient-release mechanism may have a different cost profile. The relevant question is therefore whether such a mechanism can retain useful model performance without imposing prohibitive computational or communication costs.

6.3 Operational Interpretation for Cross-Bank AML

This trade-off does not by itself identify a production architecture. The experiments establish mechanism behavior under a local simulation and a stated honest-but-curious threat model; they do not determine which institution should hold the required information or which residual exposure is legally and operationally acceptable. They do, however, clarify how a possible deployment would need to distribute trust between the coordinator and participating banks in an operational version of the evaluated protocol.

The protocol requires the coordinator to hold the global topology because it must derive the propagation coefficients and recipient plans from that structure. A regulator, a jointly governed service, or another institution trusted by all participating banks could plausibly occupy this role. Collaborative AML initiatives such as Project Aurora show that institutionally coordinated analysis with privacy-enhancing technologies is operationally relevant, while also identifying legal, technical, and organizational barriers [71]. The appropriate protection also depends on the coordinator’s trust boundary. If it is trusted with both structure and payload values, CKKS may not justify its substantial cost. If it is trusted to manage topology and execute the protocol but should not inspect source contributions, the encrypted aggregation path addresses that narrower requirement directly.

Within this conditional coordinator model, one-hop provides a reasonable starting point for further deployment study. It improves over cut-edge training in every evaluated dataset and, unlike two-hop, does not require additional aggregate-placeholder delivery. Its narrower release is particularly relevant because it also produced the strongest results in both asymmetric settings, where client heterogeneity is amplified. This does not make it private or universally preferable. Two-hop remains strongest on the primary dataset and may be justified when participating banks have more closely aligned distributions and its broader context produces a meaningful gain.

For a coordinator trusted with topology but not source contributions, the encrypted pre-training path could be combined with a complete secure-aggregation protocol for FedAvg. With independent key management and authenticated, integrity-protected communication, this combination would protect both communication lanes against external observation. An external attacker who intercepts network traffic or gains access to the server-side messages would encounter encrypted contribution values and protected model updates rather than client-specific plaintext, provided that the bank endpoints and cryptographic keys remain secure and the secure-aggregation protocol’s adversary assumptions hold. The principal

unresolved exposure would therefore be the aggregate rows intentionally released to recipient banks.

Given the observed contribution-row DP loss, a utility-oriented stack would accept rather than formally protect the one-hop recipient release. Whether this release is institutionally acceptable cannot be determined from its aggregate form alone. The FATF documents collaborative AML arrangements involving encrypted queries, shared risk indicators, and joint transaction monitoring, but also identifies legal uncertainty and jurisdiction-specific restrictions that continue to constrain such collaboration. It consequently emphasizes necessity, data minimization, purpose limitation, and appropriate retention rather than treating collaborative exchange as generally permissible [19]. Current practice is therefore restricted, although protected information sharing is not categorically excluded.

A more explicit legal basis is emerging in the European Union. From 10 July 2027, Article 75 of Regulation (EU) 2024/1624 will permit supervised information-sharing partnerships to exchange transaction information, customer risk factors, risk analyses, and suspicions where strictly necessary. The framework requires supervisory involvement, a data-protection impact assessment, recorded exchanges, differentiated access, and restrictions on further transmission [88]. Its future application means that it does not establish the acceptability of the evaluated exchange today. It does, however, provide concrete forthcoming criteria against which the aggregate-row release could be assessed. If that bounded release remains unacceptable, the evaluated mechanisms do not provide a utility-preserving alternative with a formal recipient guarantee.

The evaluated GCN would serve most naturally as a screening component within an investigative workflow. It ranks accounts by predicted risk but does not produce a case-level explanation for an individual score. This role is consistent with Basel Committee guidance, under which automated transaction-monitoring systems generate alerts that remain subject to further assessment by AML/CFT officers [12]. The forthcoming EU framework similarly requires adequate human oversight when outputs generated by AI or machine learning are exchanged within an information-sharing partnership. An institution must also assess received material itself before using it in a decision affecting a customer relationship [88]. An operational implementation could therefore use the model scores to prioritize analyst review, but would need to accompany them with locally inspectable transactions, neighborhood evidence, or another explanation layer before further action is taken.

Rather than returning only an account score, a more investigation-ready extension could identify a localized candidate pattern of related accounts and transactions. GAGNN illustrates a related direction by combining node-, transaction-, and group-level objectives to detect organized laundering behavior [61]. This would not by itself explain why the present

GCN assigned its score, but it could complement that score with a concrete structure for analysts to examine. In a cross-bank setting, releasing such a richer structure could expose more foreign information than an account-level score and would therefore require its own access-control and privacy design.

Finally, the controlled scale comparison does not establish operational throughput. It shows that serialized CKKS traffic grows with the routed ciphertext blocks under fixed bank proportions and graph-generation settings. Larger transaction networks, denser cross-bank connectivity, or additional participating banks can increase the rows delivered to multiple recipients and produce a larger multiplier than account count alone. Establishing operational suitability would therefore require a networked benchmark of the complete protocol, together with optimization of row routing, ciphertext packing, and duplicate delivery. The evaluated evidence supports a conditional architecture for further study, not a claim of production readiness. Its remaining assumptions define the validity limits considered next.

6.4 Limitations and Threats to Validity

The preceding arguments rest on a controlled experimental design whose boundaries determine how far its findings can be generalized. These boundaries concern the generated data, the learning task, the strength of the experimental comparisons, and the simulated privacy and systems setting. They do not negate the comparisons within the evaluated conditions, but they restrict conclusions about real institutions and production deployments.

The most consequential external-validity boundary is the use of synthetic data. Transaction behavior, account labels, bank assignments, and cross-bank patterns are generated through AMLSim rather than observed in operating institutions. The three dataset variants introduce controlled differences in bank populations, initial-balance ranges, and laundering-pattern allocations, but several properties change jointly between them. In particular, the strongly asymmetric dataset is supporting evidence from a second asymmetric partition rather than a third point on a single scale of difficulty. Results across these datasets therefore establish how the implemented configurations behave under the generated conditions. They cannot confirm that the observed utility or the preference for one-hop under heterogeneity would carry over to real bank data. The three-bank design similarly does not establish how the protocol behaves as the number and relative sizes of participating institutions change. The principal centralized and federated comparisons use the full 36-feature representation and therefore precede enforcement of the stricter feature boundary. The matched 32-feature comparison lowers absolute utility but preserves

the cross-client gains, while all privacy-mechanism experiments use this boundary-compliant representation. The earlier values should therefore not be interpreted as deployment estimates under the stricter boundary, although their qualitative cross-client pattern remains intact.

The graph-learning setting introduces a separate scope boundary. Each dataset is represented as a static graph aggregated over its observation period, and the model performs transductive account classification on a fixed node split. The experiments neither preserve transaction order nor evaluate future graph states, newly arriving accounts, or changes in laundering behavior. They therefore do not establish temporal or inductive performance under graph drift. These choices still permit comparison of the model and federated configurations on the same fixed graphs, but they narrow the claim to that setting.

Centralized and federated training are calibrated references rather than exactly compute-matched executions. A centralized epoch and a client-local epoch process different graph structures, while federation adds repeated model aggregation and, for one-hop and two-hop, a different first-stage computation. Consequently, their utility gap cannot be attributed exclusively to missing cross-bank propagation. The privacy-mechanism comparisons are stronger in this respect because they retain the same federated path and training schedule, but their cost measurements still reflect mechanism-specific numerical representations and operations.

The principal reported values come from one selected random seed. To examine whether the main conclusions depend on that choice, supporting four-seed panels were conducted. One-hop and two-hop outperform cut-edge at every tested seed on both principal datasets, while the CKKS comparisons retain only small differences in either direction. Because these panels use a shorter training schedule, they support the robustness of the observed ordering but do not provide uncertainty estimates for the final values. Checkpoint and operating-point selection use validation data, yet test metrics were computed and visible during development. The reported test set is therefore not a completely blinded final holdout. Together, these conditions support controlled comparative findings, but not population-level effect estimates or precise uncertainty claims.

The federated and privacy conclusions remain bounded by the implemented protocol. The coordinator retains the global topology, while recipient banks receive deterministic aggregate rows whose inference risk was not tested through reconstruction, membership, or attribute-inference attacks. Contribution-row DP nevertheless provides a formal guarantee for the only numerical feature payload released between client roles during pre-training. Each delivered aggregate row satisfies (ϵ, δ) -differential privacy with respect to substituting one clipped contribution row. This protects the inter-client pre-training release under the defined unit, but does not extend the guarantee to complete accounts, nodes, edges, client graphs, model updates, topology, or the end-to-end system.

Deployment claims are further limited by the single-process implementation. Clients and coordinator are logically separated roles rather than isolated institutional systems. CKKS uses one run-level client context rather than separate per-bank key provisioning, and pairwise masking assumes complete participation without production key establishment, secret sharing, or dropout recovery. Peak memory depends on the evaluated hardware, while serialized ciphertext volumes and model-byte accounting omit network latency, retransmission, and the infrastructure required by a complete protocol. These measurements characterize correctness and relative cost in the local implementation; they do not establish throughput, availability, malicious security, or production readiness.

Taken together, these limitations preserve a narrower conclusion. The experiments provide controlled evidence about the utility, visibility, and systems trade-offs of the implemented configurations on the generated graphs. They do not establish effectiveness on real banking networks or comprehensive privacy. Their operational evidence characterizes relative implementation costs rather than the readiness of a complete deployment. Resolving those boundaries forms the basis of the Outlook in the Conclusion.

7

Conclusion

7.1 Answers to the Research Questions

RQ1: Federated adaptation for AML. Established GCN and federated graph-learning methods can be adapted to bank-partitioned AML graphs by constructing client graphs over each bank’s account nodes, training locally, and combining model updates through FedAvg. To incorporate cross-bank neighborhood information, we realize FedGCN’s one-shot feature pre-aggregation [24] as a role-bounded protocol that routes only the required aggregate rows to each client. Conventional cut-edge federation retains substantial predictive utility but still leaves a clear gap to centralized training: AUPR decreases from 0.8017 centrally to 0.7194 on the primary dataset and from 0.7606 to 0.6529 under moderate asymmetry. Cross-client pre-aggregation materially narrows both gaps without closing either completely.

RQ2: Cross-bank graph information. Across the evaluated synthetic three-bank graphs, the strongest cross-client configuration recovers 49.3% of the observed centralized-to-cut-edge AUPR gap on the primary dataset and 42.0% under moderate asymmetry. Both one-hop and two-hop improve on cut-edge training in the symmetric and moderately asymmetric settings, as well as in the strongly asymmetric comparison and with boundary-compliant features. Two-hop performs best on the primary dataset, whereas one-hop leads in both asymmetric settings, where client heterogeneity is amplified. The preferable depth is therefore dataset-dependent, as broader cross-bank neighborhood information does not consistently yield higher utility.

RQ3: Privacy–utility–efficiency trade-offs. The central trade-off depends on whether protection targets server-visible communication or the aggregate rows released to recipient banks. Recipient-routed CKKS hides plaintext contribution values from the coordinating server during pre-training aggregation. Under the protocol’s role division, the coordinator holds the global topology required for routing, and recipient banks decrypt their routed aggregate rows. Pairwise-masked FedAvg hides individual plaintext model updates from the simulated coordinator while preserving the aggregate update. Together, these mechanisms reduce coordinator visibility in both communication lanes while leaving predictive

utility materially unchanged from their corresponding plaintext configurations, but they do not protect aggregate rows after delivery to their recipients.

Under the evaluated configuration, CKKS expands serialized pre-training traffic more than fortyfold and substantially increases peak memory, whereas pairwise masking does not inherently enlarge the model-update vector. The masking implementation validates cancellation under complete participation, but does not establish the security or full cost of a complete secure-aggregation protocol.

Contribution-row differential privacy instead protects the recipient-facing release with respect to substituting one clipped contribution row. Across all tested formal settings, predictive utility falls substantially below boundary-compliant cut-edge training, with the strongest result reaching only 0.3096 AUPR, thereby leaving little useful predictive utility. No evaluated configuration therefore combines useful cross-client utility with formal recipient-release privacy.

7.2 Outlook

The most immediate next step would be to quantify what recipient banks can infer from the routed aggregate rows. Relevant experiments include controlled reconstruction, membership-inference, and attribute-inference attacks on these releases. They should vary the number of foreign contributors and whether the release is an owned-target row in one-hop or a foreign-node placeholder row in two-hop. This would clarify how much protection aggregation and routing provide in practice and whether an additional explicit privacy guarantee is required. The resulting evidence could inform an assessment under the forthcoming EU framework for information-sharing partnerships, particularly its requirements for data minimization and differentiated access [88]. If the release remains unacceptable, more targeted safeguards should be investigated, including minimum-contributor thresholds, suppression or coarsening, alternative formal privacy units, and protected first-layer computation.

Model development provides a second direction. The present system uses a compact static GCN to support controlled federated and privacy comparisons. On their AMLSim dataset, Karim et al. report higher AUPR for several more advanced centralized graph-learning variants than the compact GCN reaches in our centralized setting [16]. Although differences in the generated data and experimental pipeline prevent a controlled comparison, their results make improved performance through richer models plausible. The role-bounded FedGCN pre-aggregation could therefore be adapted to temporal graph models, richer directed representations, and multi-objective models that jointly assess accounts, transactions, and laundering groups [61]. Temporal modeling would also align the system

7. CONCLUSION

more closely with the evolving transaction activity encountered in operational AML.

These model extensions should be considered together with greater cross-client depth. Future work could test three, four, and higher pre-aggregation depths with models whose receptive fields can use the additional information. The changing one-hop/two-hop ordering further motivates client-specific or data-dependent depth rather than one global choice. At the same time, the larger federated performance reduction under heterogeneity motivates moving beyond standard FedAvg toward heterogeneity-aware or personalized model aggregation [86].

These extensions should then be validated beyond synthetic three-bank graphs. Securely accessed institutional data from more and differently sized banks would test whether the centralized-to-federated gap, the value of cross-bank information, and the preferred exchange depth persist under operational data distributions. Temporal holdouts, evolving graphs, and newly arriving accounts would additionally test performance beyond the fixed transductive setting. Understanding recipient exposure remains necessary before aggregate rows are released in an operational collaboration, although controlled real-data validation could proceed within an appropriately governed environment. If institutional access remains unavailable, newer generators such as Tide and TransXion could provide an intermediate validation step across more varied synthetic conditions, provided the generators are configured for the required bank-partitioned graph setting [56, 57].

Finally, the local role simulation should be replaced by a distributed implementation with independently provisioned bank processes, authenticated communication, and a complete secure-aggregation protocol that handles dropout under an explicit adversary model. Evaluation across the expanded graph, bank, and cross-client-depth settings should measure end-to-end latency, throughput, communication, and memory while examining whether more efficient routing and ciphertext packing reduce the observed overhead. This would establish whether the complete protocol is operationally feasible at realistic graph and federation scales.

Supplementary Material

A

A.1 Dataset and Generator Configurations

A.1.1 IBM AML HI-Large Suitability Audit

Table A.1: Top 10 HI-Large institutions ranked by transaction participation, defined as transactions with at least one endpoint at the respective institution. Laundering-associated accounts are incident to at least one transaction labeled as laundering.

Rank	Institution ID	Accounts	Transaction participation	Laundering-associated accounts, n (%)
1	70	15	17 226 741	15 (100.00%)
2	12	3 623	1 300 052	1 185 (32.71%)
3	11	3 428	1 296 131	1 088 (31.74%)
4	27	3 800	1 245 503	587 (15.45%)
5	0	2 895	1 207 498	1 098 (37.93%)
6	20	2 907	1 178 870	1 091 (37.53%)
7	112	2 835	820 097	244 (8.61%)
8	29	2 340	764 123	282 (12.05%)
9	14	2 104	742 881	581 (27.61%)
10	15	2 585	719 038	264 (10.21%)

Collectively, the ten institutions with the highest transaction participation contain 26,532 accounts, corresponding to 1.25% of all HI-Large accounts, and retain 6,435 laundering-associated accounts, corresponding to 2.89% of all such accounts. Transactions involving at least one of these institutions account for 14.51% of the dataset, but the strict induced subgraph retains only 958,331 transactions (0.53%), including 427,769 cross-bank transactions and 2,165 transactions labeled as laundering (0.96% of all laundering-labeled transactions).

A.1.2 AMLSim Preliminary Calibration

Table A.2 reports the properties most directly tied to the principal calibration decisions. The displayed measures are not exhaustive. Across the broader calibration and validation workflow, additional checks covered per-bank activity, weak-component structure, repeated interactions, la-

bel alignment, laundering-pattern composition, and degree distributions. Section 4.1.2 describes the validation procedure applied to the retained datasets.

A.1.3 Primary Symmetric Three-Bank Dataset Profile

Tables A.3 and A.4 supplement the global dataset characteristics reported in Section 5.1.1 with their per-bank distribution.

The tables report realized output properties; the term symmetric refers to the near-equal configured bank populations and common generation inputs across banks rather than equal realized transaction activity. The broader validation procedure is described in Section 4.1.2.

A.1.4 Strongly Asymmetric Stress Dataset

Table A.5 summarizes the bank-level properties of the strongly asymmetric stress dataset.

A.2 Dataset Validation Coverage

Table A.6 records the scope of the dataset audit. It complements the methodological overview in Section 4.1.2 by collecting the checks that were applied at each stage without reproducing their implementation details. The common workflow was applied to all retained datasets, with additional shortcut diagnostics for the asymmetric variants. Table A.7 reports selected outcomes for the primary dataset in greater detail because it provides the basis for the principal experiments.

Table A.2: Principal AMLSim calibration stages evaluated with 100,000 accounts. Rows that combine multiple iterations report the final iteration. Transaction shares refer to realized individual transaction records.

Stage	Principal change	Account activity active / isolated	Graph volume records / directed pairs	Transaction shares within / cross-bank; laundering	Decision
Initial combined setup (v01)	Introduce bank-aware graph construction and the initial group preference.	60 705 / 39 295	7 442 984 / 515 139	55.38/44.62%; 0.1232%	Continue group-preference calibration.
Group-preference calibration (v02--v04)	Tune group-aware account selection while retaining broader neighbor selection.	59 984 / 40 016	6 622 866 / 504 149	55.32/44.68%; 0.1376%	Tuned group preference retained; participation remained inadequate.
Participation coverage (v05)	Assign one-shot main roles to accounts without scheduled behavior.	99 968 / 32 / 45 798	6 703 285 / 559 751	55.30/44.70%; 0.1340%	Coverage repaired; continue flow and low-activity refinement.
Bank-flow refinement (v06)	Raise the within-bank target and introduce a low-degree top-up.	99 976 / 24 / 32 793	6 701 497 / 560 410	64.92/35.08%; 0.1357%	Bank-flow target approached; low-activity tail remained.
Low-activity refinement (v07)	Broaden the targeted top-up for accounts with low base-graph degree.	99 991 / 9 / 8 563	6 739 403 / 603 439	65.85/34.15%; 0.1342%	Retained for generation with one million accounts.
Existing-edge replay (v08)	Replay existing edges within generated laundering patterns.	99 991 / 9 / 8 684	6 745 854 / 602 936	65.84/34.16%; 0.1317%	Rejected: no increase in laundering-transaction share.
Additional pattern edges (v09)	Add previously unused edges within generated laundering patterns.	99 991 / 9 / 8 582	6 745 814 / 603 819	65.88/34.12%; 0.1330%	Rejected: no material increase in laundering-transaction share.

A. SUPPLEMENTARY MATERIAL

Table A.3: Account and label profile of the primary symmetric three-bank dataset. Percentages are relative to the account count of the corresponding bank.

Bank	Accounts	SAR-labeled accounts n (%)
Bank A	333 334	33 853 (10.16%)
Bank B	333 333	33 304 (9.99%)
Bank C	333 333	33 368 (10.01%)
Total	1 000 000	100 525 (10.05%)

Table A.4: Transaction profile of the primary symmetric three-bank dataset. Records are assigned by their origin bank. Percentages are relative to the total originating records of the corresponding bank.

Bank	Total transaction records	Within-bank records n (%)	Cross-bank records n (%)	Laundering-pattern records n (%)
Bank A	47 304 506	34 399 433 (72.72%)	12 905 073 (27.28%)	28 189 (0.0596%)
Bank B	10 714 082	5 641 039 (52.65%)	5 073 043 (47.35%)	31 307 (0.2922%)
Bank C	10 488 681	5 678 297 (54.14%)	4 810 384 (45.86%)	31 189 (0.2974%)
Total	68 507 269	45 718 769 (66.74%)	22 788 500 (33.26%)	90 685 (0.1324%)

Table A.5: Selected properties of the strongly asymmetric stress dataset. Initial-balance amounts are in USD. Percentages are relative to the account count of the corresponding bank.

Scope	Accounts	Initial-balance range	Laundering patterns	SAR-labeled accounts n (%)
Bank A	500 000	25 000–70 000	450	9 839 (1.97%)
Bank B	300 000	60 000–150 000	1 350	15 542 (5.18%)
Bank C	200 000	150 000–500 000	4 200	35 382 (17.69%)
Cross-bank	–	–	2 250	–
Total	1 000 000	–	8 250	60 763 (6.08%)

Table A.6: Coverage of the dataset-validation workflow.

Validation layer	Scope	Checks and diagnostics	Purpose
Output integrity	All retained datasets	Required files; account identifiers; bank assignments; transaction endpoint resolution; account and transaction counts	Detect missing outputs, ambiguous account mappings, and transactions involving unknown accounts or banks
Target semantics	All retained datasets	SAR filtering; duplicate target rows; overlap and differences between <code>alert_accounts.csv</code> and <code>sar_accounts.csv</code> ; account-to-node label alignment	Ensure that the complete SAR-designated membership roster defines the binary account target without unexplained graph labels
Participation	All retained datasets	Active and isolated accounts; incoming, outgoing, and total transaction counts; low-activity distributions; per-bank activity summaries	Verify broad account participation and usable client-level transaction coverage
Transaction and topology breadth	All retained datasets	Record and unique-pair counts; repeated interactions; in/out degree; reciprocity; weak-component count and largest-component coverage	Characterize the relational breadth and connectivity available for graph learning
Multi-bank and laundering structure	All retained datasets	Within- and cross-bank transaction totals; directed bank-flow matrix; per-bank positive availability; bank-restricted and cross-bank pattern groupings; laundering-transaction share	Confirm that the generated output retains the intended multi-bank and labeled laundering structure
Graph integrity	All retained datasets	Source-account and graph-node correspondence; edge-index bounds; duplicate edges; self-loops; isolated nodes; bidirectional target-label agreement	Verify that graph construction preserves the accepted source tables and target semantics
Feature integrity	All retained datasets	Matrix shape and feature names; missing and non-finite values; distributional tails; class-conditional summaries	Detect malformed features and numerical failures
Split integrity	All retained datasets	Split sizes; pairwise overlap; complete node coverage; global and per-split positive counts and rates	Ensure disjoint evaluation partitions with viable positive examples
Heterogeneity diagnostics	Moderately and strongly asymmetric variants	Bank-only prevalence baseline; univariate feature rankings; exact binary label-like features; bank-level feature shifts	Test whether simple bank or feature shortcuts explain performance on the asymmetric datasets
Visual corroboration	Calibration and retained datasets	Bank-flow diagrams; laundering-pattern scope; degree and activity distributions; component and participation views	Corroborate quantitative findings and expose structural anomalies not apparent from aggregate counts alone

Table A.7: Selected validation and acceptance outcomes for the primary symmetric three-bank dataset.

Validation measure	Outcome	What it establishes
Output, target, and split integrity	No integrity failures detected	Transaction endpoints resolved, labels aligned exactly, and evaluation partitions disjoint and complete
SAR-labeled accounts represented in the transaction-derived roster	96 270 / 100 525 (95.77%)	Shows why complete pattern membership is required as the label source
Isolated accounts	86	Broad account participation
Largest weak component	999 549 (99.9549%)	Near-global weak connectivity
Unique directed account pairs	8 011 542	Relational breadth after consolidating repeated transactions
Directed-pair degree, median / 95th percentile	2 / 27	Typical and upper-range relational context
Cross-bank transaction records	33.2644%	Substantial cross-bank activity

A.3 Node Feature Definitions

Tables A.8 and A.9 define the 36 node features introduced in Section 4.2.2. Let τ denote an individual transaction record and let $a(\tau)$ denote its amount. For account i , $\mathcal{T}_i^{\text{in}}$ and $\mathcal{T}_i^{\text{out}}$ denote its incoming and outgoing transaction records, respectively. Let $\mathcal{N}^{\text{in}}(i)$ and $\mathcal{N}^{\text{out}}(i)$ denote the incoming and outgoing neighborhoods of account i . Empty feature values are represented by zero. Amount denominators are lower-bounded by 10^{-6} where necessary.

For the relational features, d_i^{in} and d_i^{out} denote the inbound and outbound degrees of account i . For account j , let $\nu_j = |\mathcal{T}_j^{\text{in}}| + |\mathcal{T}_j^{\text{out}}|$ and $d_j = d_j^{\text{in}} + d_j^{\text{out}}$.

Table A.8: Direct account and transaction feature definitions. Population standard deviation is used. All features in this table are included in the boundary-compliant feature representation.

Feature	Definition	Source	$\log(1+x)$
Initial balance	Initial account balance	Account table	Yes
Inbound record count	$ \mathcal{T}_i^{\text{in}} $	Transaction records	Yes
Outbound record count	$ \mathcal{T}_i^{\text{out}} $	Transaction records	Yes
Inbound amount sum	$m_i^{\text{in}} = \sum_{\tau \in \mathcal{T}_i^{\text{in}}} a(\tau)$	Transaction records	Yes
Outbound amount sum	$m_i^{\text{out}} = \sum_{\tau \in \mathcal{T}_i^{\text{out}}} a(\tau)$	Transaction records	Yes
Inbound amount mean	Mean of $a(\tau)$ over $\mathcal{T}_i^{\text{in}}$	Transaction records	Yes
Outbound amount mean	Mean of $a(\tau)$ over $\mathcal{T}_i^{\text{out}}$	Transaction records	Yes
Inbound amount maximum	Maximum $a(\tau)$ over $\mathcal{T}_i^{\text{in}}$	Transaction records	Yes
Outbound amount maximum	Maximum $a(\tau)$ over $\mathcal{T}_i^{\text{out}}$	Transaction records	Yes
Inbound amount standard deviation	Population standard deviation of $a(\tau)$ over $\mathcal{T}_i^{\text{in}}$	Transaction records	No
Outbound amount standard deviation	Population standard deviation of $a(\tau)$ over $\mathcal{T}_i^{\text{out}}$	Transaction records	No
Inbound amount median	Median of $a(\tau)$ over $\mathcal{T}_i^{\text{in}}$	Transaction records	No
Outbound amount median	Median of $a(\tau)$ over $\mathcal{T}_i^{\text{out}}$	Transaction records	No
Inbound counterparties	$ \mathcal{N}_i^{\text{in}}(i) $	Transaction endpoints	Yes
Outbound counterparties	$ \mathcal{N}_i^{\text{out}}(i) $	Transaction endpoints	Yes
Total counterparties	$ \mathcal{N}_i^{\text{in}}(i) \cup \mathcal{N}_i^{\text{out}}(i) $	Transaction endpoints	Yes
Inbound/outbound count ratio	$ \mathcal{T}_i^{\text{in}} / \max(\mathcal{T}_i^{\text{out}} , 1)$	Transaction records	No
Inbound/outbound amount ratio	$m_i^{\text{in}} / \max(m_i^{\text{out}}, 10^{-6})$	Transaction records	No
Net flow	$m_i^{\text{in}} - m_i^{\text{out}}$	Transaction records	No
Relative net flow	$(m_i^{\text{in}} - m_i^{\text{out}}) / \max(m_i^{\text{in}} + m_i^{\text{out}}, 10^{-6})$	Transaction records	No

Table A.9: Relational and bank-aware feature definitions. The boundary-compliant column indicates inclusion in the feature representation introduced in Section 4.5.

Feature	Definition	Source	$\log(1+x)$	Boundary-compliant
Inbound pair degree	$d_i^{\text{in}} = \mathcal{N}^{\text{in}}(i) $	Account-pair graph	Yes	Yes
Outbound pair degree	$d_i^{\text{out}} = \mathcal{N}^{\text{out}}(i) $	Account-pair graph	Yes	Yes
Total directed-pair degree	$d_i^{\text{in}} + d_i^{\text{out}}$	Account-pair graph	Yes	Yes
Weighted inbound pair degree	$\sum_{u:(u,i) \in \mathcal{E}} n_{ui} = \mathcal{T}_i^{\text{in}} $	Pair transaction counts	Yes	Yes
Weighted outbound pair degree	$\sum_{v:(i,v) \in \mathcal{E}} n_{iv} = \mathcal{T}_i^{\text{out}} $	Pair transaction counts	Yes	Yes
Weighted total pair degree	Weighted inbound plus weighted outbound pair degree	Pair transaction counts	Yes	Yes
Overall repeat intensity	$(\mathcal{T}_i^{\text{in}} + \mathcal{T}_i^{\text{out}}) / \max(\mathcal{N}^{\text{in}}(i) \cup \mathcal{N}^{\text{out}}(i) , 1)$	Records and pair graph	Yes	Yes
Inbound records per counterparty	$ \mathcal{T}_i^{\text{in}} / \max(\mathcal{N}^{\text{in}}(i) , 1)$	Records and pair graph	Yes	Yes
Outbound records per counterparty	$ \mathcal{T}_i^{\text{out}} / \max(\mathcal{N}^{\text{out}}(i) , 1)$	Records and pair graph	Yes	Yes
Cross-bank record share	Share of records incident to i whose other endpoint j satisfies $\text{bank}(j) \neq \text{bank}(i)$	Records and bank assignments	No	Yes
Cross-bank directed-pair share	Share of the $d_i^{\text{in}} + d_i^{\text{out}}$ incident directed pairs whose other endpoint belongs to another bank	Pair graph and bank assignments	No	Yes
Counterparty-bank diversity	$ \{\text{bank}(j) : j \in \mathcal{N}^{\text{in}}(i) \cup \mathcal{N}^{\text{out}}(i)\} $	Pair graph and bank assignments	Yes	Yes
Mean neighbor activity	Mean of ν_j over incoming and outgoing neighbors, counting each direction separately	Pair graph and record counts	Yes	No
Maximum neighbor activity	Maximum ν_j over incoming and outgoing neighbors	Pair graph and record counts	Yes	No
Mean neighbor pair degree	Mean of d_j over incoming and outgoing neighbors, counting each direction separately	Pair graph	Yes	No
Maximum neighbor pair degree	Maximum d_j over incoming and outgoing neighbors	Pair graph	Yes	No

A.4 Protocol Details and Mathematical Derivations

A.4.1 CKKS Transport Configuration

Table A.10 records the fixed CKKS configuration used for the encrypted pre-training experiments. With the boundary-compliant feature set, a full 64-row block contains $64 \times 32 = 2,048$ scalar values and fits within the packing capacity of the configured CKKS context.

Table A.10: Fixed CKKS and block configuration used for encrypted pre-training aggregation.

Parameter	Value
Software library	TenSEAL 0.3.16
Homomorphic-encryption scheme	CKKS
Polynomial modulus degree	8,192
Coefficient-modulus bit sizes	[60, 40, 40, 60]
Global scale	2^{40}
Feature dimension	32
Maximum rows per recipient block	64
Scalar values in a full block	2,048

A.4.2 Analytic Gaussian Calibration

The analytic contribution-row mechanism follows the scalar calibration of Balle and Wang [51]. Let Φ denote the standard normal cumulative distribution function. For candidate $v \geq 0$, the implementation evaluates

$$B_+(v) = \Phi(\sqrt{\varepsilon}v) - e^\varepsilon \Phi(-\sqrt{\varepsilon(v+2)}) - \delta, \quad (\text{A.1})$$

$$B_-(v) = \Phi(-\sqrt{\varepsilon}v) - e^\varepsilon \Phi(-\sqrt{\varepsilon(v+2)}) - \delta. \quad (\text{A.2})$$

It selects B_+ when $B_+(0) < 0$ and B_- otherwise. Starting from $[0, 1]$, the upper endpoint is doubled until the selected function changes sign. Bisection then locates its root v^* with tolerance 10^{-12} and a maximum of 10,000 iterations. The resulting scale factor is

$$\alpha = \sqrt{1 + v^*/2} \pm \sqrt{v^*/2}, \quad (\text{A.3})$$

where the minus sign is used for the B_+ branch and the plus sign for B_- . The calibrated standard deviation is

$$\sigma = \alpha \frac{\Delta_2}{\sqrt{2\varepsilon}}. \quad (\text{A.4})$$

Calibration rejects non-positive or non-finite ε and Δ_2 , values of δ outside $(0, 1)$, and parameter settings for which a finite root cannot be obtained.

A.5 Supplementary Experimental Results

A.5.1 Centralized Training Trajectory

Figure A.11 supplements the retained centralized reference reported in Section 5.2.2. It shows how the best observed validation AUPR continued to rise across the extended training horizon and marks the selected checkpoint.

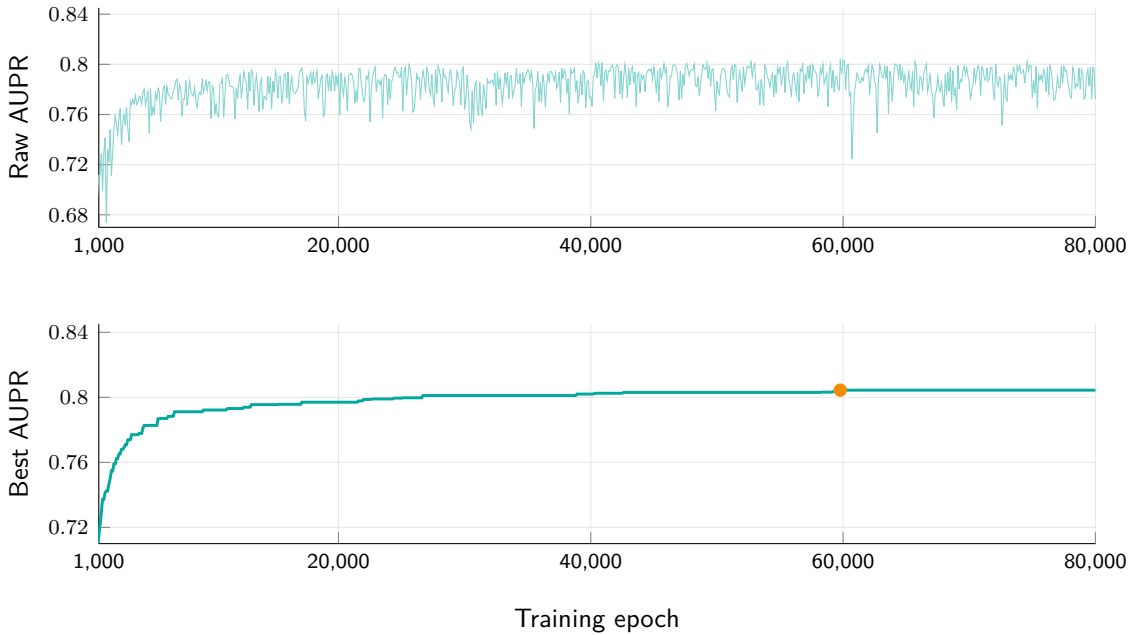


Figure A.11: Validation AUPR from epoch 1,000 onward for the retained primary run. The upper panel shows raw validation AUPR at 100-epoch intervals. The lower panel tracks the cumulative best using all evaluations logged every ten epochs and marks the selected checkpoint at epoch 59,780, where validation AUPR reaches its highest observed value.

A.5.2 Federated Configuration Calibration

Before the retained federated campaign, a matched comparison checked whether the three-layer model base remained suitable after partitioning the graph. All runs used the primary dataset, 600 federated rounds, eight local epochs per round, hidden width 128, dropout 0.2, learning rate 10^{-3} , weight decay 10^{-4} , and unit account-pair edges. The results in Table A.12

A. SUPPLEMENTARY MATERIAL

favor three layers in each federated configuration. These runs support the relative depth choice; they are not used as final utility references.

Table A.12: Matched federated model-depth comparison on the primary dataset. Values are validation AUPR after at most 600 rounds with eight local epochs per round.

Configuration	Two layers	Three layers	Difference
Cut-edge	0.4907	0.5607	+0.0700
One-hop	0.5439	0.6009	+0.0570
Two-hop	0.5801	0.6053	+0.0251

Learning rate and dropout were refined through targeted comparisons at different stages of the federated campaign rather than through one factorial sweep. Table A.13 records the comparisons that determined the retained dataset-specific values. The selected settings were subsequently carried into the local-epoch and final long-cap experiments.

Table A.13: Targeted federated learning-rate and dropout comparisons. Values are validation AUPR; dashes denote candidates not included in the respective comparison stage.

Configuration	Rounds	10^{-3}	2×10^{-3}	3×10^{-3}	Selected LR
<i>Symmetric</i>					
Cut-edge	1,000	0.5960	0.6338	–	2×10^{-3}
One-hop	2,500	–	0.7306	0.7243	2×10^{-3}
Two-hop	2,500	–	0.7245	0.7238	2×10^{-3}
<i>Moderate asymmetry</i>					
One-hop	1,000	0.5676	0.6164	0.6200	3×10^{-3}
Two-hop	1,000	0.5528	0.5939	0.6109	3×10^{-3}

Configuration	Dropout 0.1	Dropout 0.2	Selected dropout
<i>Moderate asymmetry, 5,000 rounds</i>			
Cut-edge	0.6519	0.6536	0.2
One-hop	0.6890	0.6782	0.1
Two-hop	0.6763	0.6602	0.1

A.5.3 Federated Utility Support

The seed panel covers two datasets, three federated configurations, and four fresh seeds (23, 39, 71, and 87), for 24 jobs in total. To keep this supporting robustness check computationally tractable, each run used 2,500 federated rounds and eight local epochs per round. The results support

the direction of the retained cut-edge, one-hop, and two-hop comparison but do not provide uncertainty estimates for the final 5,000-round runs. Table A.14 reports the corresponding mean and standard deviation across seeds.

Table A.14: Lower-budget four-seed support for the principal federated comparison. Entries report test-set mean $\pm$ standard deviation.

Configuration	AUPR	F1	MCC
<i>Symmetric</i>			
Cut-edge	0.6816 ± 0.0074	0.6425 ± 0.0054	0.6016 ± 0.0057
One-hop	0.7241 ± 0.0043	0.6829 ± 0.0028	0.6479 ± 0.0030
Two-hop	0.7307 ± 0.0037	0.6894 ± 0.0043	0.6544 ± 0.0049
<i>Moderate asymmetry</i>			
Cut-edge	0.6265 ± 0.0077	0.6208 ± 0.0052	0.5982 ± 0.0049
One-hop	0.6641 ± 0.0086	0.6546 ± 0.0046	0.6335 ± 0.0051
Two-hop	0.6582 ± 0.0076	0.6578 ± 0.0039	0.6370 ± 0.0041

The complete metric panel for the strongly asymmetric dataset appears in Table A.15. One-hop is strongest among the federated modes on all three measures, while the centralized reference remains stronger.

Table A.15: Retained centralized and federated results on the strongly asymmetric dataset.

Configuration	AUPR	F1	MCC
Centralized	0.8144	0.7610	0.7454
Cut-edge	0.6974	0.6758	0.6545
One-hop	0.7289	0.7147	0.6960
Two-hop	0.7041	0.7053	0.6860

A.5.4 Feature-Observability Boundary

Table A.16 reports the complete comparison referenced in Section 5.4.1. Within each training path, the 36- and 32-feature runs retain the same architecture and training budget. The difference rows summarize the observed change associated with the stricter feature representation.

Table A.16: Compute-aligned comparison of centralized and federated utility under the two feature representations. Differences report 32-feature minus 36-feature performance.

Training path	Feature set	AUPR	F1	MCC
Centralized	36 features	0.8017	0.7378	0.7105
	32 features	0.7743	0.7183	0.6886
	Difference	−0.0273	−0.0195	−0.0219
Cut-edge	36 features	0.7194	0.6729	0.6356
	32 features	0.6868	0.6444	0.6077
	Difference	−0.0326	−0.0285	−0.0279
One-hop	36 features	0.7506	0.7052	0.6732
	32 features	0.7248	0.6850	0.6510
	Difference	−0.0258	−0.0203	−0.0222
Two-hop	36 features	0.7599	0.7174	0.6864
	32 features	0.7268	0.6978	0.6644
	Difference	−0.0332	−0.0197	−0.0220

A.5.5 CKKS and Pairwise-Masked FedAvg Utility

Table A.17 provides the exact values behind the utility-preservation range reported in Section 5.4.2. The plain rows are the canonical boundary-compliant references from Table A.16; the protected rows use CKKS transport together with pairwise-masked FedAvg.

Table A.17: Predictive utility under combined CKKS transport and pairwise-masked FedAvg. AUPR differences are relative to the canonical plain result.

Mode	Mechanisms	AUPR	F1	MCC	Δ AUPR
One-hop	Plain	0.7248	0.6850	0.6510	–
	CKKS and masking	0.7204	0.6840	0.6506	−0.0044
Two-hop	Plain	0.7268	0.6978	0.6644	–
	CKKS and masking	0.7268	0.6911	0.6572	0.0000

A supporting paired panel repeats the plaintext and CKKS comparison across four additional random seeds. These runs use 2,500 rounds and eight local epochs, so they assess whether the observed utility behavior persists across initializations rather than estimate uncertainty for the retained 5,000-round results. Table A.18 reports the resulting CKKS-

minus-plaintext differences. One-hop varies in direction, whereas two-hop shows a small negative difference at each tested seed.

Table A.18: Paired CKKS-minus-plaintext test-AUPR differences across the supporting seed panel.

Mode	Random seed				Mean
	23	39	71	87	
One-hop	0.0009	0.0022	−0.0042	−0.0031	−0.0010
Two-hop	−0.0021	−0.0046	−0.0056	−0.0041	−0.0041

A.5.6 Contribution-Row DP Results

Table A.19 records the diagnostic Gaussian perturbation runs used to contextualize the formal contribution-row DP result. These settings use the same clipping bound as the formal panel, but their noise multipliers do not carry an (ϵ, δ) guarantee. Differences are measured against the unperturbed two-hop CKKS and pairwise-masking reference.

Table A.19: Diagnostic Gaussian contribution-row perturbation on the two-hop configuration. No formal differential-privacy claim applies.

Placement	Multiplier	σ	AUPR	AUROC	F1	MCC	Δ AUPR
None	–	–	0.7268	0.9453	0.6911	0.6572	–
Central	0.005	0.025	0.6875	0.9376	0.6557	0.6197	−0.0393
Local	0.005	0.025	0.6799	0.9370	0.6520	0.6146	−0.0469
Central	0.010	0.050	0.6472	0.9291	0.6283	0.5892	−0.0796
Local	0.010	0.050	0.6432	0.9279	0.6262	0.5853	−0.0836

Table A.20 provides the complete formal panel. All runs use the boundary-compliant two-hop configuration, CKKS transport, pairwise-masked FedAvg, $C_{\text{clip}} = 5$, $\Delta_2 = 10$, and $\delta = 10^{-5}$. The equivalent multiplier is σ/C_{clip} and is included only to compare noise scales; it is not the configured formal privacy parameter.

Table A.20: Complete formal contribution-row DP panel.

Placement	Calibration	ε	σ	Equivalent multiplier	AUPR	AUROC	F1	MCC
Local	Classical	1	48.448	9.690	0.2380	0.7029	0.3144	0.2253
Central	Classical	1	48.448	9.690	0.2415	0.7114	0.3081	0.2212
Local	Analytic	1	37.306	7.461	0.2382	0.7047	0.3053	0.2172
Central	Analytic	1	37.306	7.461	0.2384	0.7086	0.3138	0.2245
Local	Analytic	2	19.938	3.988	0.2470	0.7198	0.3233	0.2359
Central	Analytic	2	19.938	3.988	0.2435	0.7124	0.3105	0.2206
Local	Analytic	4	10.812	2.162	0.2484	0.7245	0.3269	0.2415
Central	Analytic	4	10.812	2.162	0.2515	0.7179	0.3151	0.2263
Local	Analytic	8	6.002	1.200	0.2508	0.7263	0.3286	0.2429
Central	Analytic	8	6.002	1.200	0.2536	0.7221	0.3218	0.2340
Local	Analytic	16	3.442	0.688	0.2545	0.7319	0.3266	0.2426
Central	Analytic	16	3.442	0.688	0.2538	0.7265	0.3220	0.2374
Local	Analytic	32	2.049	0.410	0.2861	0.7504	0.3556	0.2745
Central	Analytic	32	2.049	0.410	0.2829	0.7450	0.3451	0.2625
Local	Analytic	64	1.267	0.253	0.3057	0.7637	0.3689	0.2900
Central	Analytic	64	1.267	0.253	0.3096	0.7644	0.3670	0.2878

References

- [1] Financial Crimes Enforcement Network. *What Is Money Laundering?* URL: <https://www.fincen.gov/what-money-laundering>.
- [2] Federal Financial Institutions Examination Council. *BSA/AML Examination Manual: Introduction*. URL: <https://bsaaml.ffiec.gov/manual/Introduction/01>.
- [3] International Monetary Fund. *Money Laundering: The Importance of International Countermeasures*. Address by Michel Camdessus, Managing Director of the International Monetary Fund, to the Financial Action Task Force. 1998. URL: <https://www.imf.org/en/news/articles/2015/09/28/04/53/sp021098>.
- [4] United Nations Office on Drugs and Crime. *Estimating Illicit Financial Flows Resulting from Drug Trafficking and Other Transnational Organized Crimes*. Tech. rep. United Nations Office on Drugs and Crime, 2011. URL: https://www.unodc.org/documents/data-and-analysis/Studies/Illicit_financial_flows_2011_web.pdf.
- [5] World Bank. *GDP (Current US\$) – World*. World Development Indicators, indicator NY.GDP.MKTP.CD; 2025 observation. URL: <https://data.worldbank.org/indicator/NY.GDP.MKTP.CD?locations=1W>.
- [6] International Monetary Fund. *The IMF and the Fight Against Money Laundering and Terrorism Financing*. 2023. URL: <https://www.imf.org/en/About/Factsheets/Sheets/2023/Fight-against-money-laundering-and-terrorism-financing>.
- [7] V. Tanzi. *Money Laundering and the International Financial System*. IMF Working Paper 96/55. International Monetary Fund, 1996. URL: <https://www.elibrary.imf.org/view/journals/001/1996/055/article-A001-en.xml>.
- [8] Financial Action Task Force. *International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation: The FATF Recommendations*. Updated June 2026. 2012–2026. URL: <https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html>.

- [9] European Parliament and Council of the European Union. *Directive (EU) 2015/849 on the Prevention of the Use of the Financial System for the Purposes of Money Laundering or Terrorist Financing*. 2015. URL: <https://eur-lex.europa.eu/eli/dir/2015/849/oj/eng>.
- [10] Financial Crimes Enforcement Network. *The Bank Secrecy Act*. URL: <https://www.fincen.gov/resources/statutes-and-regulations/bank-secrecy-act>.
- [11] Financial Crimes Enforcement Network. *The Anti-Money Laundering Act of 2020*. 2021. URL: <https://www.fincen.gov/resources/statutes-and-regulations/anti-money-laundering-act-2020>.
- [12] Basel Committee on Banking Supervision. *Sound Management of Risks Related to Money Laundering and Financing of Terrorism*. Tech. rep. Originally published January 2014; revised July 2020. Bank for International Settlements, July 2020. URL: <https://www.bis.org/bcbs/publ/d505.pdf>.
- [13] M. Weber, J. Chen, T. Suzumura, A. Pareja, T. Ma, H. Kanezashi, T. Kaler, C. E. Leiserson, and T. B. Schardl. *Scalable Graph Learning for Anti-Money Laundering: A First Look*. 2018. arXiv: [1812.00076](https://arxiv.org/abs/1812.00076).
- [14] T. N. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *5th International Conference on Learning Representations*. 2017. URL: <https://openreview.net/forum?id=SJU4ayYgl>.
- [15] J. Chen, T. Ma, and C. Xiao. “FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling”. In: *International Conference on Learning Representations*. 2018. URL: <https://arxiv.org/abs/1801.10247>.
- [16] M. R. Karim, F. Hermesen, S. A. Chala, P. De Perthuis, and A. Mandal. “Scalable Semi-Supervised Graph Learning Techniques for Anti Money Laundering”. In: *IEEE Access* 12 (2024), pp. 50012–50029. DOI: [10.1109/ACCESS.2024.3383784](https://doi.org/10.1109/ACCESS.2024.3383784).
- [17] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson. *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*. 2019. arXiv: [1908.02591](https://arxiv.org/abs/1908.02591) [cs.SI].
- [18] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson. “Benchmarking Graph Neural Networks”. In: *Journal of Machine Learning Research* 24.43 (2023), pp. 1–48. URL: <https://jmlr.org/papers/v24/22-0567.html>.

-
- [19] Financial Action Task Force. *Stocktake on Data Pooling, Collaborative Analytics and Data Protection*. Tech. rep. FATF/OECD, July 2021. URL: <https://www.fatf-gafi.org/content/dam/fatf/documents/Stocktake-Datapooling-Collaborative-Analytics.pdf>.
- [20] Z. Tian, Y. Ding, X. Yu, E. Gong, J. Liu, and K. Ren. “Towards Collaborative Anti-Money Laundering Among Financial Institutions”. In: *Proceedings of the ACM Web Conference 2025*. 2025, pp. 4722–4733. DOI: [10.1145/3696410.3714576](https://doi.org/10.1145/3696410.3714576).
- [21] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>.
- [22] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. “Communication-Efficient Learning of Deep Networks from Decentralized Data”. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Vol. 54. Proceedings of Machine Learning Research. 2017, pp. 1273–1282. URL: <https://proceedings.mlr.press/v54/mcmahan17a.html>.
- [23] Q. Yang, Y. Liu, T. Chen, and Y. Tong. “Federated Machine Learning: Concept and Applications”. In: *ACM Transactions on Intelligent Systems and Technology* 10.2, 12 (2019), pp. 1–19. DOI: [10.1145/3298981](https://doi.org/10.1145/3298981).
- [24] Y. Yao, W. Jin, S. Ravi, and C. Joe-Wong. “FedGCN: Convergence-Communication Tradeoffs in Federated Training of Graph Convolutional Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc., 2023, pp. 79748–79760. DOI: [10.52202/075280-3491](https://doi.org/10.52202/075280-3491).
- [25] Q. Li, Y. Diao, Q. Chen, and B. He. “Federated Learning on Non-IID Data Silos: An Experimental Study”. In: *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 965–978. DOI: [10.1109/ICDE53745.2022.00077](https://doi.org/10.1109/ICDE53745.2022.00077).
- [26] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov. “Exploiting Unintended Feature Leakage in Collaborative Learning”. In: *2019 IEEE Symposium on Security and Privacy*. IEEE, 2019, pp. 691–706. DOI: [10.1109/SP.2019.00029](https://doi.org/10.1109/SP.2019.00029).
- [27] T. Suzumura and H. Kanezashi. *AMLSim: Anti-Money Laundering Simulator*. GitHub repository, inspected upstream commit 7338a4bc b1af9bcfea2201ad7daccfe2a4d569ca. 2021. URL: <https://github.com/IBM/AMLSim>.

- [28] Y. Yao. *FedGCN Software Repository*. GitHub repository, commit 378438d0a5dbfa0f1cf859a3d6f3aaa4cff232a3. 2025. URL: <https://github.com/yh-yao/FedGCN>.
- [29] Y. Yao, Y. Li, X. Fan, J. Li, K. Liu, W. Jin, S. Ravi, P. S. Yu, and C. Joe-Wong. *FedGraph: A Research Library and Benchmark for Federated Graph Learning*. Presented at the non-archival NeurIPS 2025 Workshop on New Perspectives in Graph Machine Learning. 2024. arXiv: [2410.06340v4](https://arxiv.org/abs/2410.06340).
- [30] FedGraph Contributors. *FedGraph Software Repository*. GitHub repository, commit 9bea6d942c98ea7bc31a7e2de0ede2583f839cfd. 2025. URL: <https://github.com/FedGraph/fedgraph>.
- [31] J. H. Cheon, A. Kim, M. Kim, and Y. Song. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *Advances in Cryptology – ASIACRYPT 2017*. Vol. 10624. Lecture Notes in Computer Science. Springer, 2017, pp. 409–437. DOI: [10.1007/978-3-319-70694-8_15](https://doi.org/10.1007/978-3-319-70694-8_15).
- [32] Financial Crimes Enforcement Network. *Reporting Suspicious Activity: A Quick Reference Guide for Money Services Businesses*. URL: https://www.fincen.gov/system/files/shared/report_reference.pdf.
- [33] Financial Crimes Enforcement Network. *Frequently Asked Questions Regarding the FinCEN Suspicious Activity Report (SAR)*. URL: <https://www.fincen.gov/resources/frequently-asked-questions-regarding-fincen-suspicious-activity-report-sar>.
- [34] R. Wu, B. Ma, H. Jin, W. Zhao, W. Wang, and T. Zhang. “GRANDE: A Neural Model over Directed Multigraphs with Application to Anti-Money Laundering”. In: *2022 IEEE International Conference on Data Mining (ICDM)*. 2022, pp. 558–567. DOI: [10.1109/ICDM54844.2022.00066](https://doi.org/10.1109/ICDM54844.2022.00066).
- [35] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. “The Graph Neural Network Model”. In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80. DOI: [10.1109/TNN.2008.2005605](https://doi.org/10.1109/TNN.2008.2005605).
- [36] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun. “Graph Neural Networks: A Review of Methods and Applications”. In: *AI Open* 1 (2020), pp. 57–81. DOI: [10.1016/j.aiopen.2021.01.001](https://doi.org/10.1016/j.aiopen.2021.01.001).
- [37] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry”. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1263–1272. URL: <https://proceedings.mlr.press/v70/gilmer17a.html>.

-
- [38] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar. “A Review of Graph Neural Networks: Concepts, Architectures, Techniques, Challenges, Datasets, Applications, and Future Directions”. In: *Journal of Big Data* 11.1, 18 (2024). DOI: [10.1186/s40537-023-00876-4](https://doi.org/10.1186/s40537-023-00876-4).
- [39] J. Chen, J. Zhu, and L. Song. “Stochastic Training of Graph Convolutional Networks with Variance Reduction”. In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 942–950. URL: <https://proceedings.mlr.press/v80/chen18p.html>.
- [40] M. Fey and J. E. Lenssen. “Fast Graph Representation Learning with PyTorch Geometric”. In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. 2019. arXiv: [1903.02428](https://arxiv.org/abs/1903.02428).
- [41] P. Kairouz, H. B. McMahan, B. Avent, et al. “Advances and Open Problems in Federated Learning”. In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pp. 1–210. DOI: [10.1561/22000000083](https://doi.org/10.1561/22000000083).
- [42] X. Fu, B. Zhang, Y. Dong, C. Chen, and J. Li. “Federated Graph Machine Learning: A Survey of Concepts, Techniques, and Applications”. In: *ACM SIGKDD Explorations Newsletter* 24.2 (2022), pp. 32–47. DOI: [10.1145/3575637.3575644](https://doi.org/10.1145/3575637.3575644).
- [43] O. Goldreich. *Foundations of Cryptography, Volume 2: Basic Applications*. Cambridge University Press, 2004. URL: <https://www.wisdom.weizmann.ac.il/~oded/foc-vol2.html>.
- [44] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller. “Inverting Gradients - How easy is it to break privacy in federated learning?” In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 16937–16947. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/c4ede56bbd98819ae6112b20ac6bf145-Paper.pdf.
- [45] P. Paillier. “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes”. In: *Advances in Cryptology – EUROCRYPT ’99*. Vol. 1592. Lecture Notes in Computer Science. Springer, 1999, pp. 223–238. DOI: [10.1007/3-540-48910-X_16](https://doi.org/10.1007/3-540-48910-X_16).
- [46] J. Fan and F. Vercauteren. *Somewhat Practical Fully Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2012/144. 2012. URL: <https://eprint.iacr.org/2012/144>.
- [47] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. “(Leveled) Fully Homomorphic Encryption without Bootstrapping”. In: *ACM Transactions on Computation Theory* 6.3, 13 (2014), pp. 1–36. DOI: [10.1145/2633600](https://doi.org/10.1145/2633600).

- [48] Microsoft Research. *Microsoft SEAL: Encoder Examples, Version 4.1.2*. Official source documentation. URL: https://github.com/microsoft/SEAL/blob/v4.1.2/native/examples/2_encoders.cpp.
- [49] S. P. Kasiviswanathan, K. Nissim, S. Raskhodnikova, and A. Smith. “Analyzing Graphs with Node Differential Privacy”. In: *Theory of Cryptography: 10th Theory of Cryptography Conference*. Vol. 7785. Lecture Notes in Computer Science. Springer, 2013, pp. 457–476. DOI: [10.1007/978-3-642-36594-2_26](https://doi.org/10.1007/978-3-642-36594-2_26).
- [50] C. Dwork and A. Roth. “The Algorithmic Foundations of Differential Privacy”. In: *Foundations and Trends in Theoretical Computer Science* 9.3–4 (2014), pp. 211–407. DOI: [10.1561/04000000042](https://doi.org/10.1561/04000000042).
- [51] B. Balle and Y.-X. Wang. “Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising”. In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. 2018, pp. 394–403. URL: <https://proceedings.mlr.press/v80/balle18a.html>.
- [52] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth. “Practical Secure Aggregation for Privacy-Preserving Machine Learning”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2017, pp. 1175–1191. DOI: [10.1145/3133956.3133982](https://doi.org/10.1145/3133956.3133982).
- [53] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu. “Realistic Synthetic Financial Transactions for Anti-Money Laundering Models”. In: *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc., 2023, pp. 29851–29874. DOI: [10.52202/075280-1300](https://doi.org/10.52202/075280-1300).
- [54] B. Egressy, L. von Niederhäusern, J. Blanuša, E. Altman, R. Wattenhofer, and K. Atasu. “Provably Powerful Graph Neural Networks for Directed Multigraphs”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 10. 2024, pp. 11838–11846. DOI: [10.1609/aaai.v38i10.29069](https://doi.org/10.1609/aaai.v38i10.29069).
- [55] Z. Guo, W. Wu, C. Chen, L. Zhang, O. Kotevska, and R. K. Madhuri. *Provably Communication-Efficient and Privacy-Preserving Federated Graph Neural Networks*. 2026. arXiv: [2605.26243](https://arxiv.org/abs/2605.26243).
- [56] M. van den Beukel, J. M. Rožanec, and A.-L. Varbanescu. *Tide: A Customisable Dataset Generator for Anti-Money Laundering Research*. 2026. arXiv: [2603.01863](https://arxiv.org/abs/2603.01863).
- [57] K. Chen et al. “TransXion: A High-Fidelity Graph Benchmark for Realistic Anti-Money Laundering”. In: *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2026, pp. 8707–8718. DOI: [10.1145/3770855.3817550](https://doi.org/10.1145/3770855.3817550).

-
- [58] R. I. T. Jensen, J. Ferwerda, K. S. Jørgensen, E. R. Jensen, M. Borg, M. P. Krogh, J. B. Jensen, and A. Iosifidis. “A Synthetic Data Set to Benchmark Anti-Money Laundering Methods”. In: *Scientific Data* 10.1, 661 (2023). DOI: [10.1038/s41597-023-02569-2](https://doi.org/10.1038/s41597-023-02569-2).
 - [59] D. Commey, M. Nkoom, Y. Alsenani, S. G. Hounsinnou, and G. V. Crosby. *FedGraph-VASP: Privacy-Preserving Federated Graph Learning with Post-Quantum Security for Cross-Institutional Anti-Money Laundering*. 2026. arXiv: [2601.17935](https://arxiv.org/abs/2601.17935).
 - [60] C. Bellei, M. Xu, R. Phillips, T. Robinson, M. Weber, T. Kaler, C. E. Leiserson, Arvind, and J. Chen. “The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset”. In: *KDD Workshop on Machine Learning in Finance*. 2024. arXiv: [2404.19109](https://arxiv.org/abs/2404.19109).
 - [61] D. Cheng, Y. Ye, S. Xiang, Z. Ma, Y. Zhang, and C. Jiang. “Anti-Money Laundering by Group-Aware Deep Graph Learning”. In: *IEEE Transactions on Knowledge and Data Engineering* 35.12 (2023), pp. 12444–12457. DOI: [10.1109/TKDE.2023.3272396](https://doi.org/10.1109/TKDE.2023.3272396).
 - [62] M. Cardoso, P. Saleiro, and P. Bizarro. “LaundroGraph: Self-Supervised Graph Representation Learning for Anti-Money Laundering”. In: *Proceedings of the 3rd ACM International Conference on AI in Finance*. 2022, pp. 130–138. DOI: [10.1145/3533271.3561727](https://doi.org/10.1145/3533271.3561727).
 - [63] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu. “Subgraph Federated Learning with Missing Neighbor Generation”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 6671–6682. URL: <https://proceedings.neurips.cc/paper/2021/hash/34adeb8e3242824038aa65460a47c29e-Abstract.html>.
 - [64] P. Naman and Y. Simmhan. “OptimES: Optimizing Federated Learning Using Remote Embeddings for Graph Neural Networks”. In: *Journal of Parallel and Distributed Computing* 211, 105227 (2026). DOI: [10.1016/j.jpdc.2026.105227](https://doi.org/10.1016/j.jpdc.2026.105227).
 - [65] Z. Tian, Y. Ding, R. Zhang, Y. Tang, J. Liu, and K. Ren. “Toward Federated Learning of Deep Graph Neural Networks”. In: *IEEE Transactions on Knowledge and Data Engineering* 38.3 (2026), pp. 2028–2039. DOI: [10.1109/TKDE.2026.3652029](https://doi.org/10.1109/TKDE.2026.3652029).
 - [66] X. Ni, X. Xu, L. Lyu, C. Meng, and W. Wang. *A Vertical Federated Learning Framework for Graph Convolutional Network*. 2021. arXiv: [2106.11593](https://arxiv.org/abs/2106.11593).
 - [67] R. Ran, W. Wang, G. Quan, J. Yin, N. Xu, and W. Wen. “CryptoGCN: Fast and Scalable Homomorphically Encrypted Graph Convolutional Network Inference”. In: *Advances in Neural Information Processing Systems*. Vol. 35. Curran Associates, Inc., 2022, pp. 37676–37689. DOI: [10.52202/068431-2731](https://doi.org/10.52202/068431-2731).

- [68] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, and D. Gatica-Perez. “GAP: Differentially Private Graph Neural Networks with Aggregation Perturbation”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2023, pp. 3223–3240. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/sajadmanesh>.
- [69] T. Suzumura et al. *Towards Federated Graph Learning for Collaborative Financial Crimes Detection*. 2019. arXiv: [1909.12946](https://arxiv.org/abs/1909.12946).
- [70] M. B. van Egmond, V. Dunning, S. van den Berg, T. Rooijakkers, A. Sangers, T. Poppe, and J. Veldsink. “Privacy-Preserving Anti-Money Laundering Using Secure Multi-Party Computation”. In: *Financial Cryptography and Data Security: 28th International Conference, FC 2024, Revised Selected Papers, Part II*. Vol. 14745. Lecture Notes in Computer Science. Springer, 2025, pp. 331–349. DOI: [10.1007/978-3-031-78679-2_18](https://doi.org/10.1007/978-3-031-78679-2_18).
- [71] BIS Innovation Hub. *Project Aurora: The Power of Data, Technology and Collaboration to Combat Money Laundering Across Institutions and Borders*. Tech. rep. Bank for International Settlements, 2023. URL: <https://www.bis.org/publ/othp66.htm>.
- [72] R. Kanagavelu, M. Nepal, P. Ning, K. Cai, J. Xu, F. Gao, Y. Liu, S. M. R. Goh, and Q. Wei. *DPxFin: Adaptive Differential Privacy for Anti-Money Laundering Detection via Reputation-Weighted Federated Learning*. 2026. arXiv: [2603.19314](https://arxiv.org/abs/2603.19314).
- [73] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. “Efficient Back-Prop”. In: *Neural Networks: Tricks of the Trade*. Ed. by G. B. Orr and K.-R. Müller. Vol. 1524. Lecture Notes in Computer Science. Springer, 1998, pp. 9–50. DOI: [10.1007/3-540-49430-8_2](https://doi.org/10.1007/3-540-49430-8_2).
- [74] S. Kapoor and A. Narayanan. “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science”. In: *Patterns* 4.9, 100804 (2023). DOI: [10.1016/j.patter.2023.100804](https://doi.org/10.1016/j.patter.2023.100804).
- [75] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56 (2014), pp. 1929–1958. URL: <https://jmlr.org/papers/v15/srivastava14a.html>.
- [76] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [77] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization”. In: *3rd International Conference on Learning Representations*. 2015. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980).

-
- [78] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal. *TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption*. 2021. arXiv: [2104.03152](https://arxiv.org/abs/2104.03152).
 - [79] scikit-learn Developers. *scikit-learn: average_precision_score Documentation*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html.
 - [80] T. Saito and M. Rehmsmeier. “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets”. In: *PLOS ONE* 10.3, e0118432 (2015). DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432).
 - [81] D. Chicco and G. Jurman. “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation”. In: *BMC Genomics* 21.1, 6 (2020). DOI: [10.1186/s12864-019-6413-7](https://doi.org/10.1186/s12864-019-6413-7).
 - [82] G. C. Cawley and N. L. C. Talbot. “On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation”. In: *Journal of Machine Learning Research* 11.70 (2010), pp. 2079–2107. URL: <https://www.jmlr.org/papers/v11/cawley10a.html>.
 - [83] A. Alshantti and A. Rasheed. “Self-Organising Map Based Framework for Investigating Accounts Suspected of Money Laundering”. In: *Frontiers in Artificial Intelligence* 4, 761925 (2021). DOI: [10.3389/frai.2021.761925](https://doi.org/10.3389/frai.2021.761925).
 - [84] RWTH Aachen University IT Center. *CLAIX-2023 Compute Nodes*. Updated April 16, 2026. 2026. URL: <https://help.itc.rwth-aachen.de/en/service/rhr4fjjutttf/article/fbd107191cf14c4b8307f44f545cf68a/>.
 - [85] J. P. Near, D. Darais, N. Lefkovitz, and G. S. Howarth. *Guidelines for Evaluating Differential Privacy Guarantees*. NIST Special Publication 800-226. National Institute of Standards and Technology, 2025. DOI: [10.6028/NIST.SP.800-226](https://doi.org/10.6028/NIST.SP.800-226).
 - [86] J. Baek, W. Jeong, J. Jin, J. Yoon, and S. J. Hwang. “Personalized Subgraph Federated Learning”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. 2023, pp. 1396–1415. URL: <https://proceedings.mlr.press/v202/baek23a.html>.
 - [87] Z. Liu, S. Chen, J. Ye, J. Fan, H. Li, and X. Li. “DHSA: Efficient Doubly Homomorphic Secure Aggregation for Cross-Silo Federated Learning”. In: *The Journal of Supercomputing* 79.3 (2023), pp. 2819–2849. DOI: [10.1007/s11227-022-04745-4](https://doi.org/10.1007/s11227-022-04745-4).
 - [88] European Parliament and Council of the European Union. *Regulation (EU) 2024/1624 on the Prevention of the Use of the Financial System for the Purposes of Money Laundering or Terrorist Financing*. 2024. URL: <https://eur-lex.europa.eu/eli/reg/2024/1624/oj/eng>.

License

This thesis contains others' intellectual and creative property which has been cited or marked appropriately. The original creators may have copyrighted or published their material under a different licence. All residual work, including figures and the typography layout, are hereby declared subject to the “Creative Commons Attribution 4.0 International” licence.



<https://creativecommons.org/licenses/by/4.0/legalcode>